

Lab2

Presenter : 鄭祺家

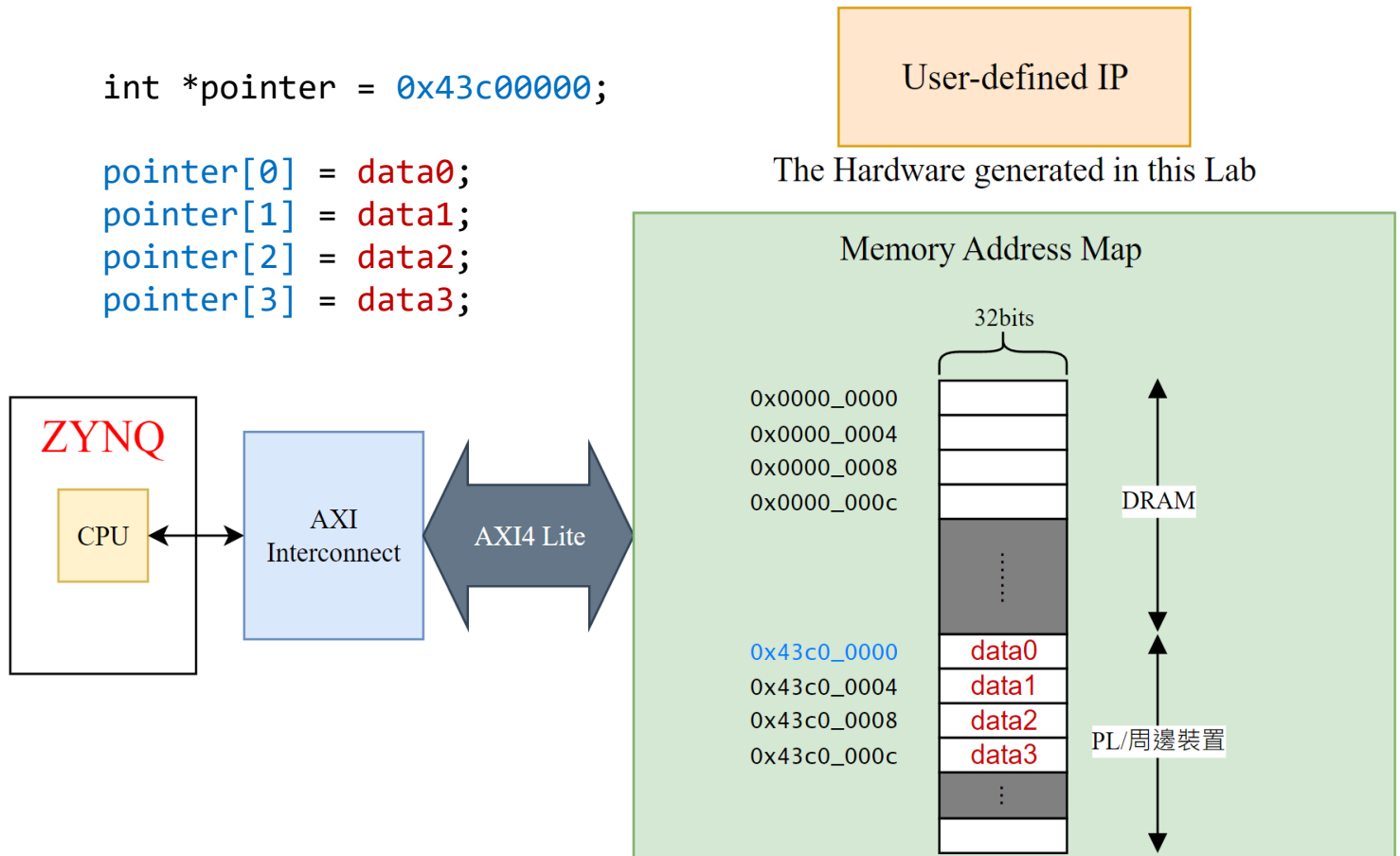
Date : 2026/04/17

Agenda

- ▶ Part 1 : 建立 AXI slave IP 並使用 AXI-Lite 協定溝通
- ▶ Part 2 : 建立 AXI Master 並與 CPU 進行溝通
- ▶ Part 3 : 練習題

Communication between ZYNQ and Hardware IP

- ▶ ZedBoard內部的暫存器可以被生成如右圖的**記憶體位址**。每四個address位置儲存32bits的資料。
- ▶ ZedBoard中的CPU(ZYNQ)會將收到的**記憶體位置**與**資料**透過AXI Bus傳送到FPGA。
- ▶ 我們這次Lab生成的硬體也會分配到**記憶體位置**(通常是從0x43c0_0000起始)，起始位址可以在開啟SDK後查詢。
- ▶ 在SDK中，我們可以撰寫C語言使CPU直接對該**記憶體位址**進行讀寫。



Communication between ZYNQ and Hardware IP

- ▶ 我們可以使用int指標指到硬體生成的起始位置(0x43c0_0000)。
- ▶ 指標的**位置**與**資料**會經過AXI-Lite協定進行控制。

```
//save the value to slave register
```

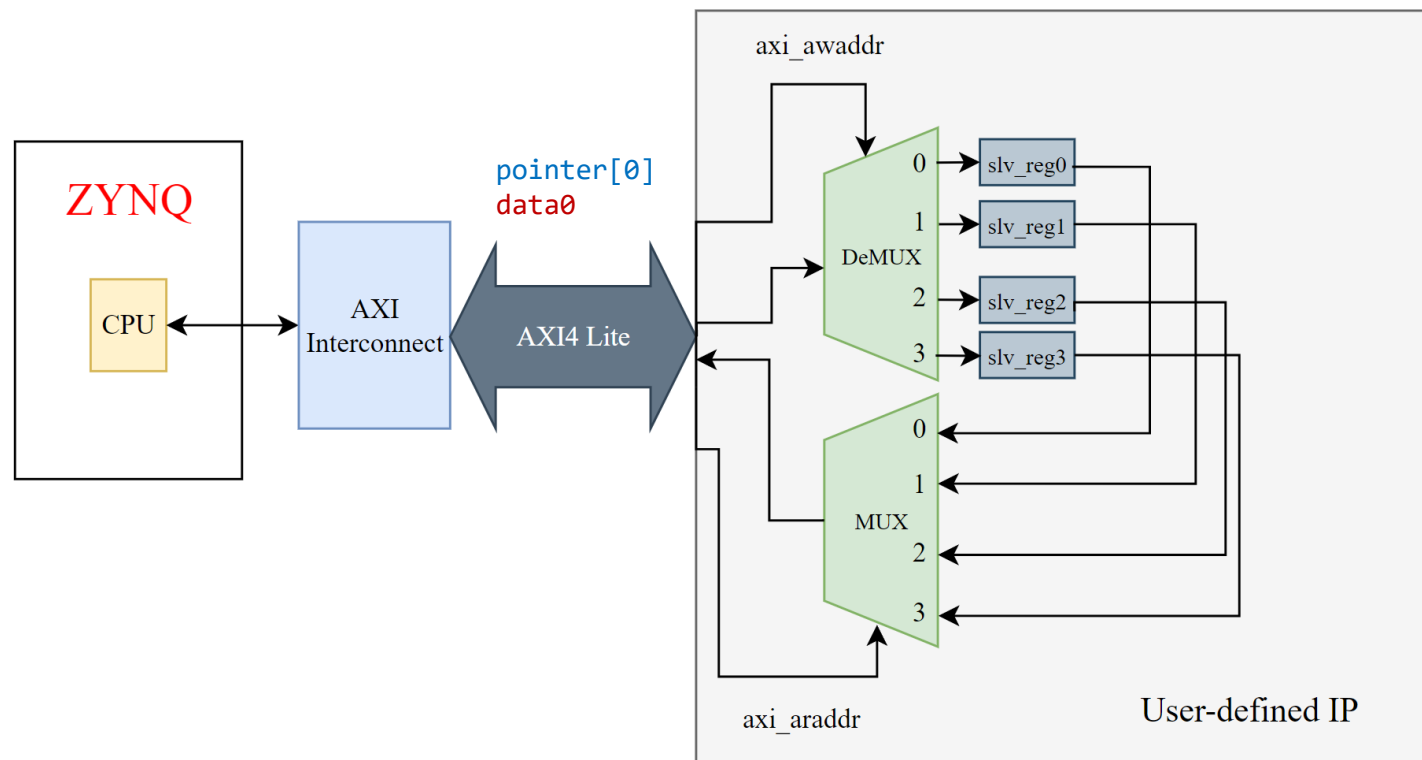
```
int *pointer = 0x43c00000;
```

```
pointer[0] = data0;
```

```
pointer[1] = data1;
```

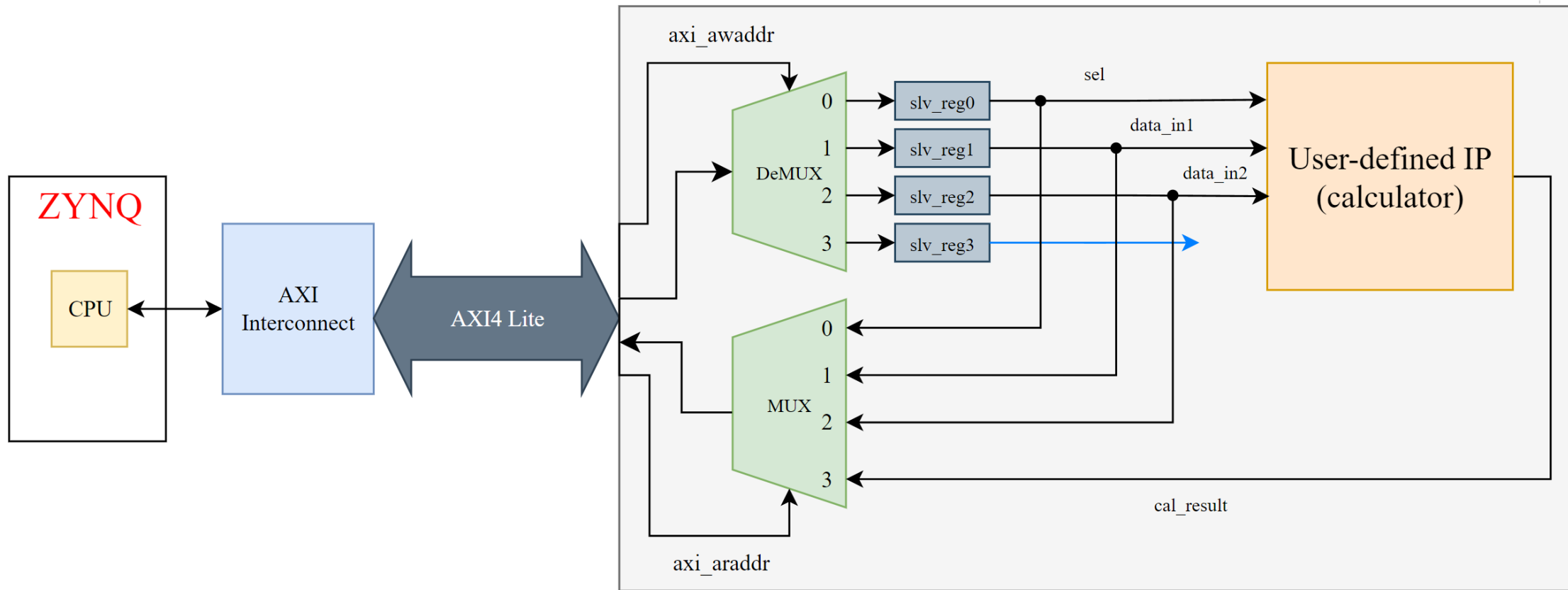
```
pointer[2] = data2;
```

```
pointer[3] = data3;
```



Communication between ZYNQ and Hardware IP

- 修改slave內部，增加一個calculator的硬體，並學習於SDK平台上撰寫C語言控制硬體。

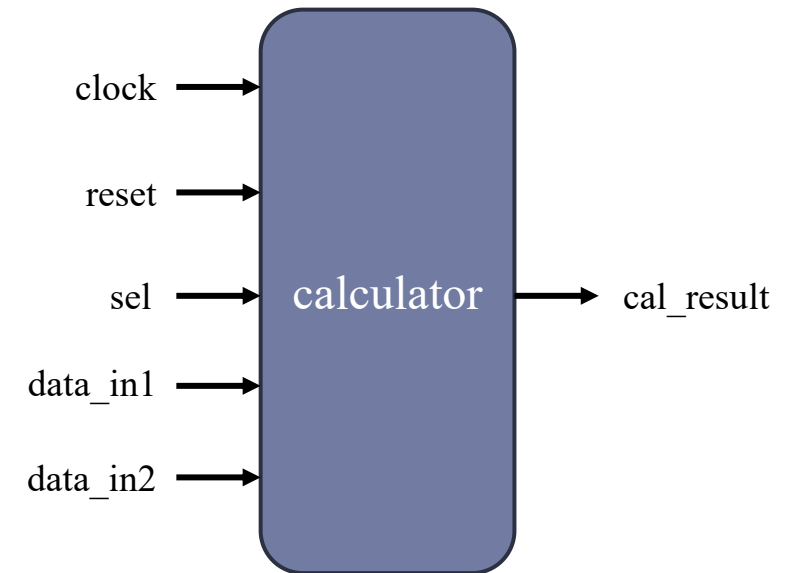


修改後的slave模板

Slave IP Description

- ▶ 設計一個calculator，可以支援加法，減法與乘法的運算。
 - ▶ 建立一個擁有 8 個 register 的 slave IP
 - ▶ 將data_in1 與 data_in2 與 sel 訊號透過 CPU 傳給 slave IP 的三個 registers
 - ▶ 使用 CPU 讀出兩個值運算後的結果

Ports Name	Input/Output	Bit width	Description
clk	Input	1	Clock signal of IP.
reset	Input	1	Reset signal of IP.
data_in1	Input	16	Input data for calculation.
data_in2	Input	16	Input data for calculation.
sel	Input	3	Select signal for which operation 000 : ADD 001 : SUB 010 : MUL
cal_result	Output	32	Output for the calculation.

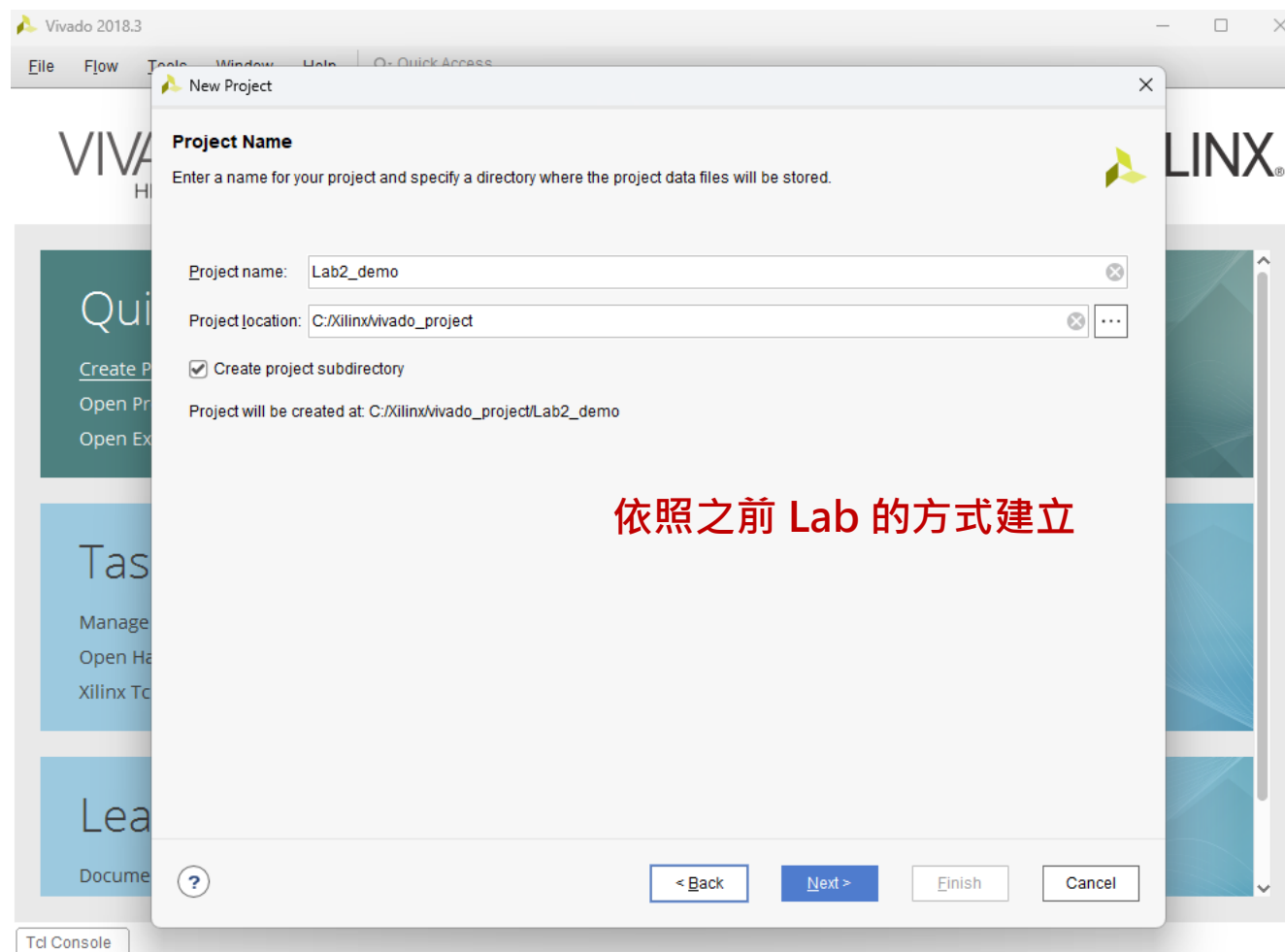


Part 1.1

- ▶ 建立 slave IP
- ▶ 編輯 slave IP 內容
- ▶ 建立 Hardware IP
- ▶ 加入 ILA debug core

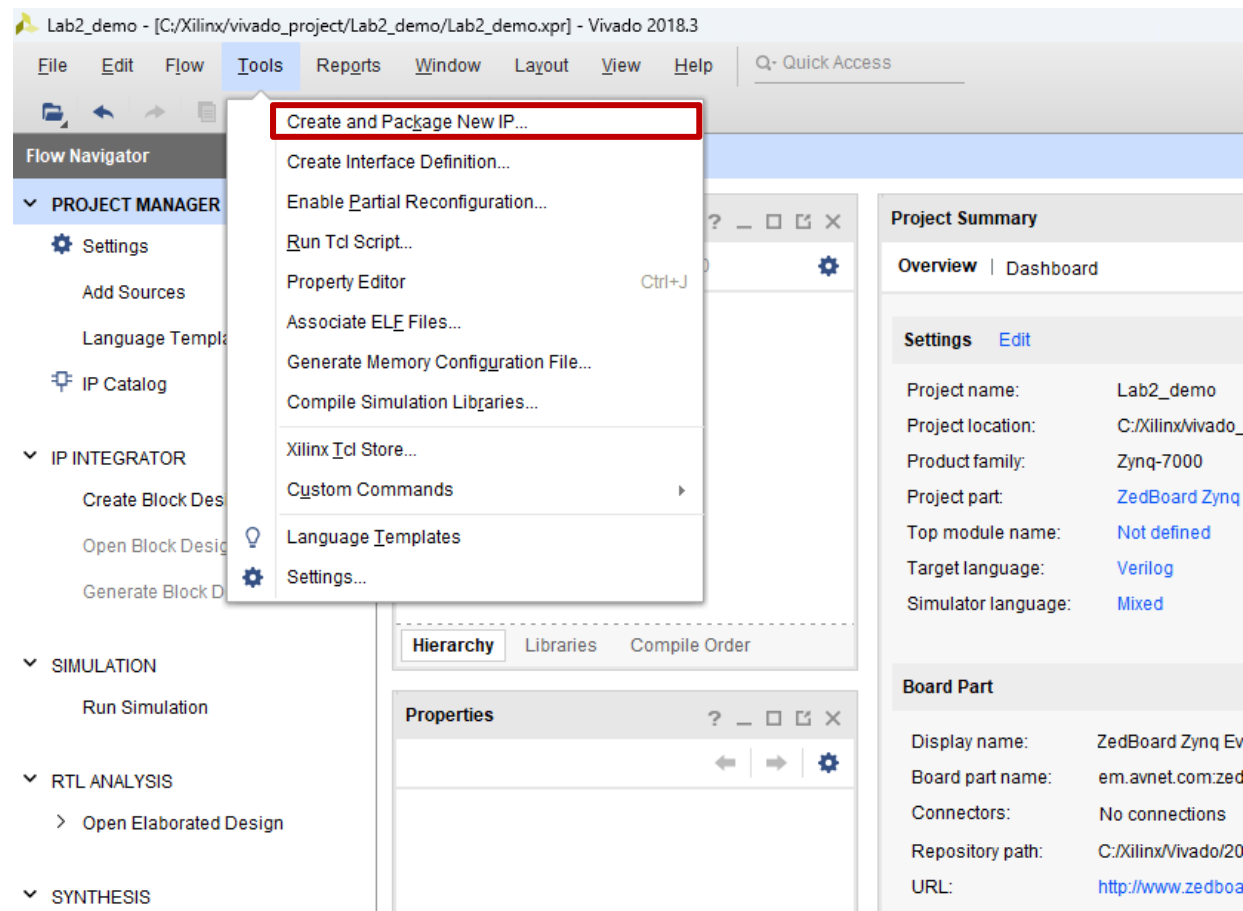
建立slave IP

▶ 1.先建立一個新的專案



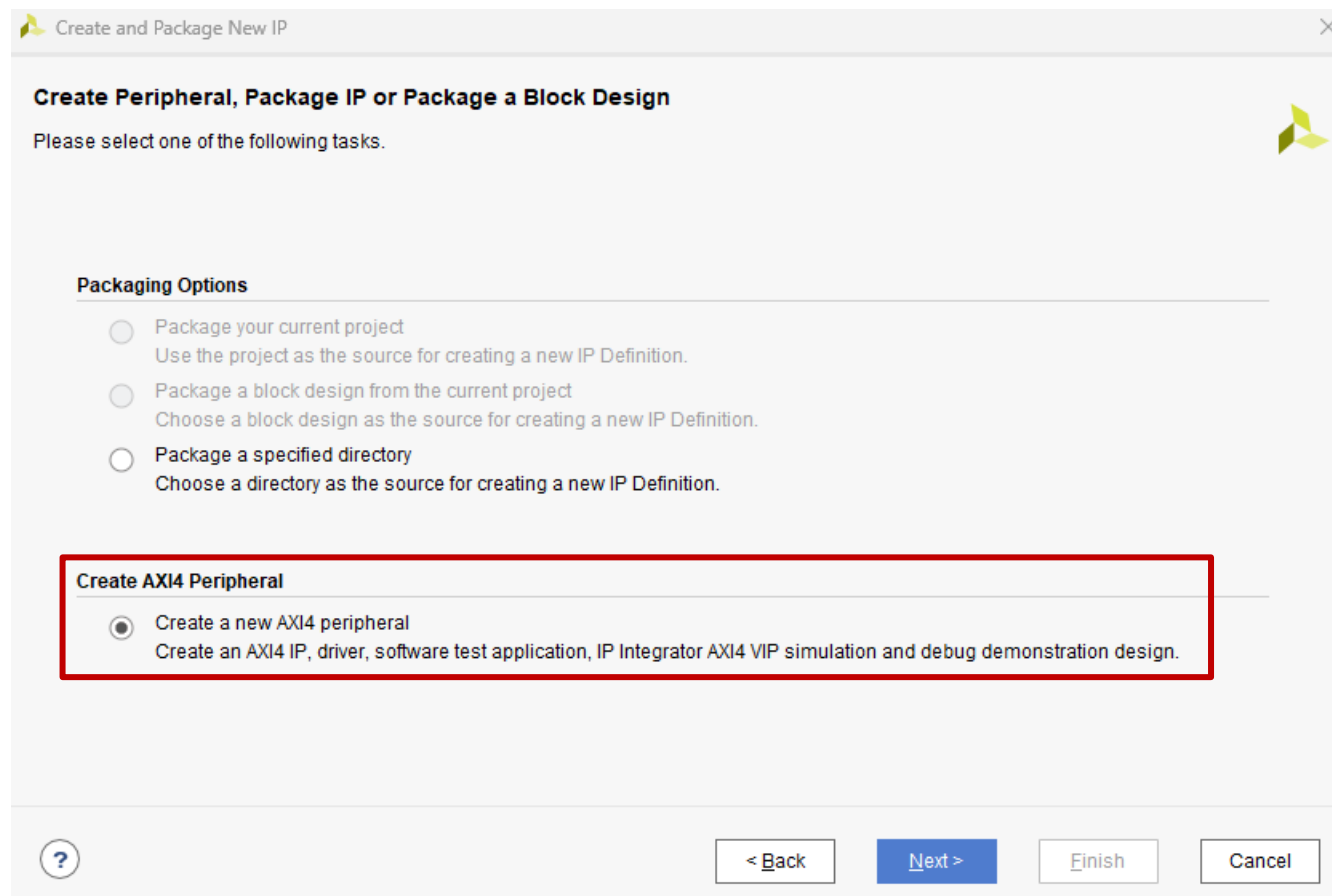
建立slave IP

- ▶ 1.先建立一個新的專案
- ▶ 2.Tools → Create and Package New IP



建立slave IP

- ▶ 1.先建立一個新的專案
- ▶ 2.Tools → Create and Package New IP
- ▶ 3.Next → Create a new AXI4 peripheral



建立slave IP

- ▶ 1.先建立一個新的專案
- ▶ 2.Tools → Create and Package New IP
- ▶ 3.Next → Create a new AXI4 peripheral
- ▶ 4.修改 Name → Next

Create and Package New IP

Peripheral Details
Specify name, version and description for the new peripheral

1.

Name: Lab2_IP

Version: 1.0

Display name: Lab2_IP_v1.0

Description: My new AXI IP

IP location: C:/Xilinx/Vivado_project/Lab2_demo/./ip_repo

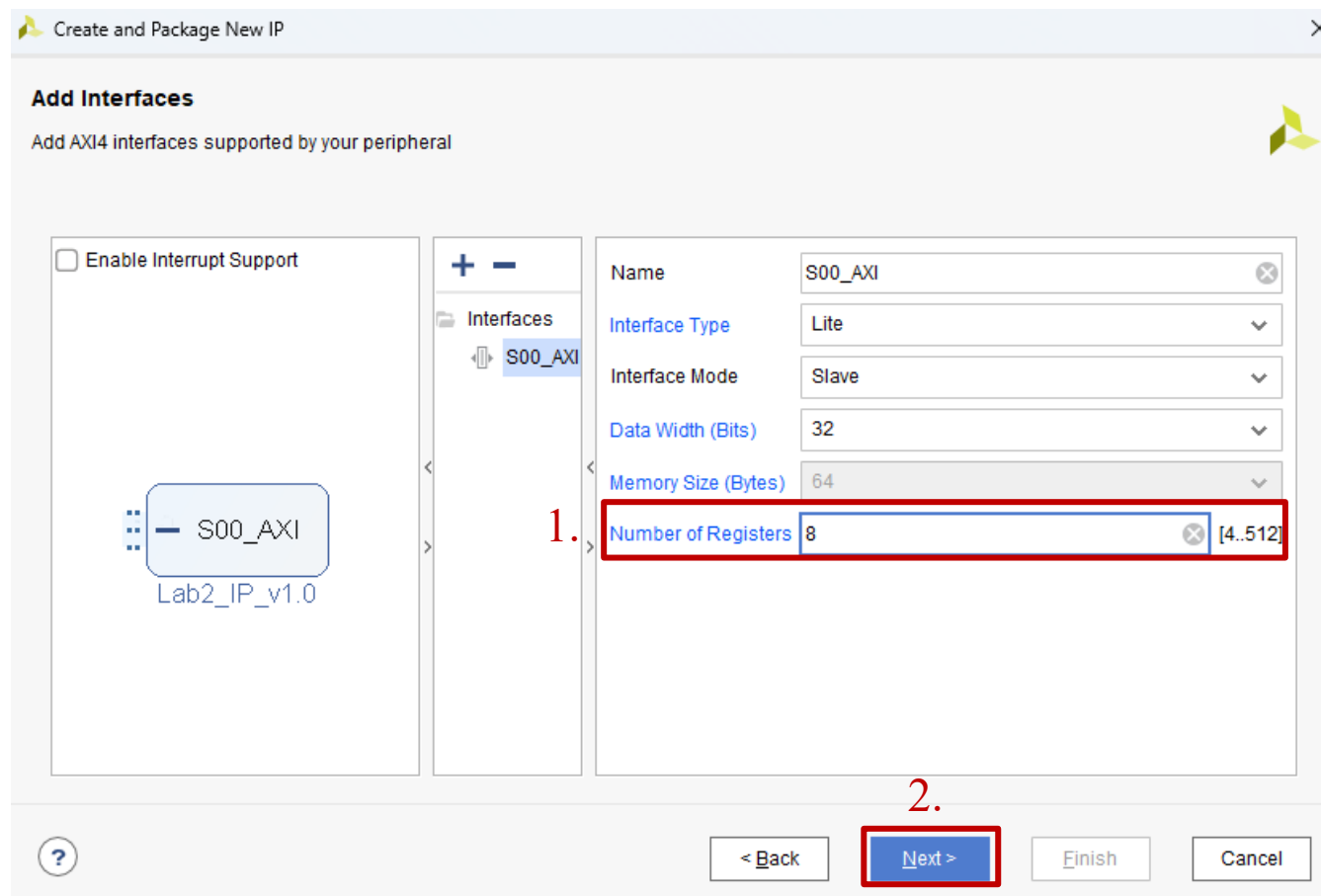
☐ Overwrite existing

2.

< Back Next > Finish Cancel

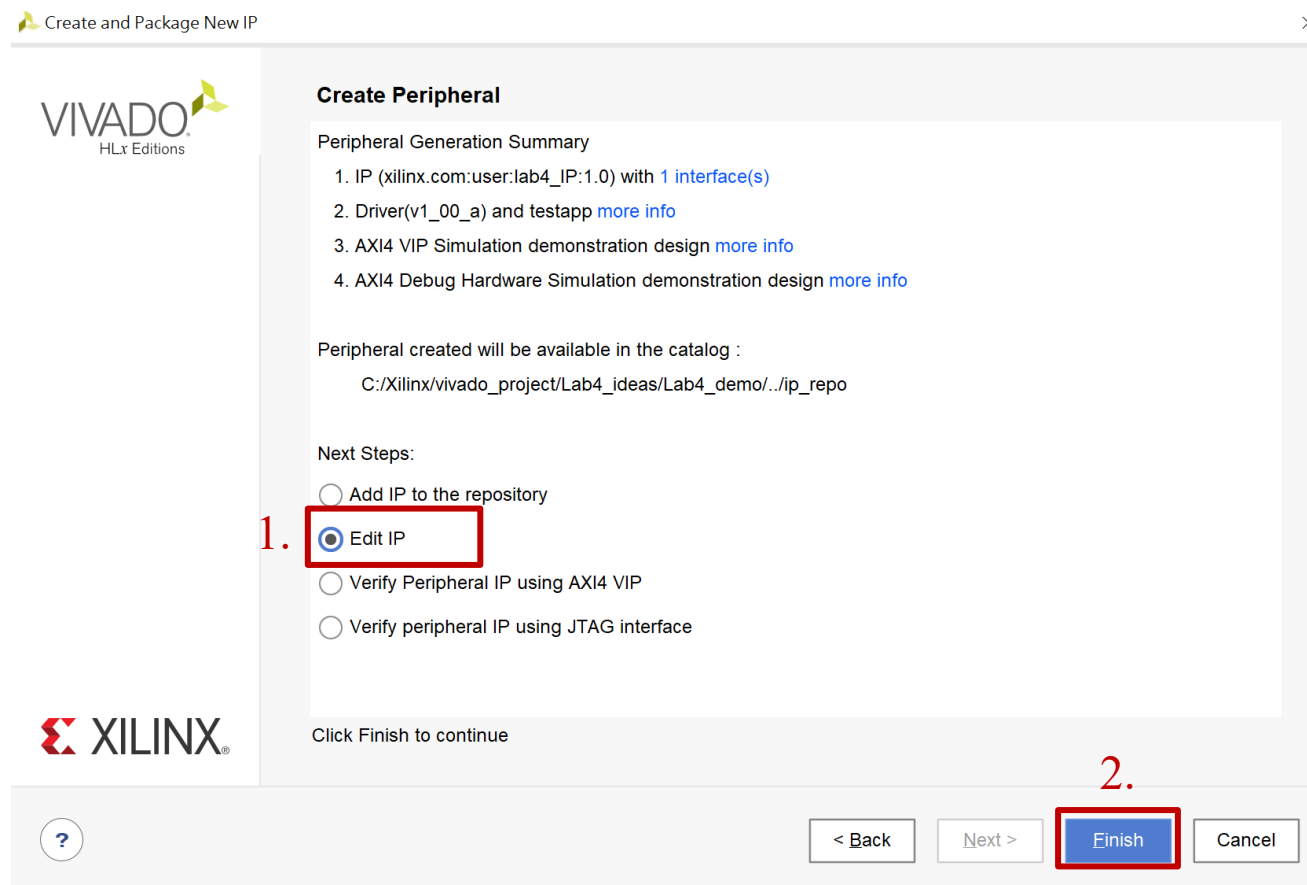
建立slave IP

- ▶ 1.先建立一個新的專案
- ▶ 2.Tools → Create and Package New IP
- ▶ 3.Next → Create a new AXI4 peripheral
- ▶ 4.修改 Name → Next
- ▶ 5.建立擁有 8 個 register 的 slave IP → Next



建立slave IP

- ▶ 1.先建立一個新的專案
- ▶ 2.Tools → Create and Package New IP
- ▶ 3.Next → Create a new AXI4 peripheral
- ▶ 4.修改 Name → Next
- ▶ 5.建立擁有 8 個 register 的 slave IP → Next
- ▶ 6.Edit IP → Finish

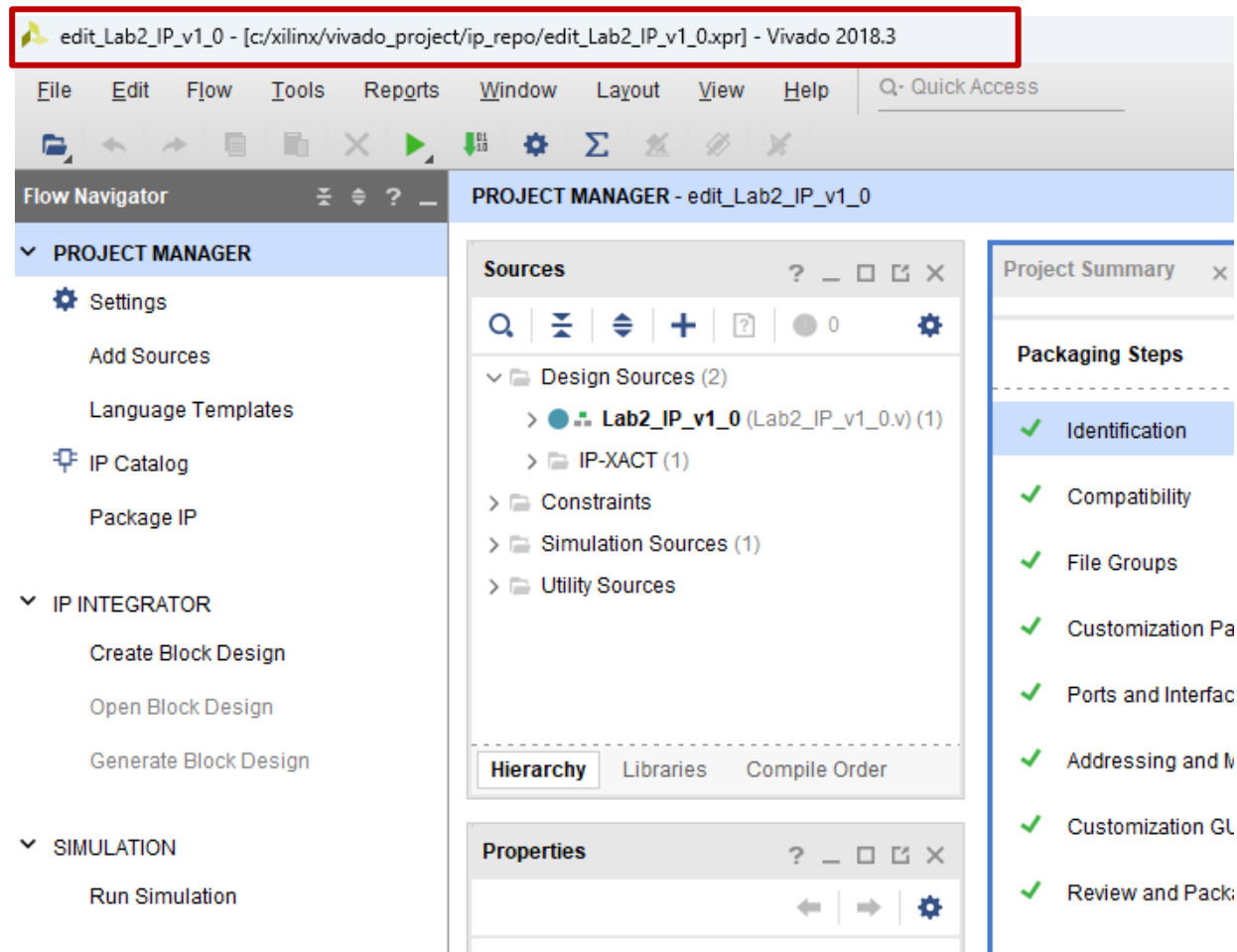


Part 1.1

- ▶ 建立 slave IP
- ▶ 編輯 slave IP 內容
- ▶ 建立 Hardware IP
- ▶ 加入 ILA debug core

編輯 slave IP 內容

- ▶ 1.確認開啟 edit_Lab2_IP 的介面 (編輯slave IP)

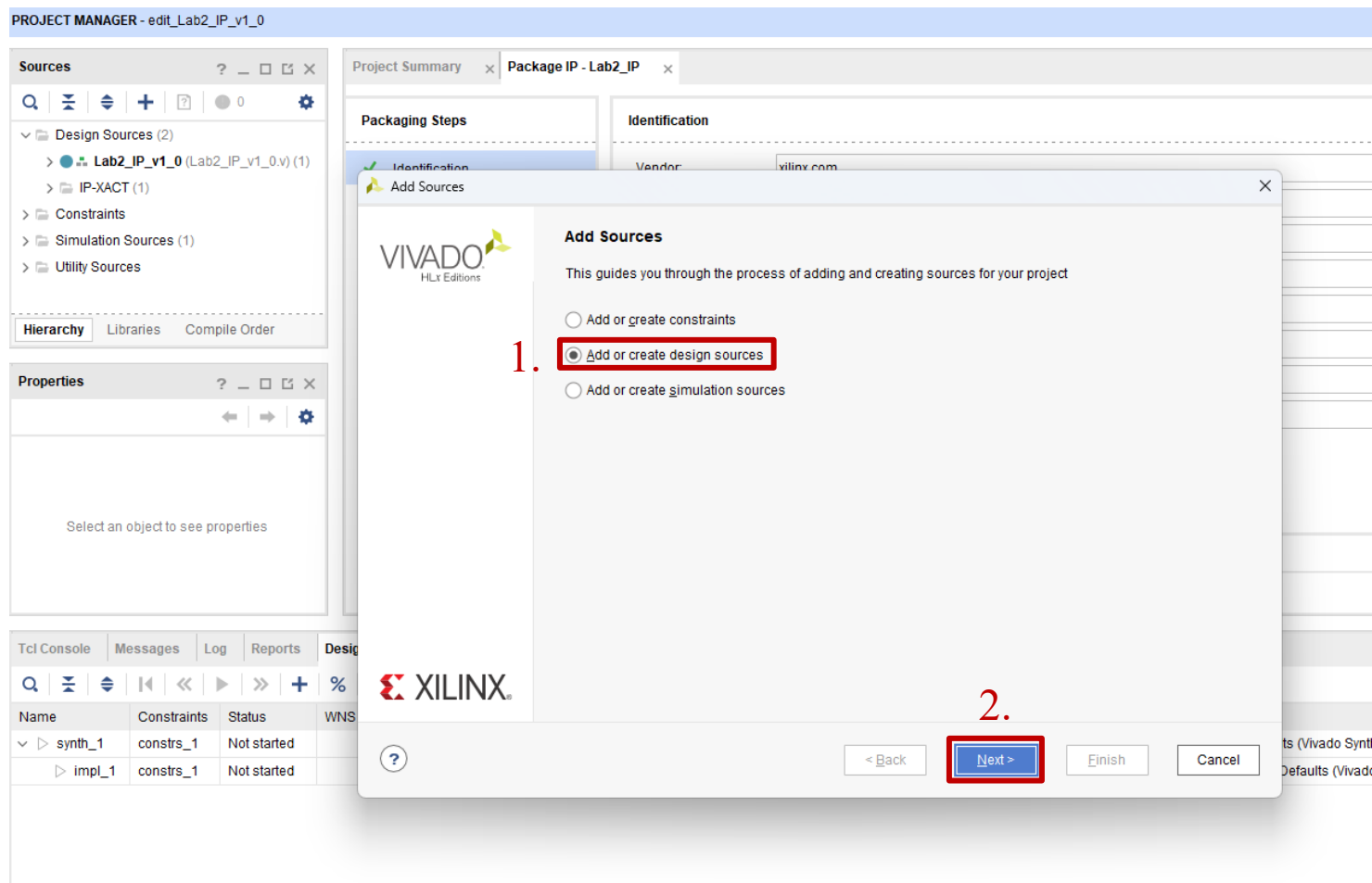


編輯 slave IP 內容

▶ 1. 確認開啟 edit_Lab2_IP 的介面

▶ 2. 將 Calculator.v 加入專案中

(Add source → Add or create design...)



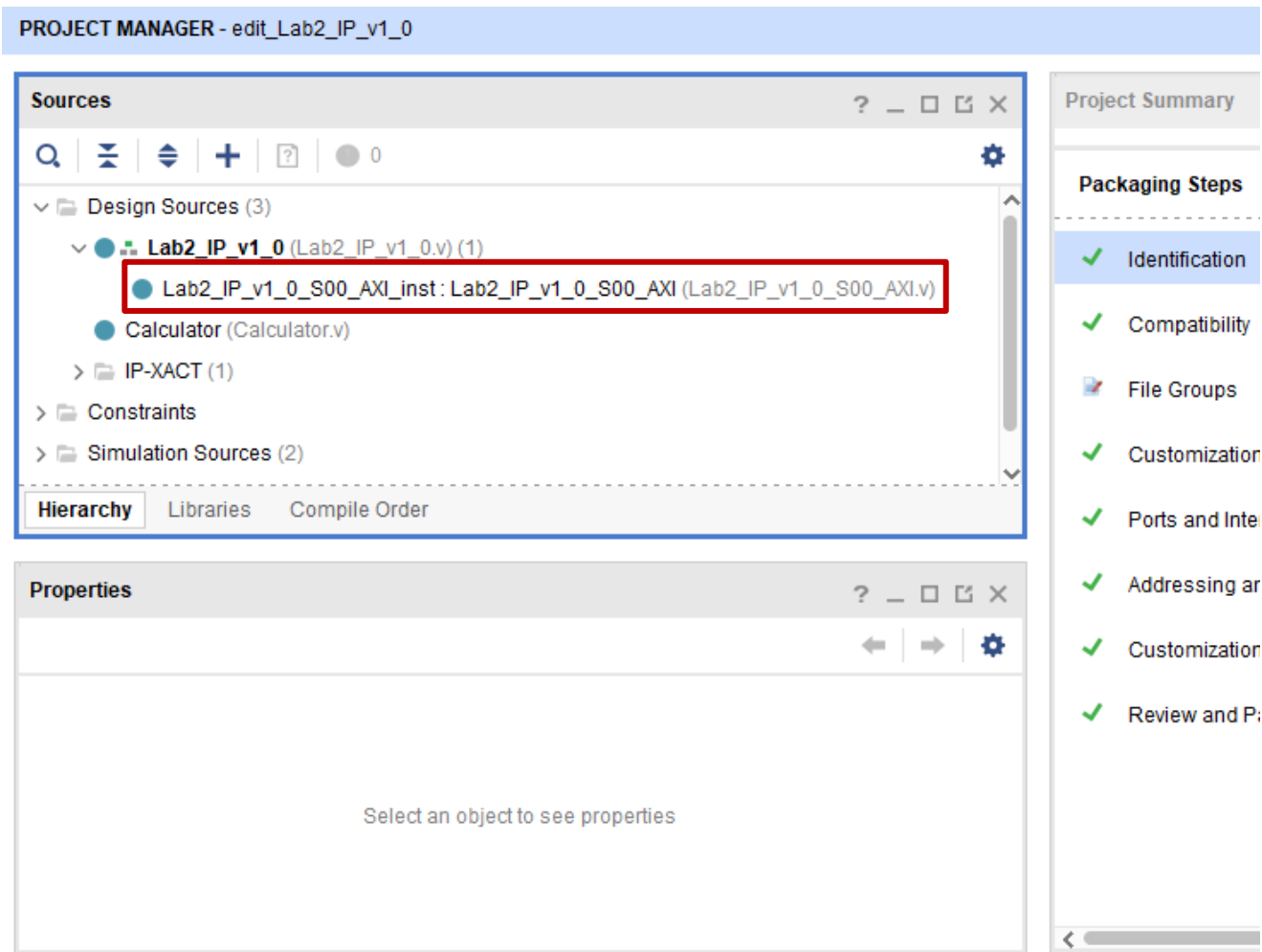
編輯 slave IP 內容

▶ 1.確認開啟 edit_Lab2_IP 的介面

▶ 2.將 Calculator.v 加入專案中

(Add source → Add or create design...)

▶ 3.點開 Lab2_IP_v1_S00_AXI.v 進行編輯



編輯 slave IP 內容

▶ 1. 確認開啟 edit_Lab2_IP 的介面

▶ 2. 將 Calculator.v 加入專案中

(Add source → Add or create design...)

▶ 3. 點開 Lab2_IP_v1_S00_AXI.v 進行編輯

▶ 4. 增加 bit width 的 Parameter，slave template 的 output

```
1  `timescale 1 ns / 1 ps
2
3
4  module Lab2_IP_v1_0_S00_AXI #
5  (
6      // Users to add parameters here
7      1. parameter integer DATA_BW = 16,
8          // User parameters ends
9          // Do not modify the parameters beyond this line
10
11      // Width of S_AXI data bus
12      parameter integer C_S_AXI_DATA_WIDTH    = 32,
13      // Width of S_AXI address bus
14      parameter integer C_S_AXI_ADDR_WIDTH    = 5
15  )
16
17
18
19
20
21
22
23
24
25
26
27
28  output wire S_AXI_RVALID,
29  // Read ready. This signal indicates that the master can
30  // accept the read data and response information.
31  input wire S_AXI_RREADY,
32  //signal for ILA debugging
33  output wire s_axi_awaready,
34  2. output wire [2:0] sel,
35      output wire [DATA_BW-1:0] data_in1,
36      output wire [DATA_BW-1:0] data_in2,
37      output wire [DATA_BW*2-1:0] cal_result
38  );
```

編輯 slave IP 內容

- ▶ 1.確認開啟 edit_Lab2_IP 的介面
- ▶ 2.將 Calculator.v 加入專案中
(Add source → Add or create design...)
- ▶ 3.點開 Lab2_IP_v1_S00_AXI.v 進行編輯
- ▶ 4.增加 bit width 的 Parameter，slave template 的 output
- ▶ 5.將 reg_data_out 的 slv_reg3 改為 cal_result

```
410
411 // Implement memory mapped register select and read logic generation
412 // Slave register read enable is asserted when valid address is available
413 // and the slave is ready to accept the read address.
414 assign slv_reg_rden = axi_arready & S_AXI_ARVALID & ~axi_rvalid;
415 always @(*)
416 begin
417     // Address decoding for reading registers
418     case ( axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] )
419         3'h0 : reg_data_out <= slv_reg0;
420         3'h1 : reg_data_out <= slv_reg1;
421         3'h2 : reg_data_out <= slv_reg2;
422         3'h3 : reg_data_out <= cal_result;
423         3'h4 : reg_data_out <= slv_reg4;
424         3'h5 : reg_data_out <= slv_reg5;
425         3'h6 : reg_data_out <= slv_reg6;
426         3'h7 : reg_data_out <= slv_reg7;
427         default : reg_data_out <= 0;
428     endcase
429 end
```

編輯 slave IP 內容

- ▶ 1. 確認開啟 edit_Lab2_IP 的介面
- ▶ 2. 將 Calculator.v 加入專案中
(Add source → Add or create design...)
- ▶ 3. 點開 Lab2_IP_v1_S00_AXI.v 進行編輯
- ▶ 4. 增加 bit width 的 Parameter，slave template 的 output
- ▶ 5. 將 reg_data_out 的 slv_reg3 改為 cal_result
- ▶ 6. 實例化 calculator module 並修改相關程式

```
449
450 // Add user logic here
451 assign s_axi_awready = (S_AXI_AWREADY == 1) ? 1 : 0;
452 assign sel = slv_reg0;
453 1. assign data_in1 = slv_reg1;
454 assign data_in2 = slv_reg2;
455
456 Calculator #(
457     .DATA_BW(DATA_BW)
458 )u_calculator(
459     .clk(S_AXI_ACLK),
460     .reset(!S_AXI_ARESETN),
461     2. .sel(sel),
462     .data_in1(data_in1),
463     .data_in2(data_in2),
464     .cal_result(cal_result)
465 );
466 // User logic ends
```

編輯 slave IP 內容

▶ 1. 確認開啟 edit_Lab2_IP 的介面

▶ 2. 將 Calculator.v 加入專案中

(Add source → Add or create design...)

▶ 3. 點開 Lab2_IP_v1_S00_AXI.v 進行編輯

▶ 4. 增加 bit width 的 Parameter，slave template 的 output

▶ 5. 將 reg_data_out 的 slv_reg3 改為 cal_result

▶ 6. 實例化 calculator module 並修改相關程式

▶ 7. 點選 Lab2_IP_v1_0.v 去修改 Top Wrapper 與 slave template 的接線

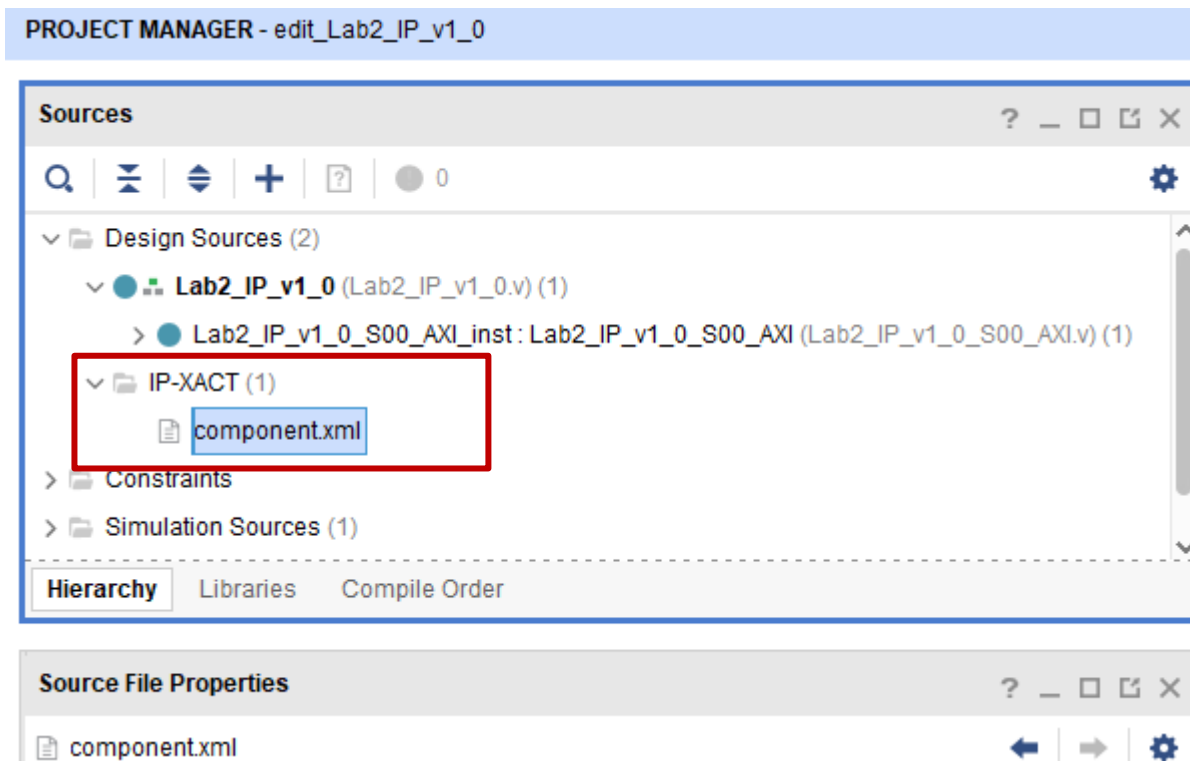
```
module Lab2_IP_v1_0 #  
(  
    1. // Users to add parameters here  
    parameter integer DATA_BW = 16,  
    // User parameters ends  
    // Do not modify the parameters beyond this line
```

```
    output wire [1 : 0] s00_axi_rresp,  
    output wire s00_axi_rvalid,  
    input wire s00_axi_rready,  
    //ILA signal  
    output wire s_axi_awready,  
    output wire [2:0] sel,  
    2. output wire [DATA_BW-1:0] data_in1,  
    output wire [DATA_BW-1:0] data_in2,  
    output wire [DATA_BW*2-1:0] cal_result  
);
```

```
    .S_AXI_RREADY(s00_axi_rready),  
    // ILA signal  
    .s_axi_awready(s_axi_awready),  
    .sel(sel),  
    3. .data_in1(data_in1),  
    .data_in2(data_in2),  
    .cal_result(cal_result)  
);
```

編輯 slave IP 內容

- ▶ 8.點選 source 中的 IP-XACT→ component.xml



編輯 slave IP 內容

- ▶ 8.點選 source 中的 IP-XACT→ component.xml
- ▶ 9.點選 File Groups 中的 Merge Changes from File Groups Wizard...

1. **File Groups**

2. **Merge changes from File Groups Wizard**

Name	Library Name	Type	Is Include	Used In Constant	File Group Name
Standard			☐	☐	
Advanced			☐	☐	
Verilog Synthesis (2)			☐	☐	
Verilog Simulation (2)			☐	☐	
Software Driver (6)			☐	☐	
UI Layout (1)			☐	☐	
Block Diagram (1)			☐	☐	

編輯 slave IP 內容

- ▶ 8.點選 Lab2_IP_v1_0.v 去修改 Top Wrapper 與 slave template 的接線
- ▶ 9.點選 File Groups 中的 Merge Changes from File Group Wizard...
- ▶ 10.點選 Customization Parameters 中的 Merge Changes from Customization Parameters Wizard...

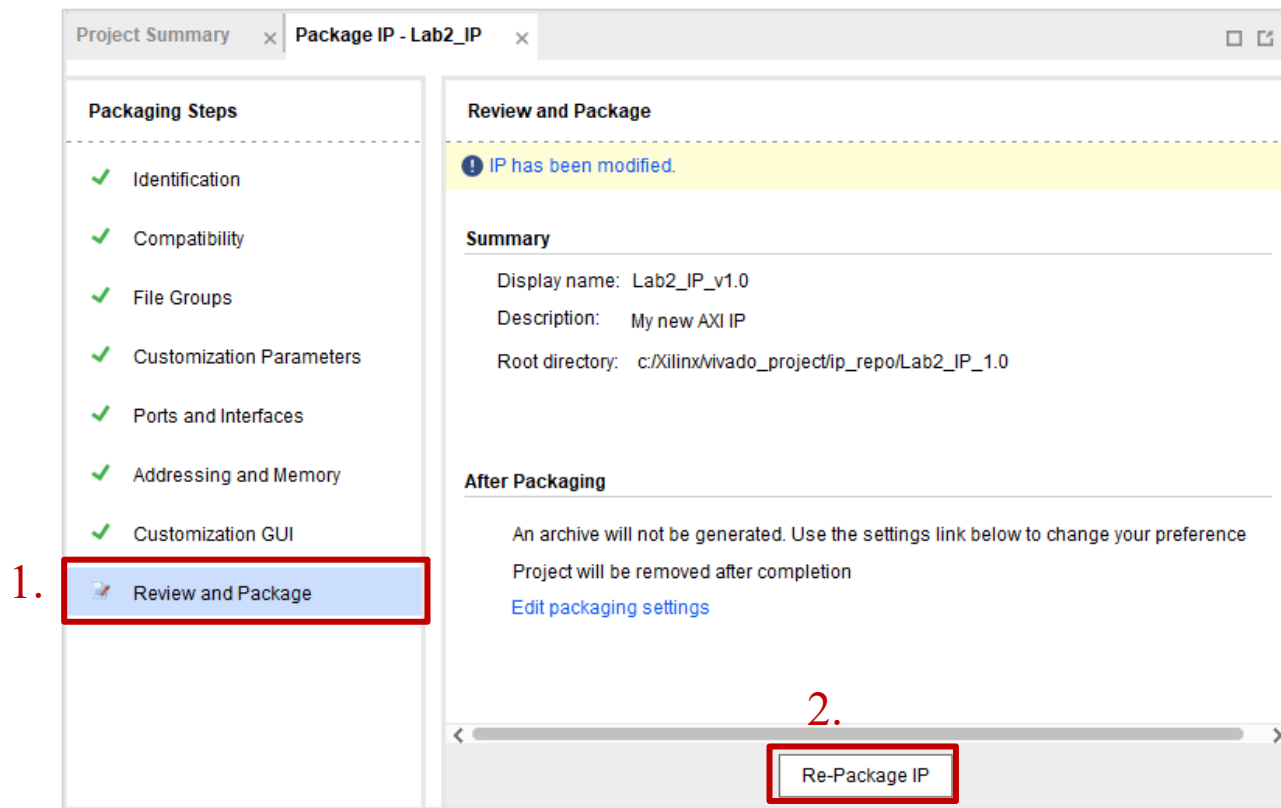
1. Customization Parameters

2. Merge changes from Customization Parameters Wizard

Name	Description	Display Name
Customization Parameters		
C_S00_AXI_DATA_WIDTH	Width of S_AXI data bus	C S00 AXI DATA WIDTH
C_S00_AXI_ADDR_WIDTH	Width of S_AXI address bus	C S00 AXI ADDR WIDT
C_S00_AXI_BASEADDR		C S00 AXI BASEADDR
C_S00_AXI_HIGHADDR		C S00 AXI HIGHADDR

編輯 slave IP 內容

- ▶ 8.點選 Lab2_IP_v1_0.v 去修改 Top Wrapper 與 slave template 的接線
- ▶ 9.點選 File Groups 中的 Merge Changes from File Group Wizard...
- ▶ 10.點選 Customization Parameters 中的 Merge Changes from Customization Parameters Wizard...
- ▶ 11.點選 Review and Package → Re-Package IP 完成編輯並關閉 edit_Lab2_IP 視窗

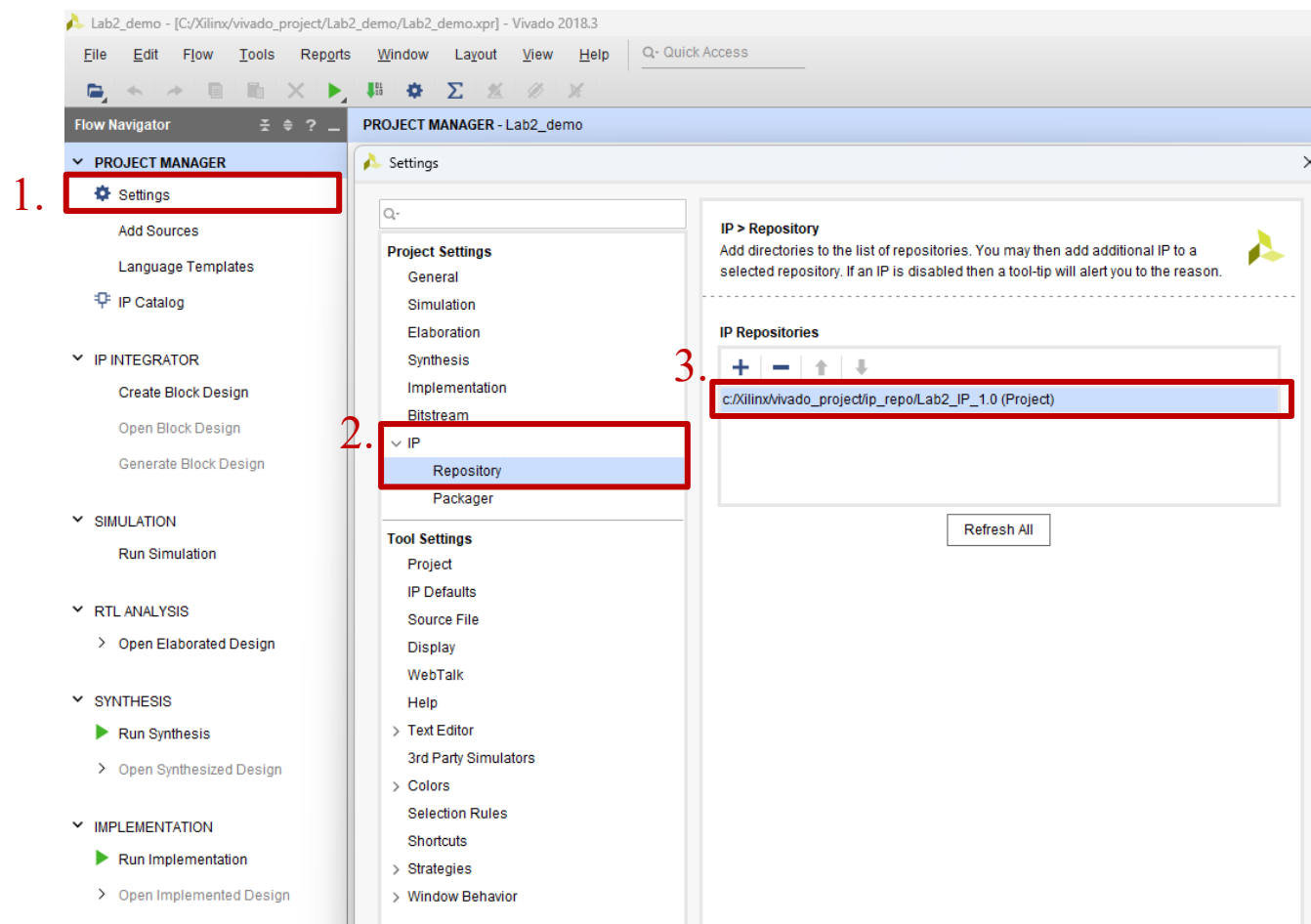


Part 1.1

- ▶ 建立 slave IP
- ▶ 編輯 slave IP 內容
- ▶ 建立 Hardware IP
- ▶ 加入 ILA debug core

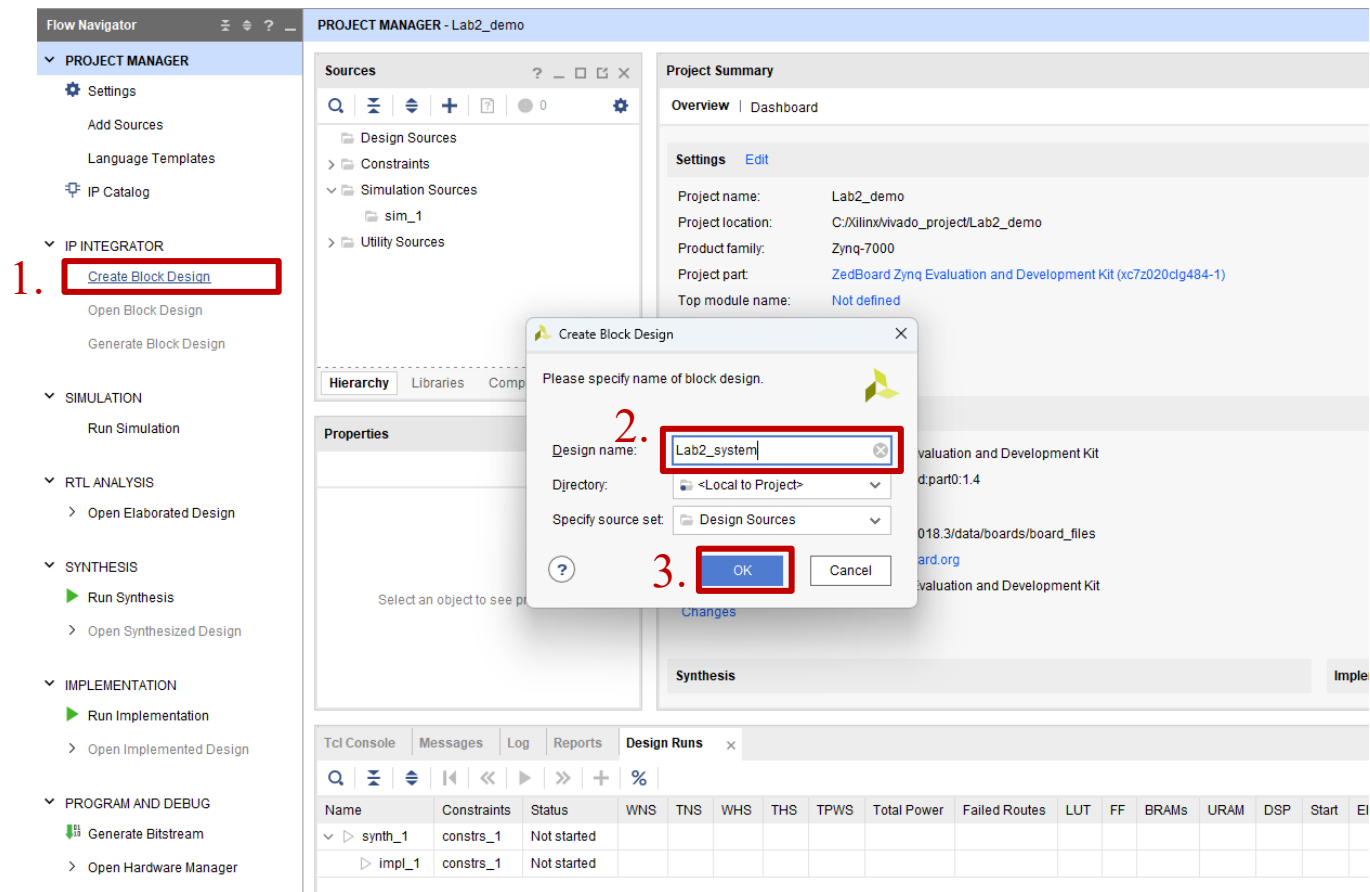
建立Hardware IP

- ▶ 1.點選 Setting → IP → Repository → 確認
IP 已經被建立



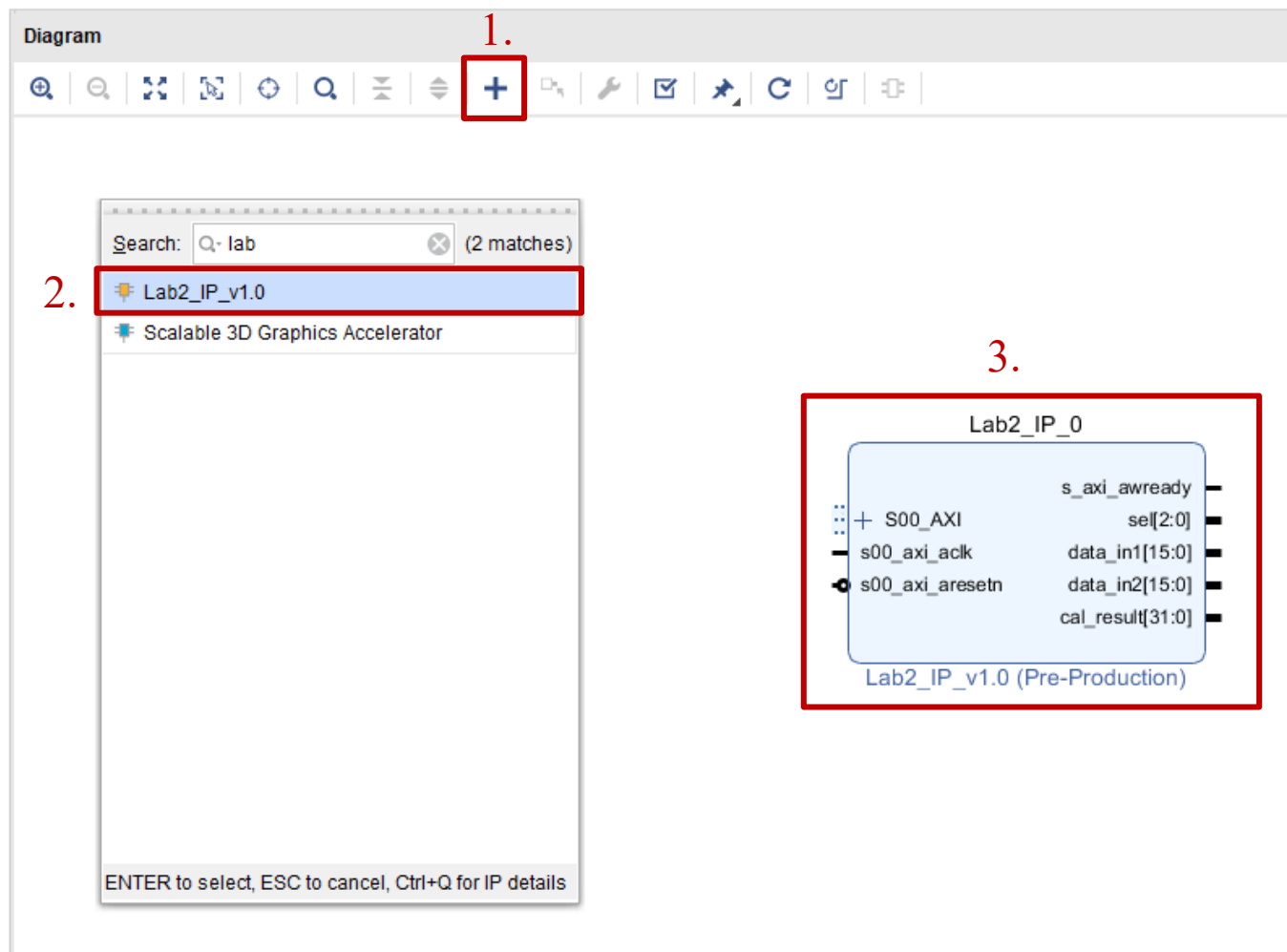
建立Hardware IP

- ▶ 1.點選 Setting → IP → Repository → 確認
IP 已經被建立
- ▶ 2.點選 Create Block Design → OK



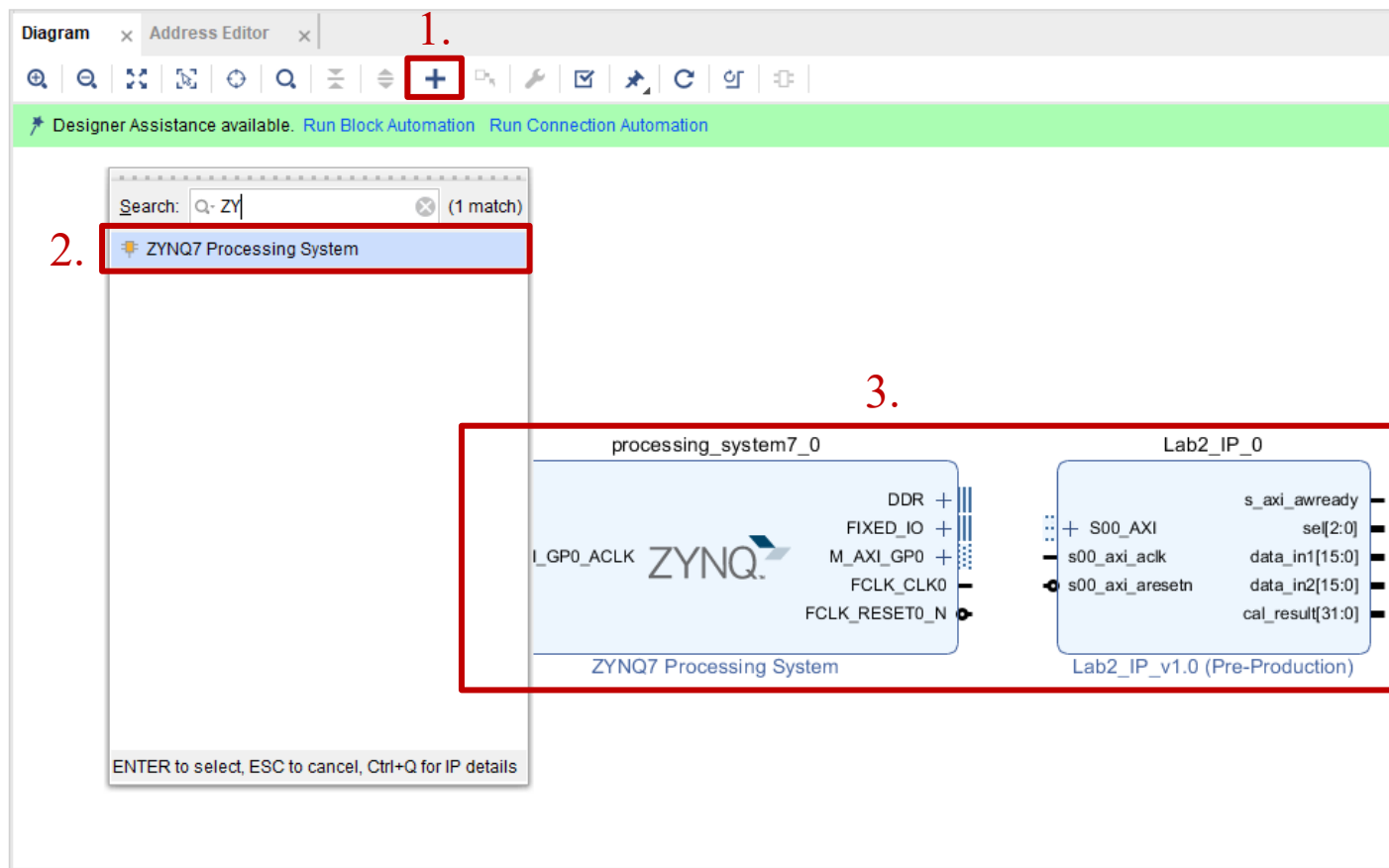
建立Hardware IP

- ▶ 1.點選 Setting → IP → Repository → 確認 IP 已經被建立
- ▶ 2.點選 Create Block Design → OK
- ▶ 3.點選 **+** 搜尋你的 IP 並新增到 block design



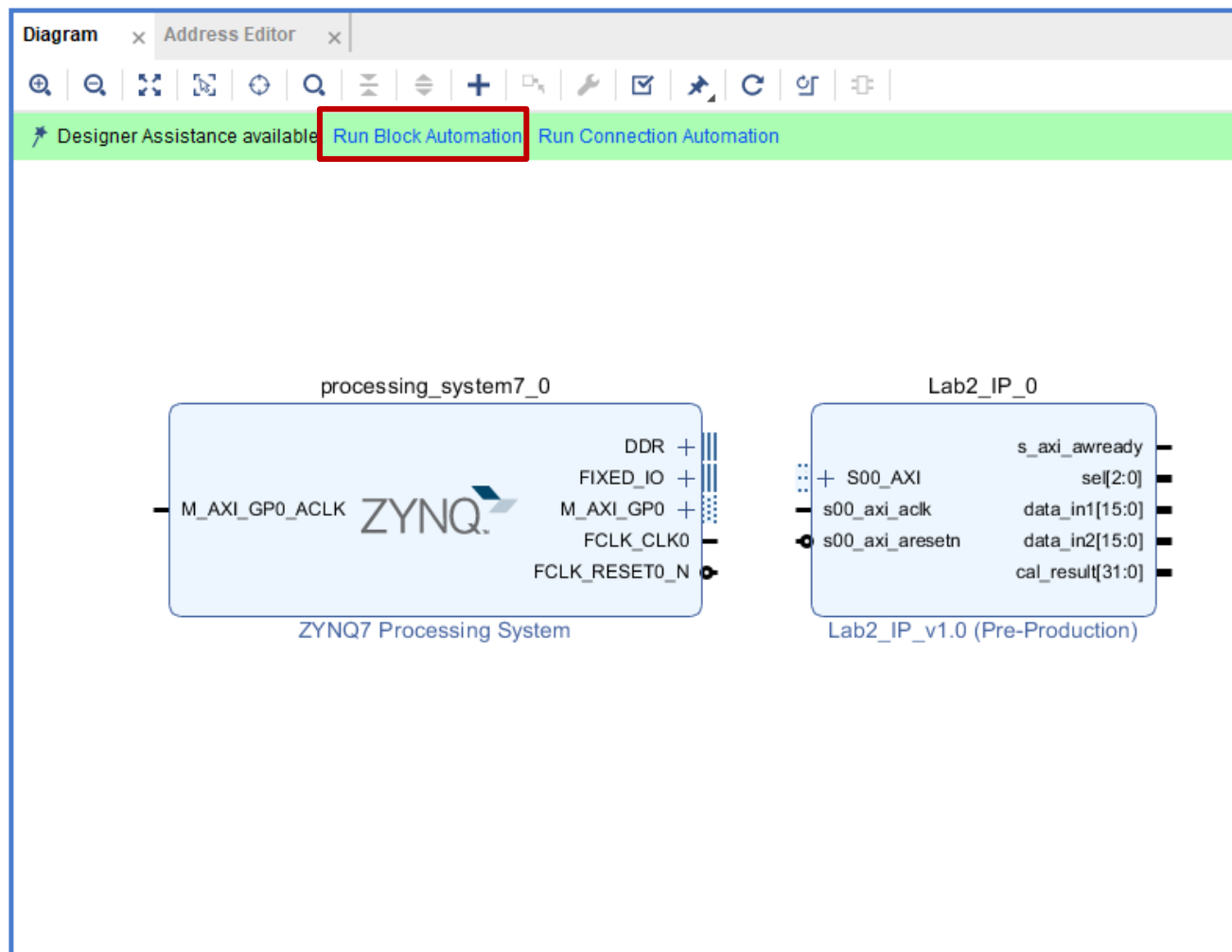
建立Hardware IP

- ▶ 1.點選 Setting → IP → Repository → 確認 IP 已經被建立
- ▶ 2.點選 Create Block Design → OK
- ▶ 3.點選 **+** 搜尋你的 IP 並新增到 block design
- ▶ 4.點選 **+** 搜尋 ZYNQ



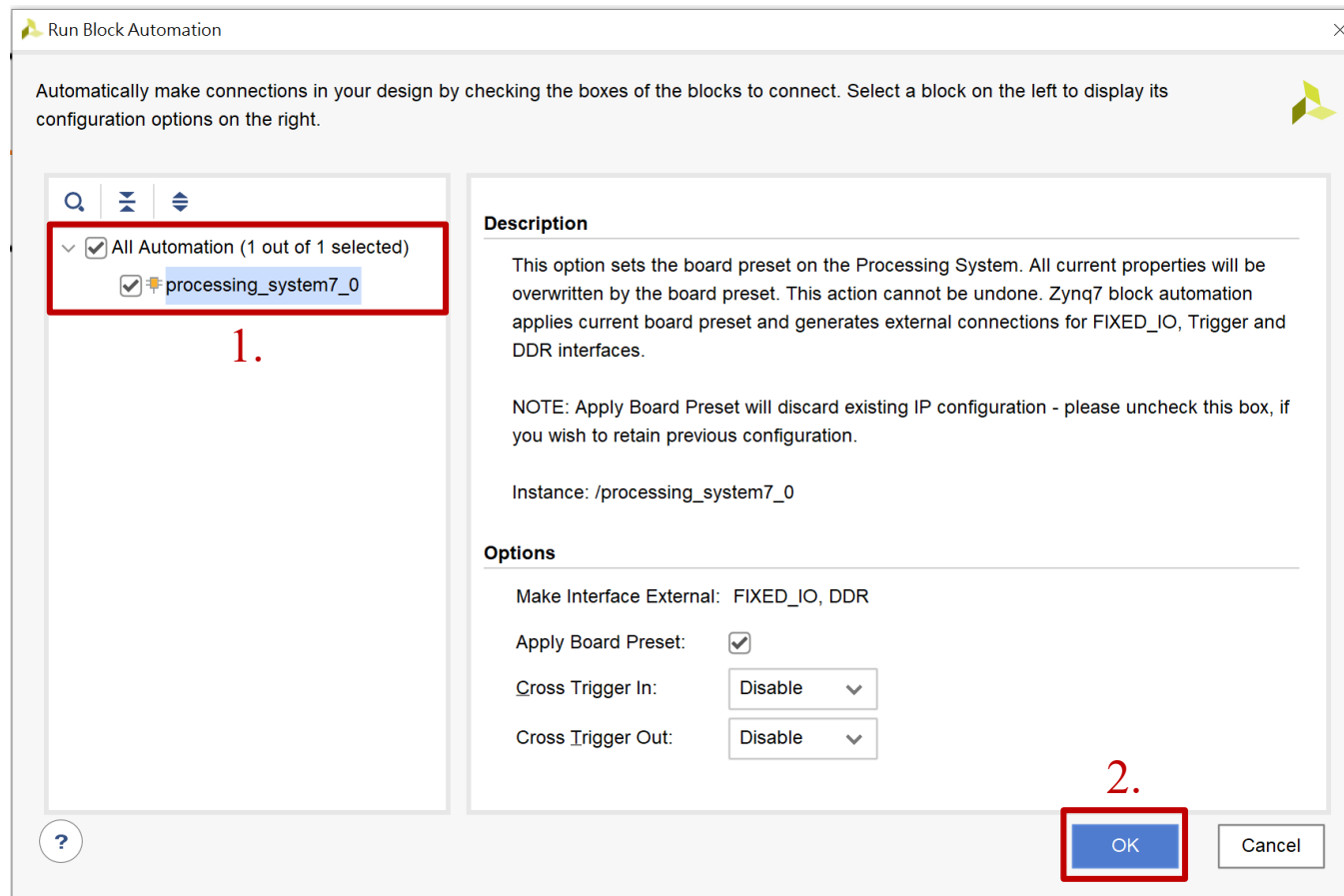
建立Hardware IP

- ▶ 1.點選 Setting → IP → Repository → 確認 IP 已經被建立
- ▶ 2.點選 Create Block Design → OK
- ▶ 3.點選 **+** 搜尋你的 IP 並新增到 block design
- ▶ 4.點選 **+** 搜尋 ZYNQ
- ▶ 5.點選 **Run Block Automation**



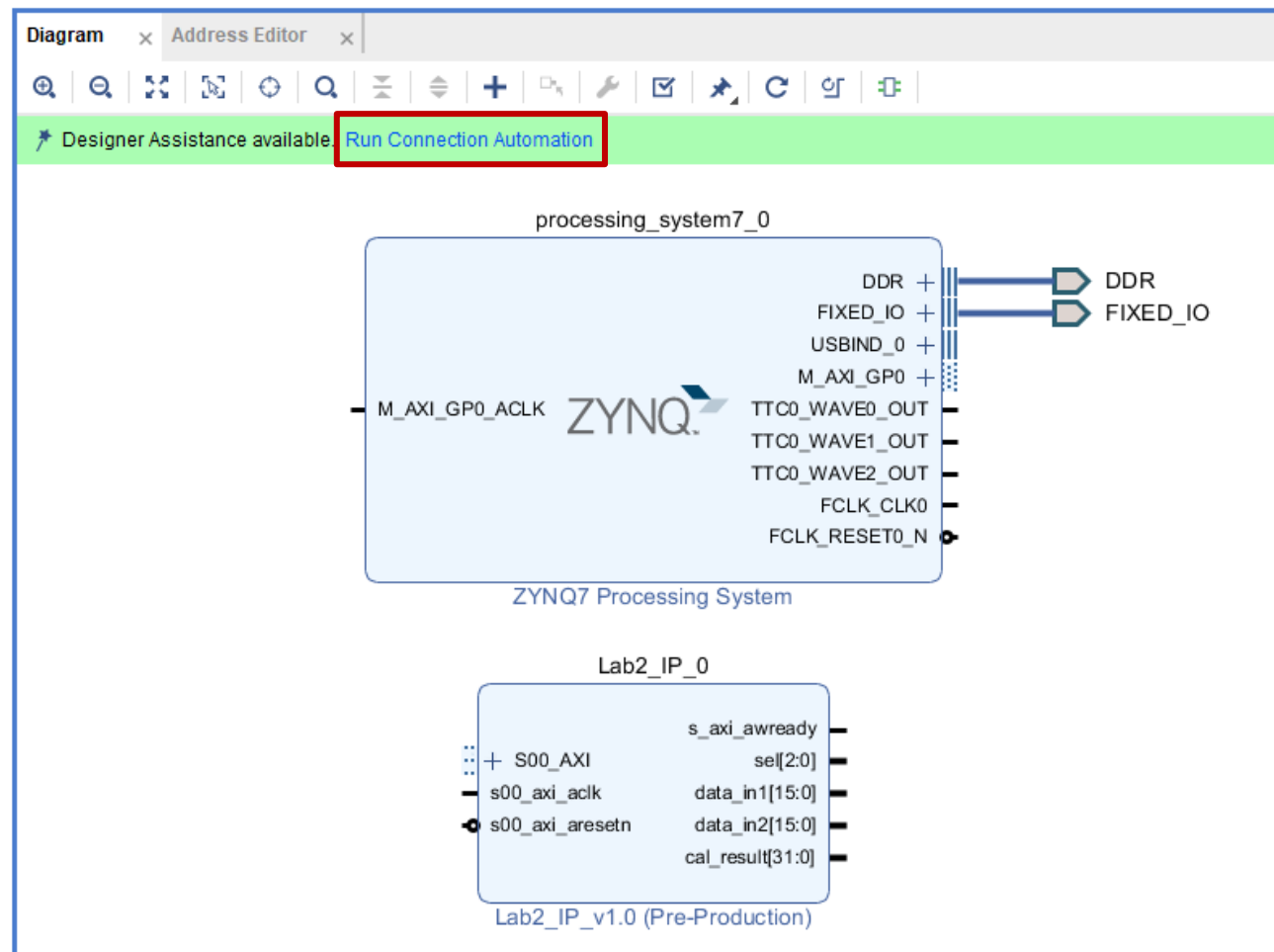
建立Hardware IP

- ▶ 1.點選 Setting → IP → Repository → 確認 IP 已經被建立
- ▶ 2.點選 Create Block Design → OK
- ▶ 3.點選 **+** 搜尋你的 IP 並新增到 block design
- ▶ 4.點選 **+** 搜尋 ZYNQ
- ▶ 5.點選 Run Block Automation
- ▶ 6.確認勾選 → OK



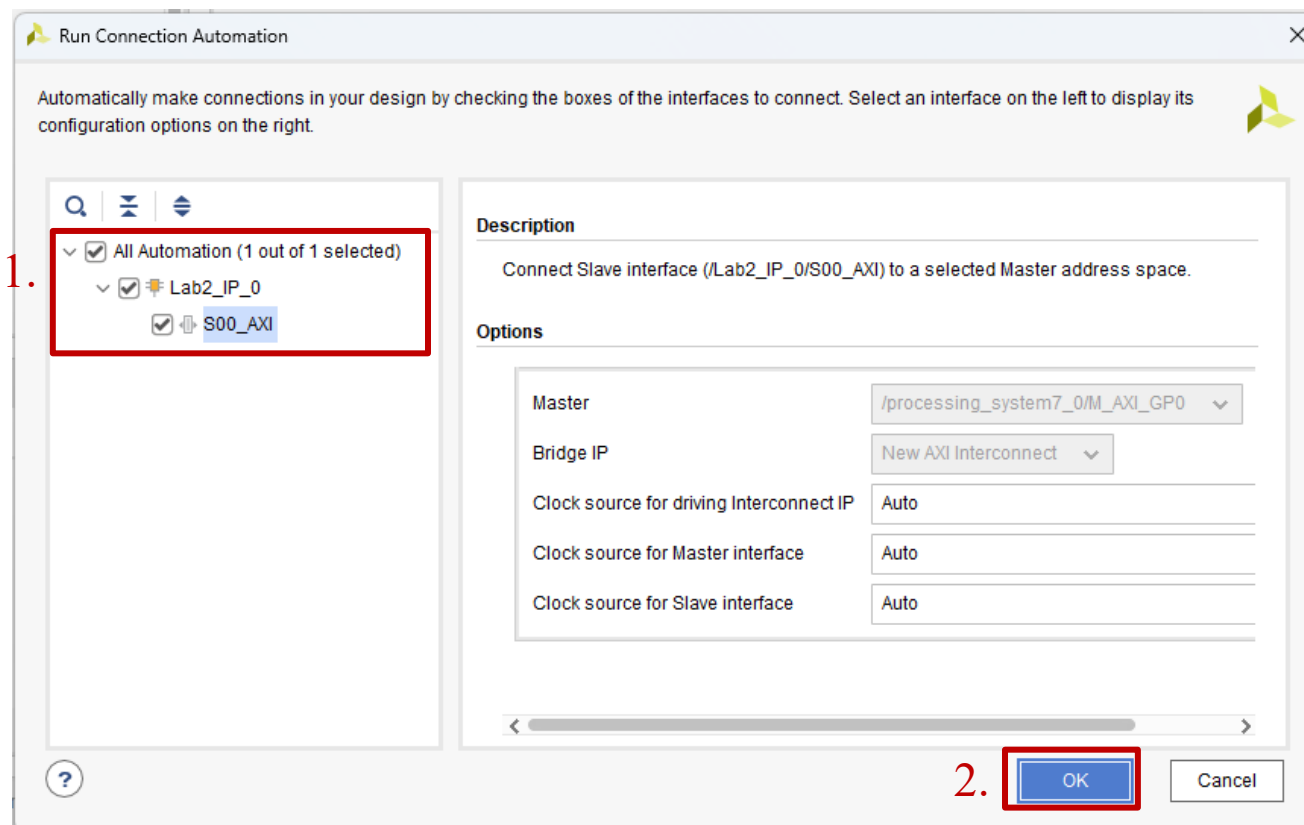
建立Hardware IP

- ▶ 1.點選 Setting → IP → Repository → 確認 IP 已經被建立
- ▶ 2.點選 Create Block Design → OK
- ▶ 3.點選 **+** 搜尋你的 IP 並新增到 block design
- ▶ 4.點選 **+** 搜尋 ZYNQ
- ▶ 5.點選 Run Block Automation
- ▶ 6.確認勾選 → OK
- ▶ 7.點選 Run Connection Automation



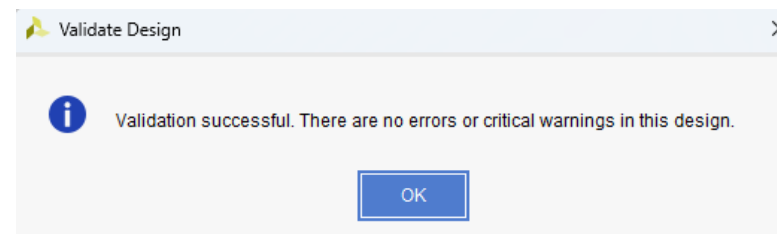
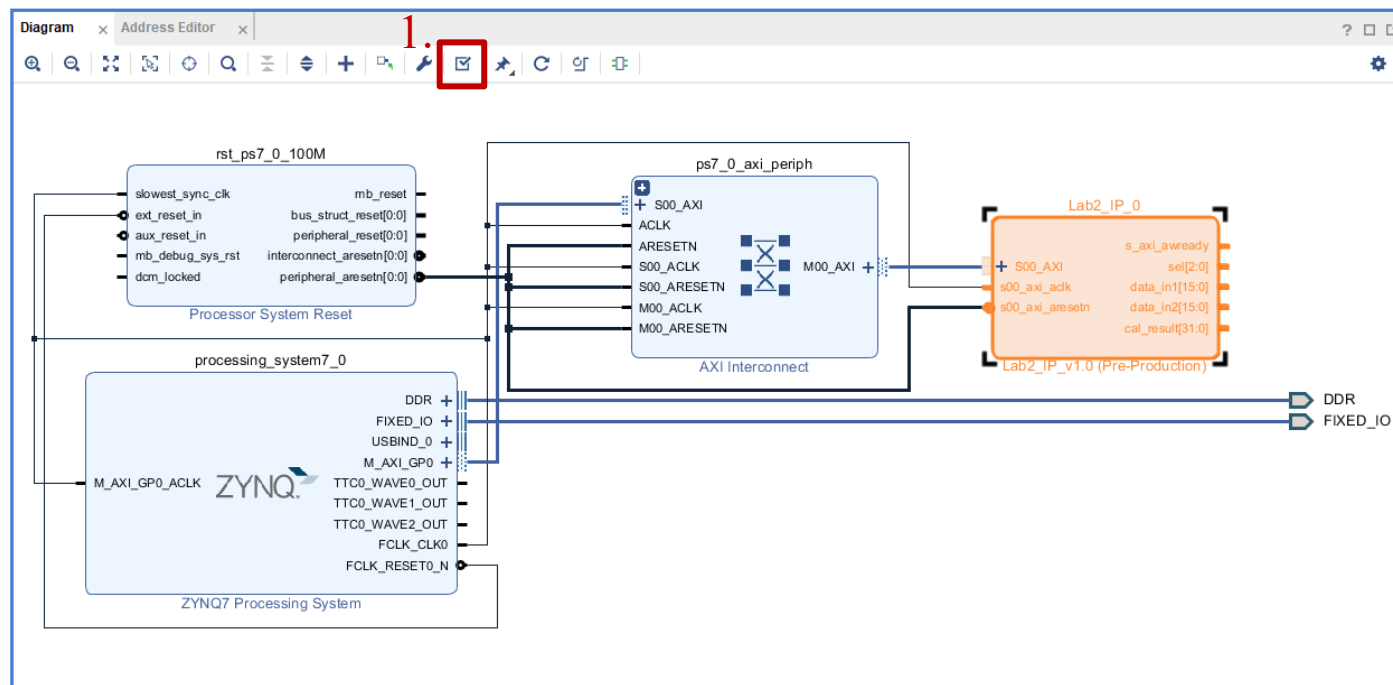
建立Hardware IP

- ▶ 1.點選 Setting → IP → Repository → 確認 IP 已經被建立
- ▶ 2.點選 Create Block Design → OK
- ▶ 3.點選 **+** 搜尋你的 IP 並新增到 block design
- ▶ 4.點選 **+** 搜尋 ZYNQ
- ▶ 5.點選 Run Block Automation
- ▶ 6.確認勾選 → OK
- ▶ 7.點選 Run Connection Automation
- ▶ 8.確認勾選 → OK



建立Hardware IP

- ▶ 1.點選 Setting → IP → Repository → 確認 IP 已經被建立
- ▶ 2.點選 Create Block Design → OK
- ▶ 3.點選  搜尋你的 IP 並新增到 block design
- ▶ 4.點選  搜尋 ZYNQ
- ▶ 5.點選 Run Block Automation
- ▶ 6.確認勾選 → OK
- ▶ 7.點選 Run Connection Automation
- ▶ 8.確認勾選 → OK
- ▶ 9.點選 validate design 確認接線情況

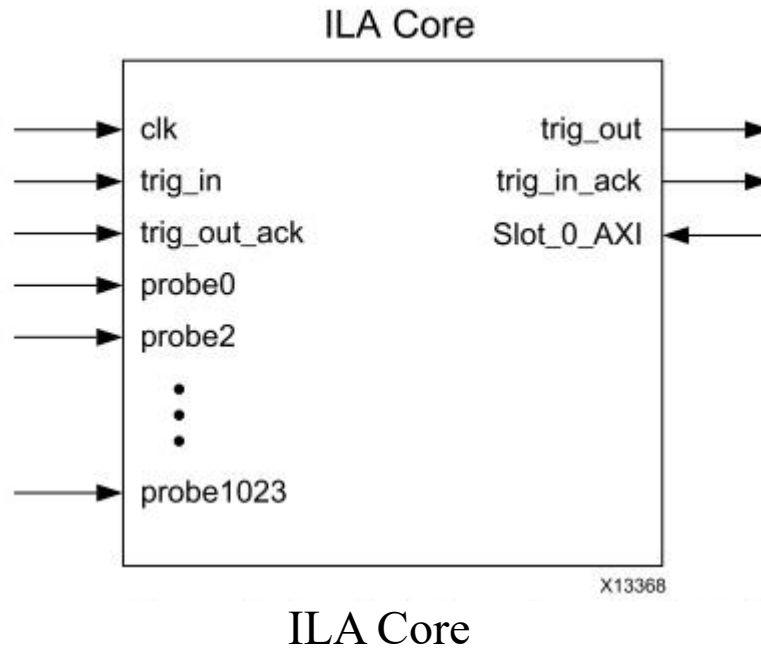


Part 1.1

- ▶ 建立 slave IP
- ▶ 編輯 slave IP 內容
- ▶ 建立 Hardware IP
- ▶ 加入 ILA debug core

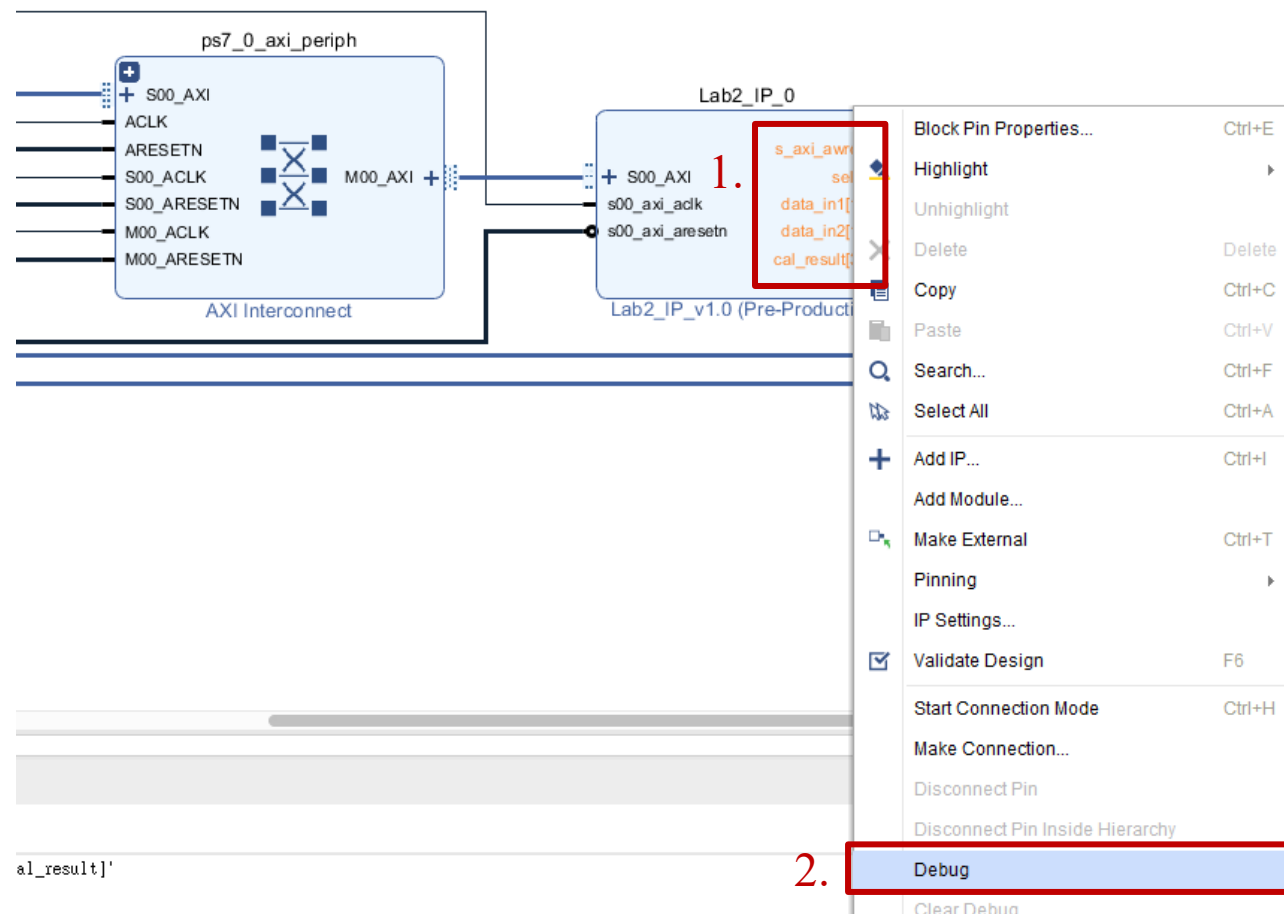
加入 ILA Core

- ▶ FPGA 內部的電路，是無法直接查看波型的，因此在 vivado 中有一個 IP 為 ILA(Integrated Logic Analyzer)，可以用來分析 FPGA 內部電路的訊號。



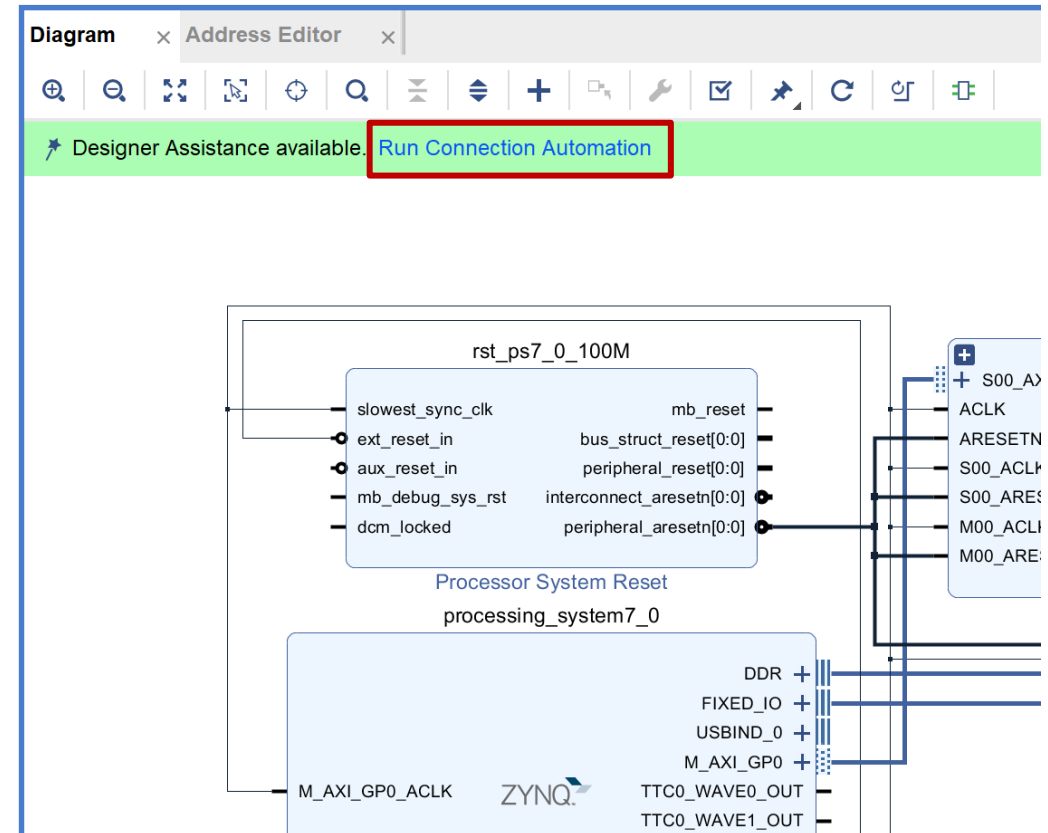
加入 ILA Core

- 1. 選取 slave IP 中的 output port → Debug



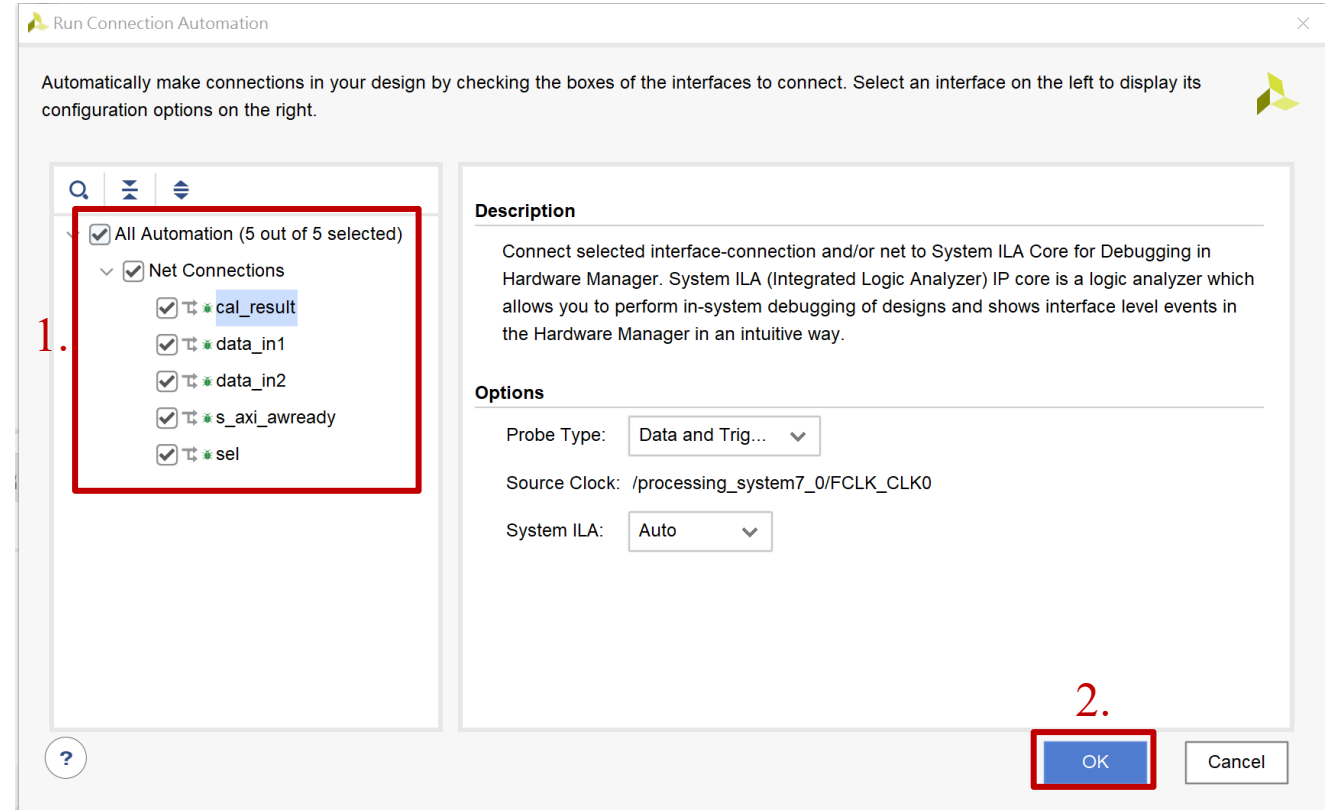
加入 ILA Core

- ▶ 1. 選取 slave IP 中的 output port → Debug
- ▶ 2. 點選 Run Connection Automation



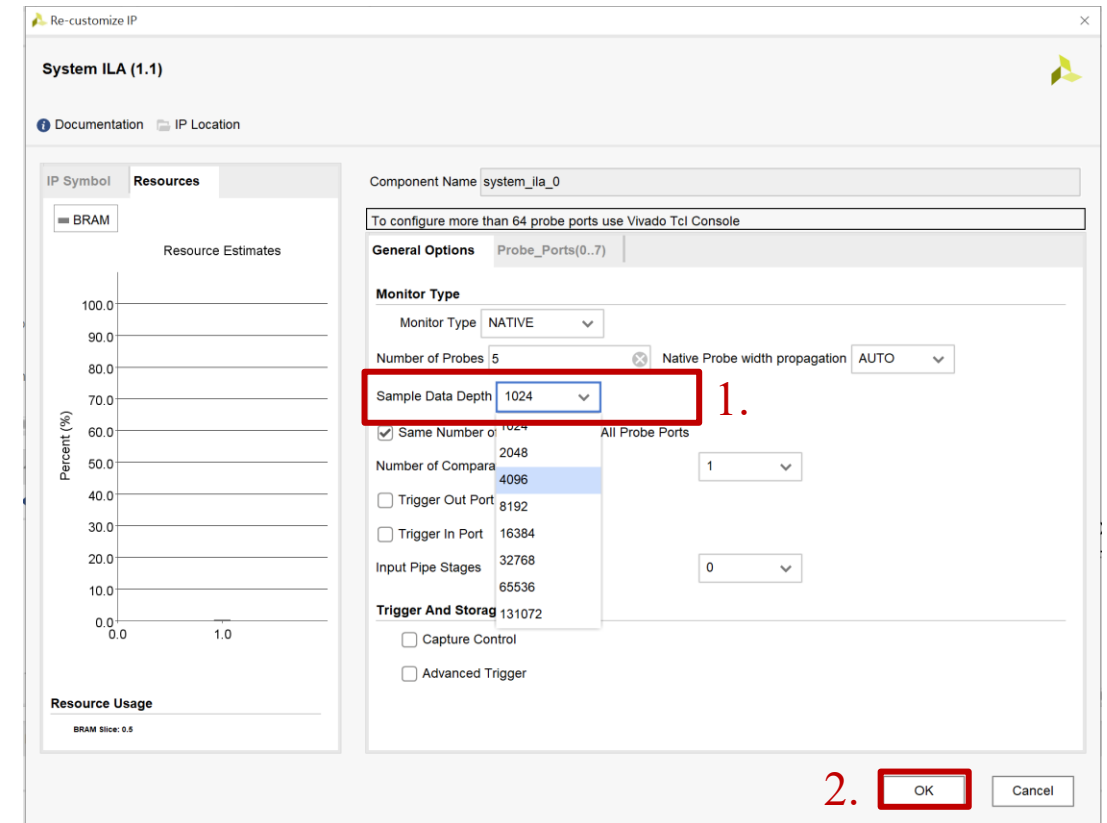
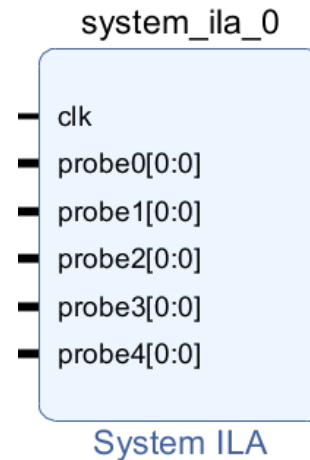
加入 ILA Core

- ▶ 1. 選取 slave IP 中的 output port → Debug
- ▶ 2. 點選 Run Connection Automation
- ▶ 3. 點選 All automation(全選)



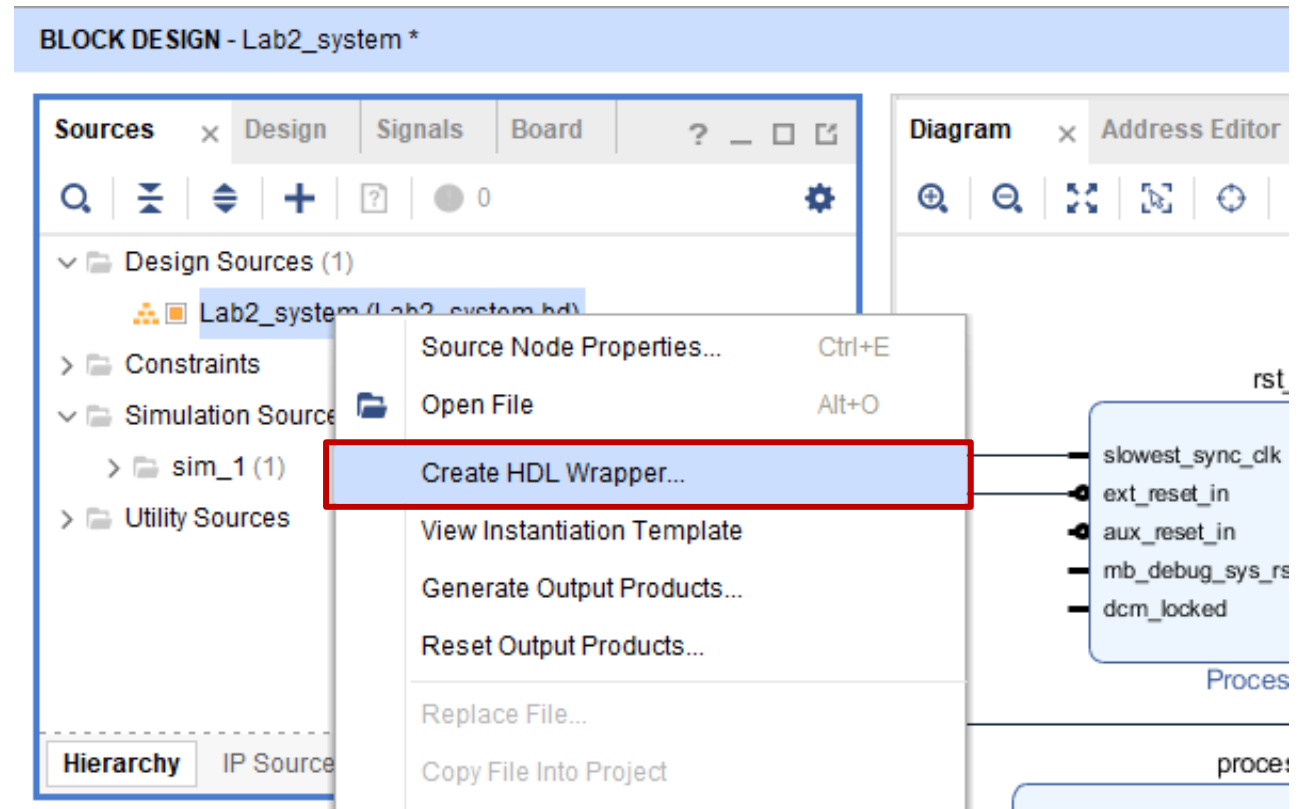
加入 ILA Core

- ▶ 1.選取 slave IP 中的 output port → Debug
- ▶ 2.點選 Run Connection Automation
- ▶ 3.點選 All automation(全選)
- ▶ 4.double click ILA core 後可以選擇 sample data depth → OK



加入 ILA Core

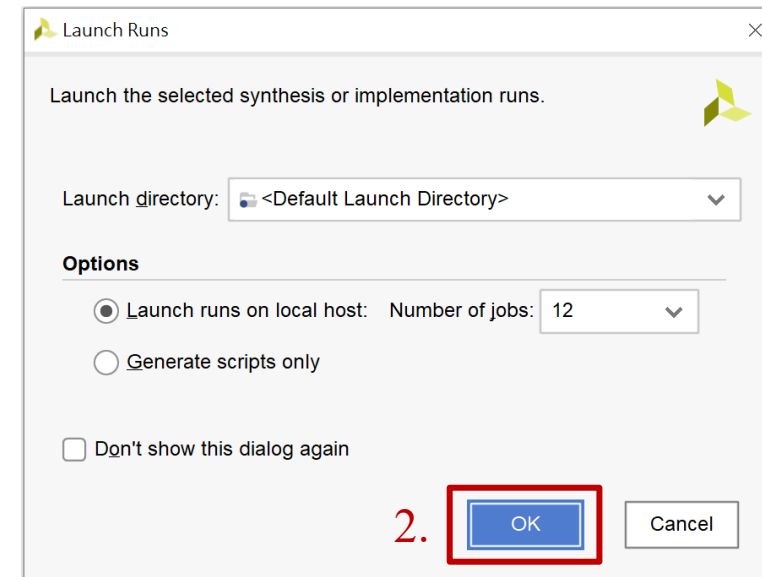
- ▶ 1.選取 slave IP 中的 output port → Debug
- ▶ 2.點選 Run Connection Automation
- ▶ 3.點選 All automation(全選)
- ▶ 4.double click ILA core 後可以選擇 sample data depth → OK
- ▶ 5.點選 Create HDL Wrapper 包裝硬體



加入 ILA Core

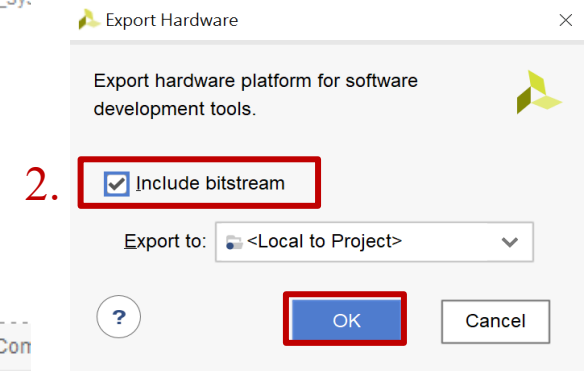
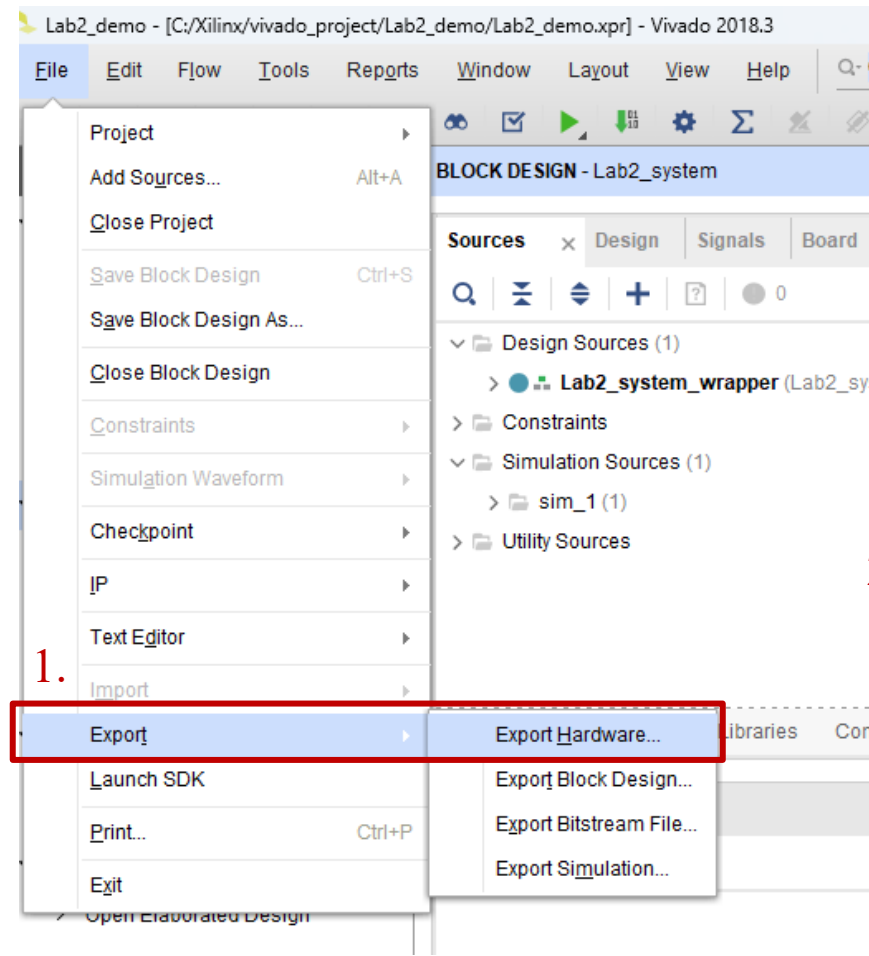
- ▶ 1.選取 slave IP 中的 output port → Debug
- ▶ 2.點選 Run Connection Automation
- ▶ 3.點選 All automation(全選)
- ▶ 4.double click ILA core 後可以選擇 sample data depth → OK
- ▶ 5. 點選 Create HDL Wrapper 包裝硬體
- ▶ 6. 點選 **Generate Bitstream** → OK

- ▼ SYNTHESIS
 - ▶ Run Synthesis
 - > Open Synthesized Design
- ▼ IMPLEMENTATION
 - ▶ Run Implementation
 - > Open Implemented Design
- ▼ PROGRAM AND DEBUG
 - ▶ **Generate Bitstream** 1.
 - > Open Hardware Manager



加入 ILA Core

- ▶ 1. 選取 slave IP 中的 output port → Debug
- ▶ 2. 點選 Run Connection Automation
- ▶ 3. 點選 All automation(全選)
- ▶ 4. double click ILA core 後可以選擇 sample data depth → OK
- ▶ 5. 點選 Create HDL Wrapper 包裝硬體
- ▶ 6. 點選 Generate Bitstream → OK
- ▶ 7. 點選 File → Export Hardware → Include bitstream → OK

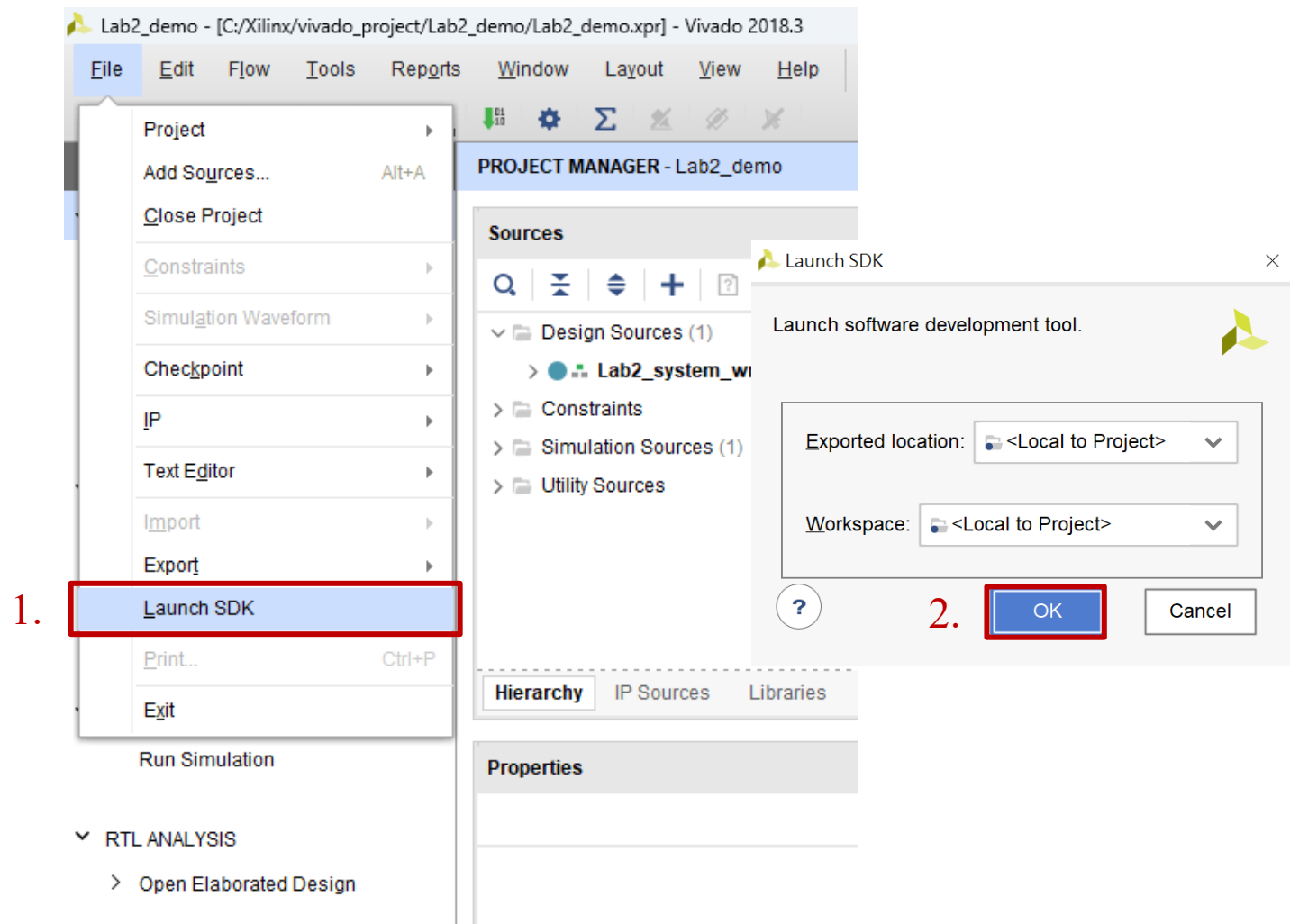


Part 1.2

- ▶ SDK 操作
- ▶ 撰寫 SDK 程式
- ▶ 燒錄 SDK 並執行結果

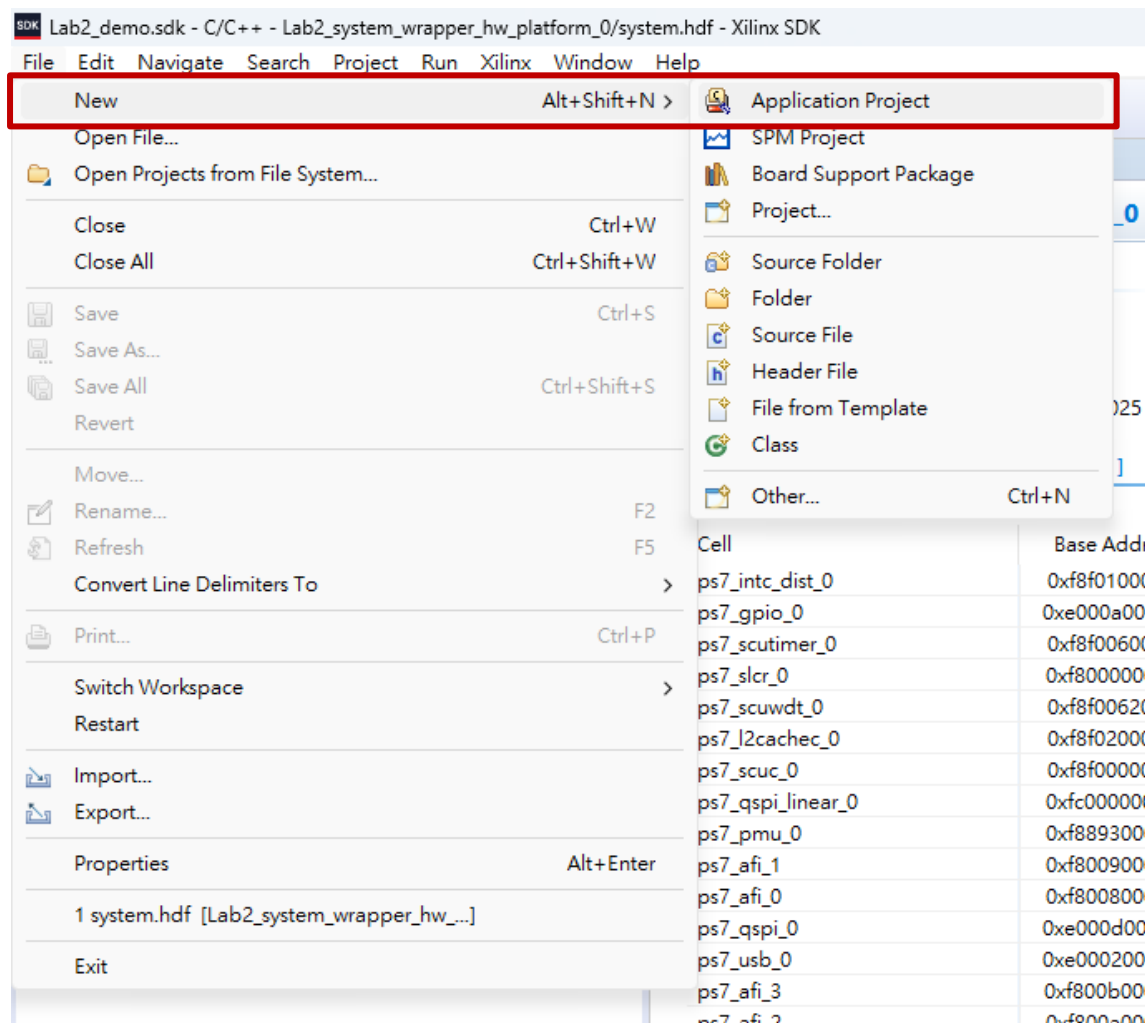
SDK操作

- ▶ 1.點選 File → Launch SDK → OK



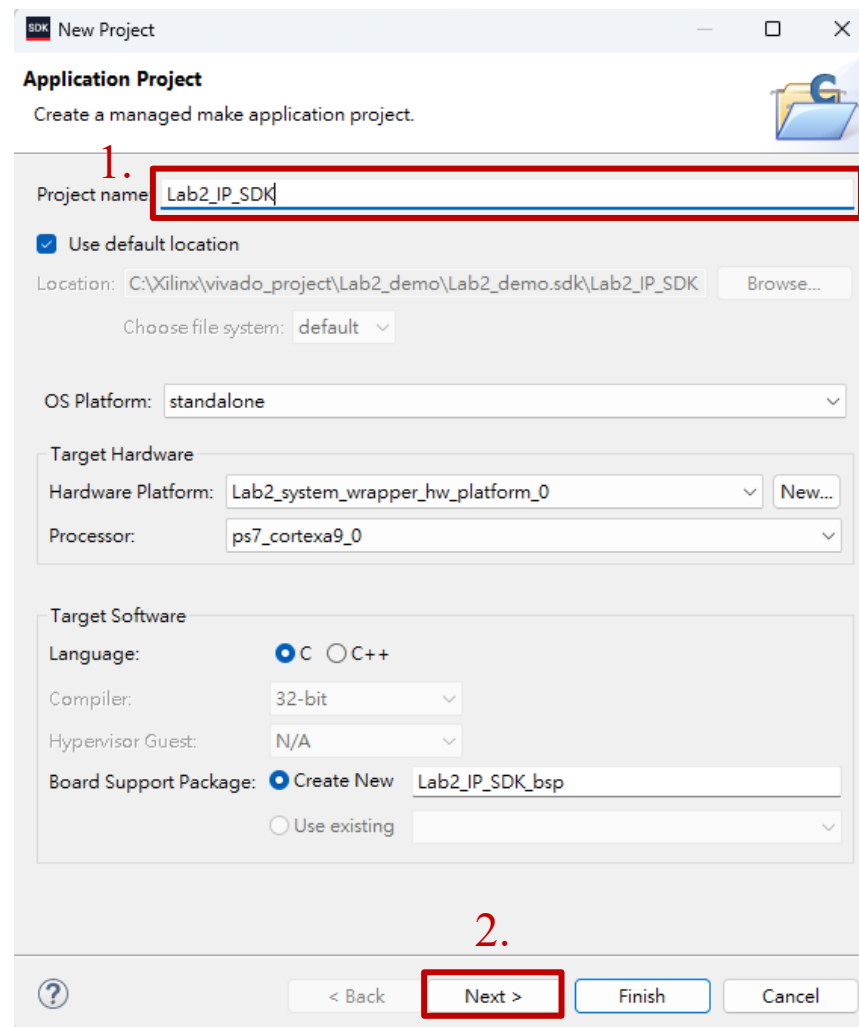
SDK操作

- ▶ 1.點選 File → Launch SDK → OK
- ▶ 2.點選 File → New → Application Project



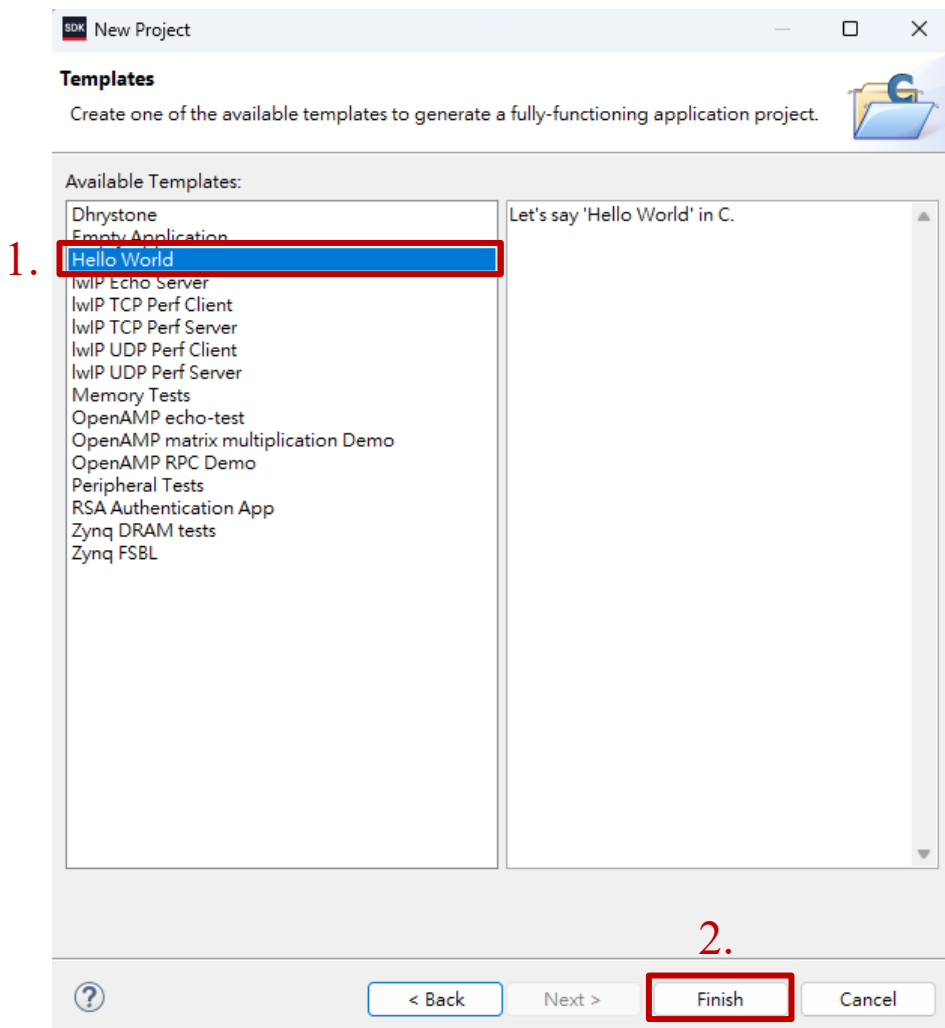
SDK操作

- ▶ 1.點選 File → Launch SDK → OK
- ▶ 2.點選 File → New → Application Project
- ▶ 3.建立 SDK專案 → Next



SDK操作

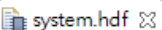
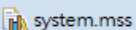
- ▶ 1.點選 File → Launch SDK → OK
- ▶ 2.點選 File → New → Application Project
- ▶ 3.建立 SDK專案 → Next
- ▶ 4.選取 Hello World → Finish



SDK操作

- ▶ 1.點選 File → Launch SDK → OK
- ▶ 2.點選 File → New → Application Project
- ▶ 3.建立 SDK專案 → Next
- ▶ 4.選取 Hello World → Finish
- ▶ 5.開啟 system.hdf 找到我們 IP 的 base address

(在一個數位系統中，周邊硬體通常會被定址到一個記憶體區間讓開發人員進行暫存器的讀寫)

1.  system.hdf  system.mss

Lab2_system_wrapper_hw_platform_0 Hardware Platform Specification

Design Information

Target FPGA Device: 7z020
Part: xc7z020clg484-1
Created With: Vivado 2018.3
Created On: Mon Mar 31 20:13:36 2025

Address Map for processor ps7_cortexa9_0[0-1]

Cell	Base Addr	High Addr	Slave I/f	Mem/Reg
ps7_intc_dist_0	0xf8f01000	0xf8f01fff		REGISTER
ps7_gpio_0	0xe000a000	0xe000afff		REGISTER
ps7_scutimer_0	0xf8f00600	0xf8f0061f		REGISTER
ps7_dma_ns	0xf8004000	0xf8004fff		REGISTER
ps7_sd_0	0xe0100000	0xe0100fff		REGISTER
ps7_gpv_0	0xf8900000	0xf89fffff		REGISTER
ps7_ram_1	0xffff0000	0xffffdfff		MEMORY
ps7_ram_0	0x00000000	0x0002ffff		MEMORY
Lab2_IP_0	0x43c00000	0x43c0ffff	S00_AXI	REGISTER

IP blocks present in the design

ps7_0_axi_periph	axi_interconnect	2.1
ps7_intc_dist_0	ps7_intc_dist	1.00.a
rst_ps7_0_100M	proc_sys_reset	5.0
ps7_gpio_0	ps7_gpio	1.00.a
ps7_scutimer_0	ps7_scutimer	1.00.a
Lab2_IP_0	Lab2_IP	1.0
ps7_slcr_0	ps7_slcr	1.00.a

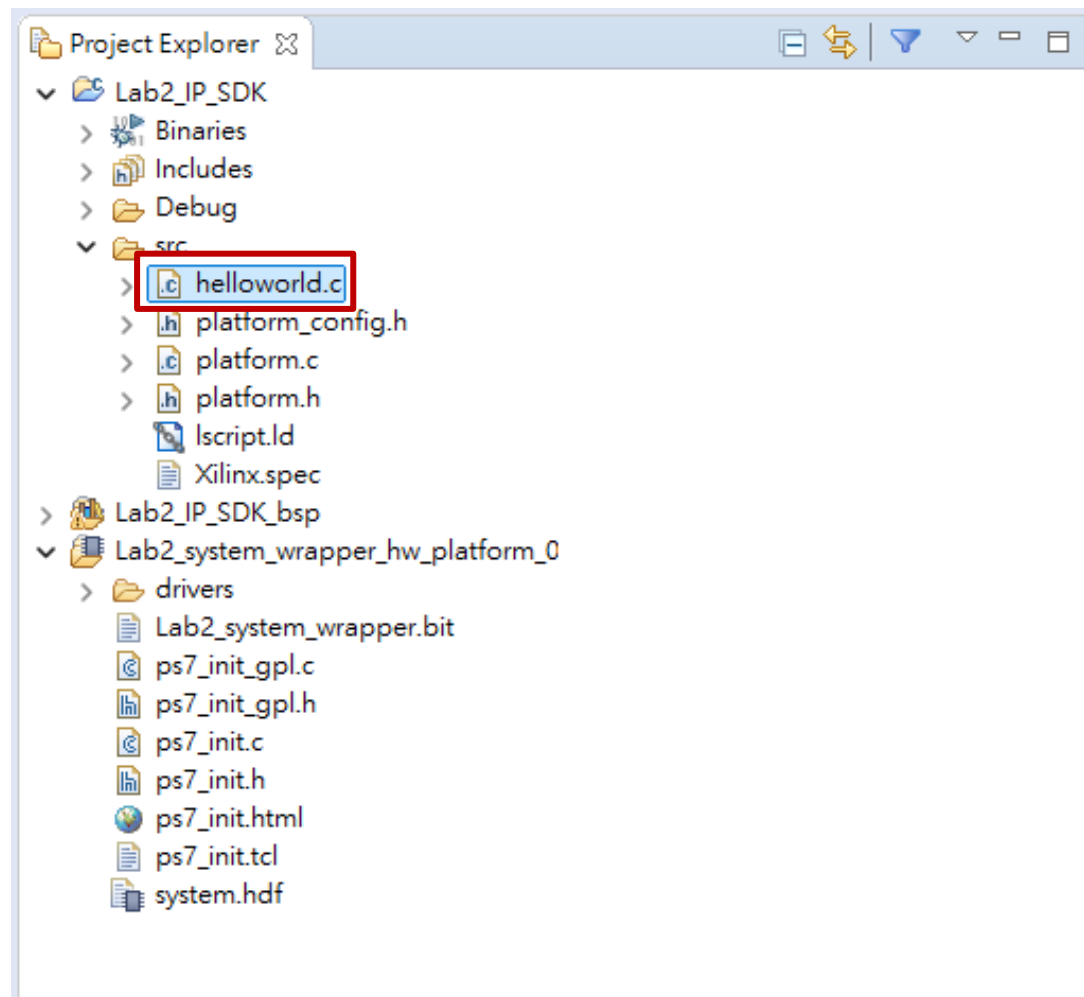
Overview

SDK操作

- ▶ 1.點選 File → Launch SDK → OK
- ▶ 2.點選 File → New → Application Project
- ▶ 3.建立 SDK專案 → Next
- ▶ 4.選取 Hello World → Finish
- ▶ 5.開啟 system.hdf 找到我們 IP 的 base address

(在一個數位系統中，周邊硬體通常會被定址到一個記憶體區間讓開發人員進行暫存器的讀寫)

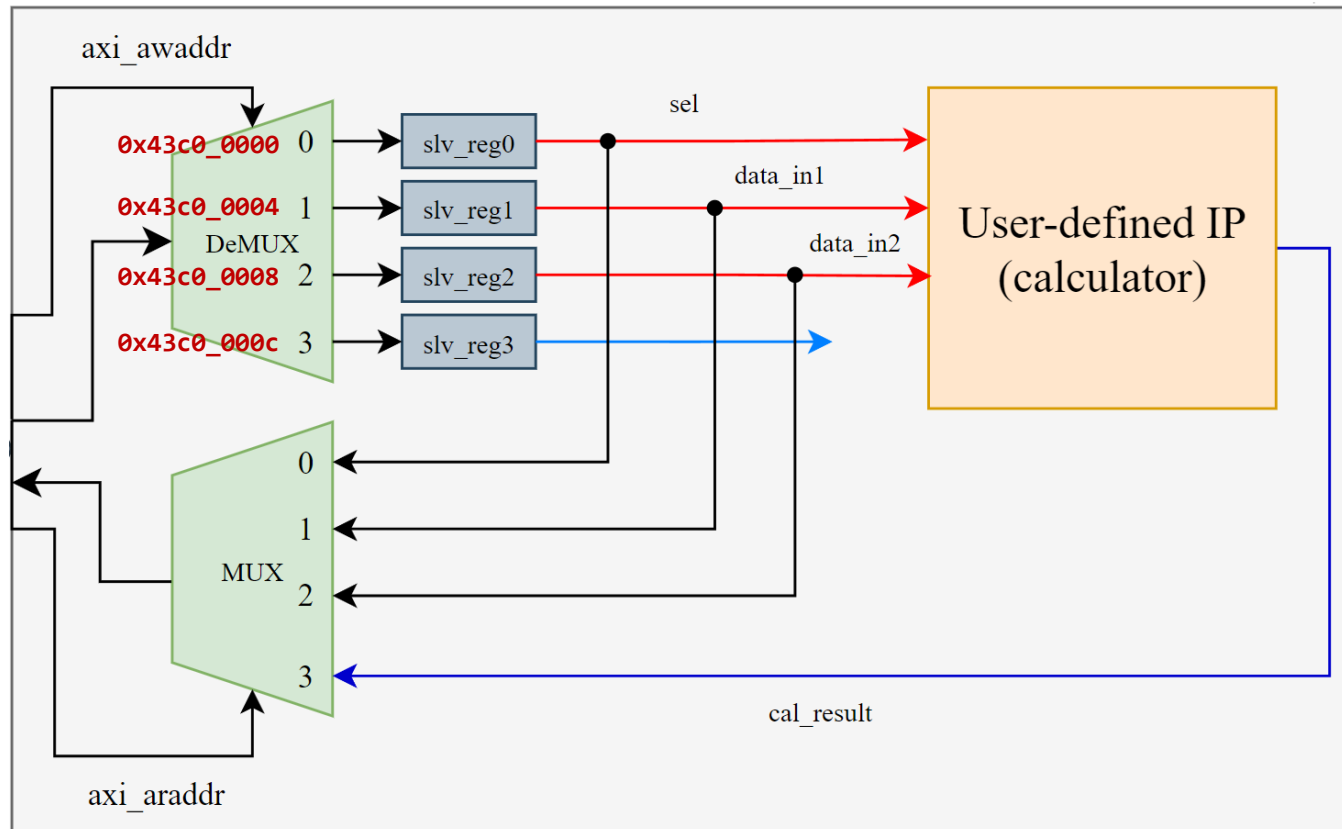
- ▶ 6.點選 Lab2_IP_SDK → src →
helloworld.c 編輯主程式



Part 1.2

- ▶ SDK 操作
- ▶ 撰寫 SDK 程式
- ▶ 燒錄 SDK 並執行結果

撰寫SDK程式



```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
```

```
#define slv_reg0 0
#define slv_reg1 1
#define slv_reg2 2
#define slv_reg3 3
#define slv_reg4 4
```

```
int main()
{
    init_platform();

    print("Hello World\n\r");

    volatile int *pointer = (int*)0x43c00000;

    // test ADD function
    printf("Test the function of ADD\n");
    // select signal
    pointer[slv_reg0] = 0;
    // calculated data
    pointer[slv_reg1] = 100;
    pointer[slv_reg2] = 200;

    printf("data_in1 = %d\n", pointer[1]);
    printf("data_in2 = %d\n", pointer[2]);
    printf("The ADD result is %d\n", pointer[3]);
    printf("-----\n");

    // test SUB function
    printf("Test the function of SUB\n");
    // select signal
    pointer[slv_reg0] = 1;
    // calculated data
    pointer[slv_reg1] = 100;
    pointer[slv_reg2] = 200;

    printf("data_in1 = %d\n", pointer[1]);
    printf("data_in2 = %d\n", pointer[2]);
    printf("The SUB result is %d\n", pointer[3]);
    printf("-----\n");

    // test MUL function
    printf("Test the function of MUL\n");
    // select signal
    pointer[slv_reg0] = 2;
    // calculated data
    pointer[slv_reg1] = 100;
    pointer[slv_reg2] = 200;

    printf("data_in1 = %d\n", pointer[1]);
    printf("data_in2 = %d\n", pointer[2]);
    printf("The MUL result is %d\n", pointer[3]);
    printf("-----\n");
}
```

Cell	Base Addr	High Addr
ps7_intc_dist_0	0xf8f01000	0xf8f01fff
ps7_gpio_0	0xe000a000	0xe000afff
ps7_ram_1	0xffff0000	0xffffdfff
ps7_ram_0	0x00000000	0x0002ffff
Lab2_IP_0	0x43c00000	0x43c0ffff

撰寫SDK程式

- ▶ 1.定義暫存器的偏移(offset)
- ▶ 2.定義指向 slave IP 記憶體位置的指標
- ▶ 3.將值寫入暫存器
- ▶ 4.print 輸出結果

(1)

```
#define slv_reg0 0
#define slv_reg1 1
#define slv_reg2 2
#define slv_reg3 3
#define slv_reg4 4
```

(2)

```
volatile int *pointer = (int*)0x43c00000;
```

(3)

```
// test ADD function
printf("Test the function of ADD\n");
// select signal
pointer[slv_reg0] = 0;
// calculated data
pointer[slv_reg1] = 100;
pointer[slv_reg2] = 200;
```

(4)

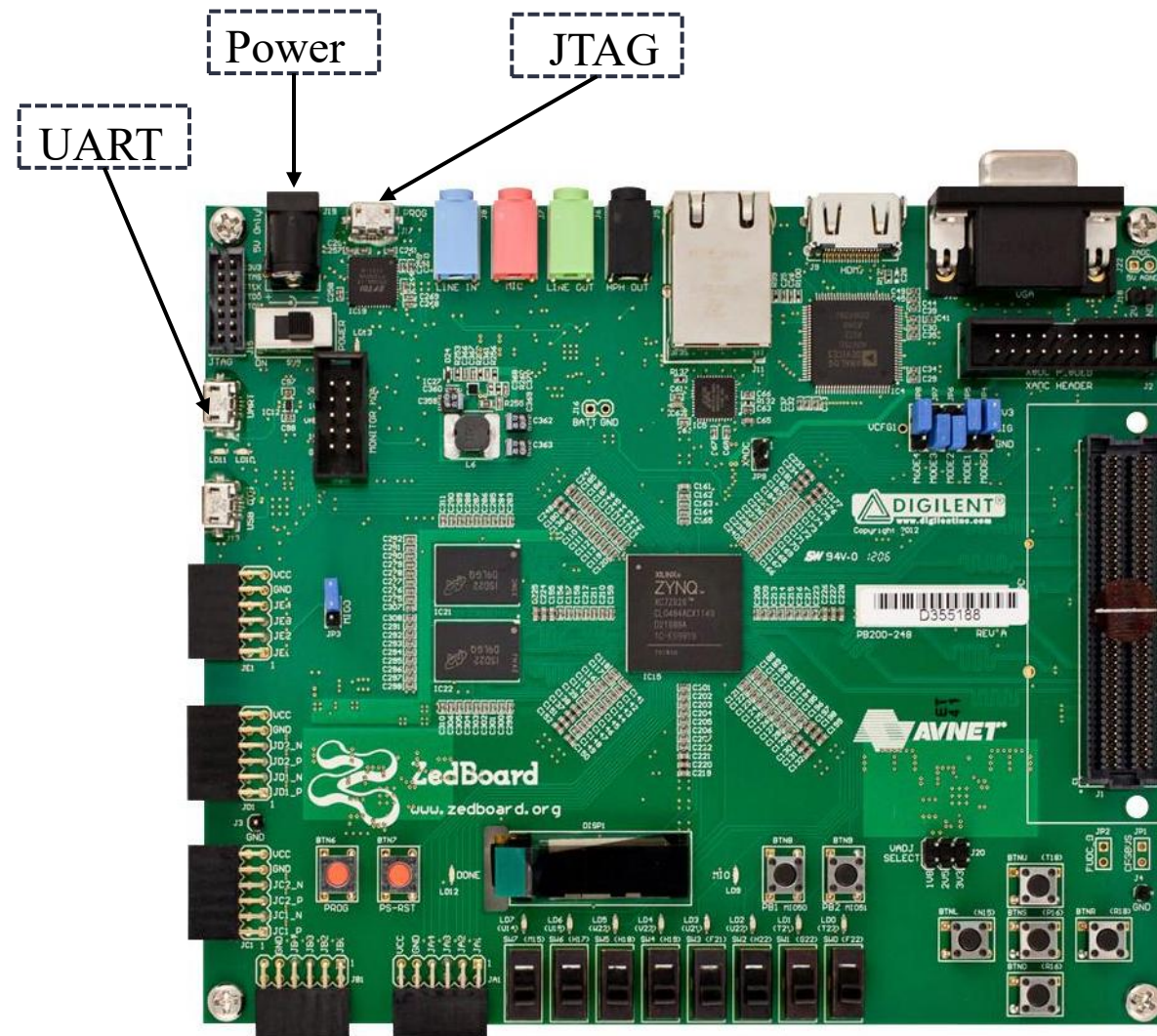
```
printf("data_in1 = %d\n" ,pointer[1]);
printf("data_in2 = %d\n" ,pointer[2]);
printf("The ADD result is %d\n" ,pointer[3]);
printf("-----\n");
```

Part 1.2

- ▶ SDK 操作
- ▶ 撰寫 SDK 程式
- ▶ 燒錄 SDK 並執行結果

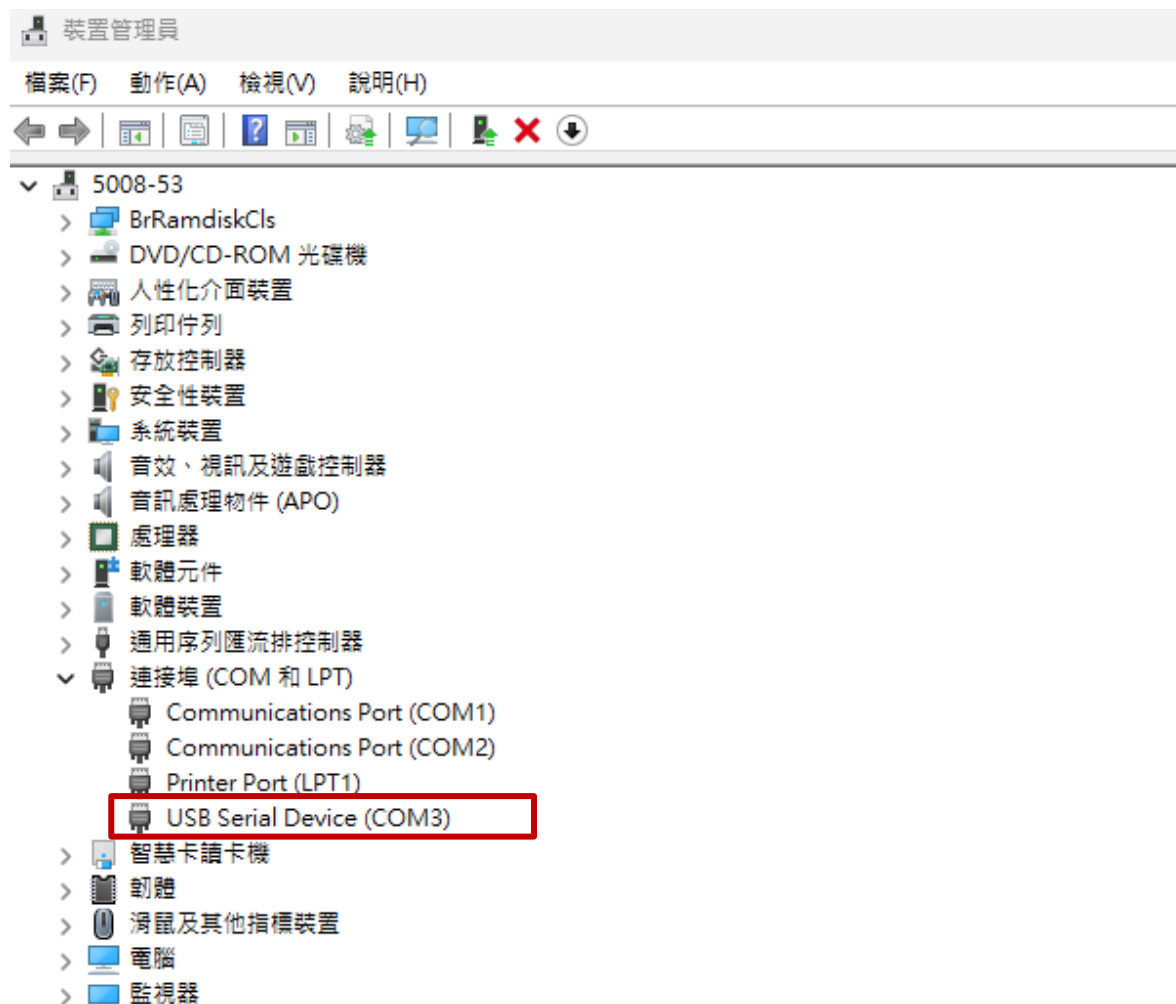
燒錄SDK並執行結果

- 1. 確認 C 程式完成後，將 FPGA 接好開啟電源



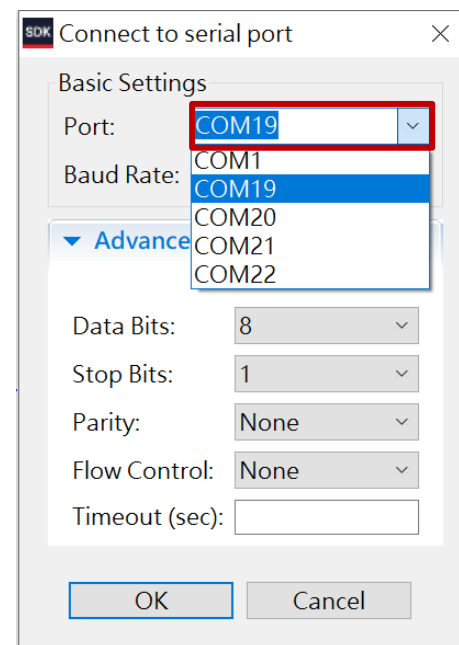
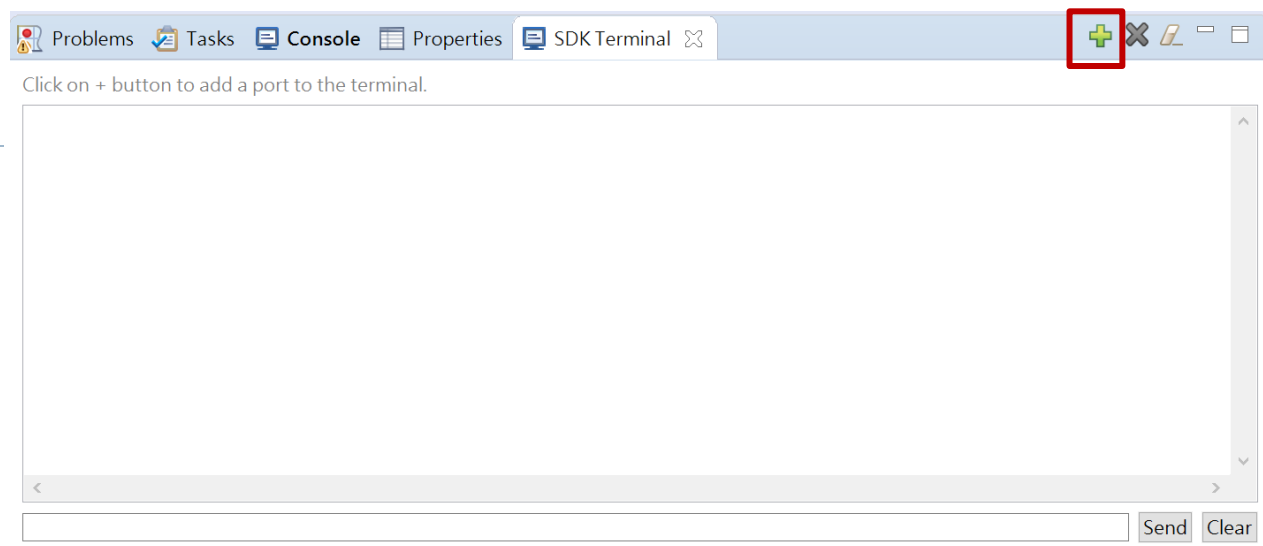
燒錄SDK並執行結果

- ▶ 1.確認 C 程式完成後，將 FPGA 接好開啟電源
- ▶ 2.裝置管理員中尋找 COM 腳



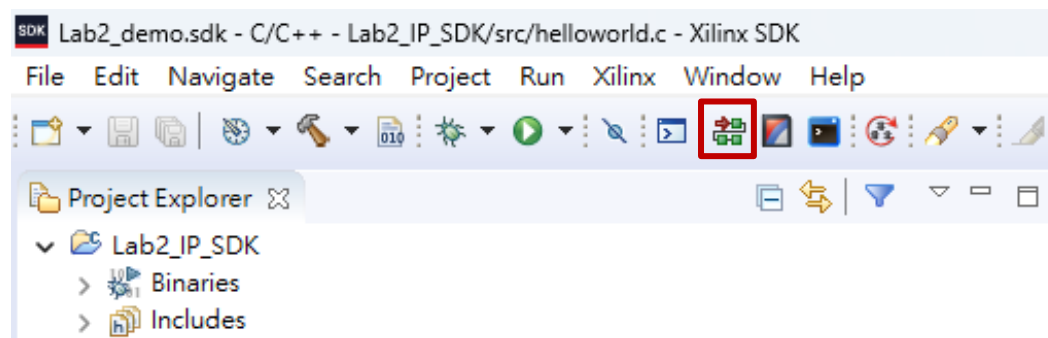
燒錄SDK並執行結果

- ▶ 1.確認 C 程式完成後，將 FPGA 接好開啟電源
- ▶ 2.裝置管理員中尋找 COM 腳
- ▶ 3.點選下面 SDK Terminal  按鈕，選取 COM 腳



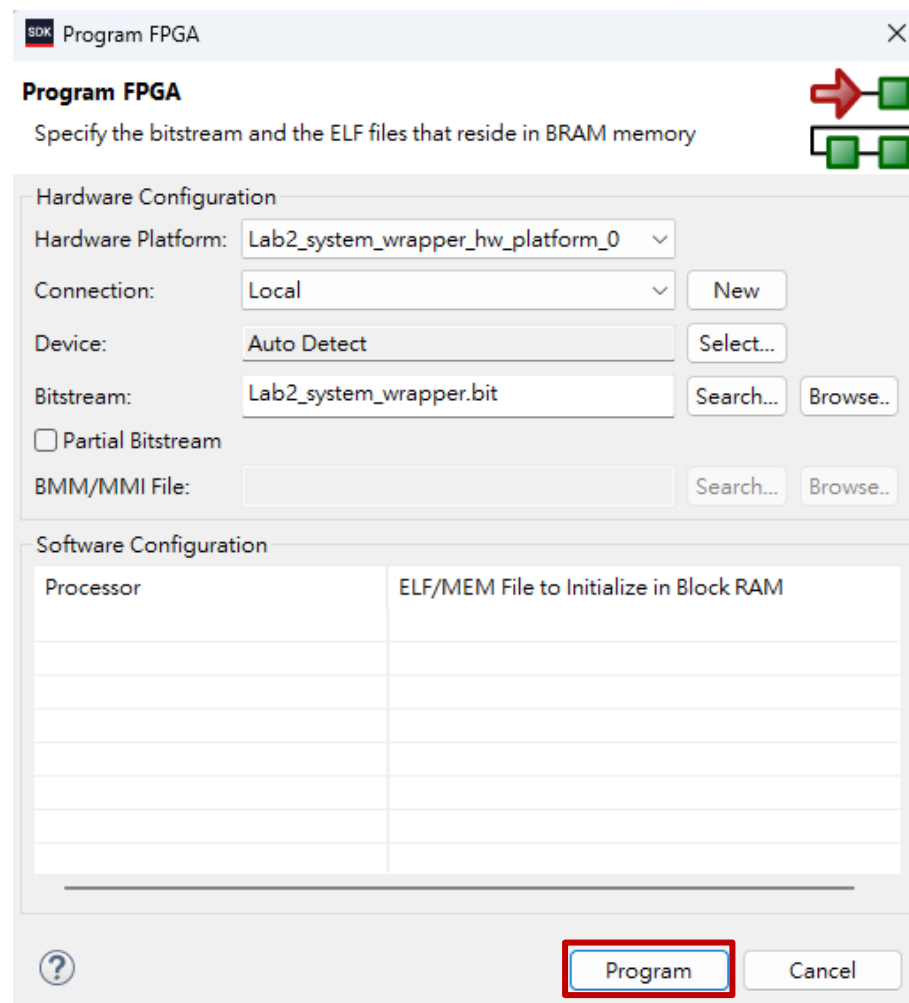
燒錄SDK並執行結果

- ▶ 1.確認 C 程式完成後，將 FPGA 接好開啟電源
- ▶ 2.裝置管理員中尋找 COM 腳
- ▶ 3.點選下面 SDK Terminal  按鈕，選取 COM 腳
- ▶ 4.點選  開啟 Program 介面



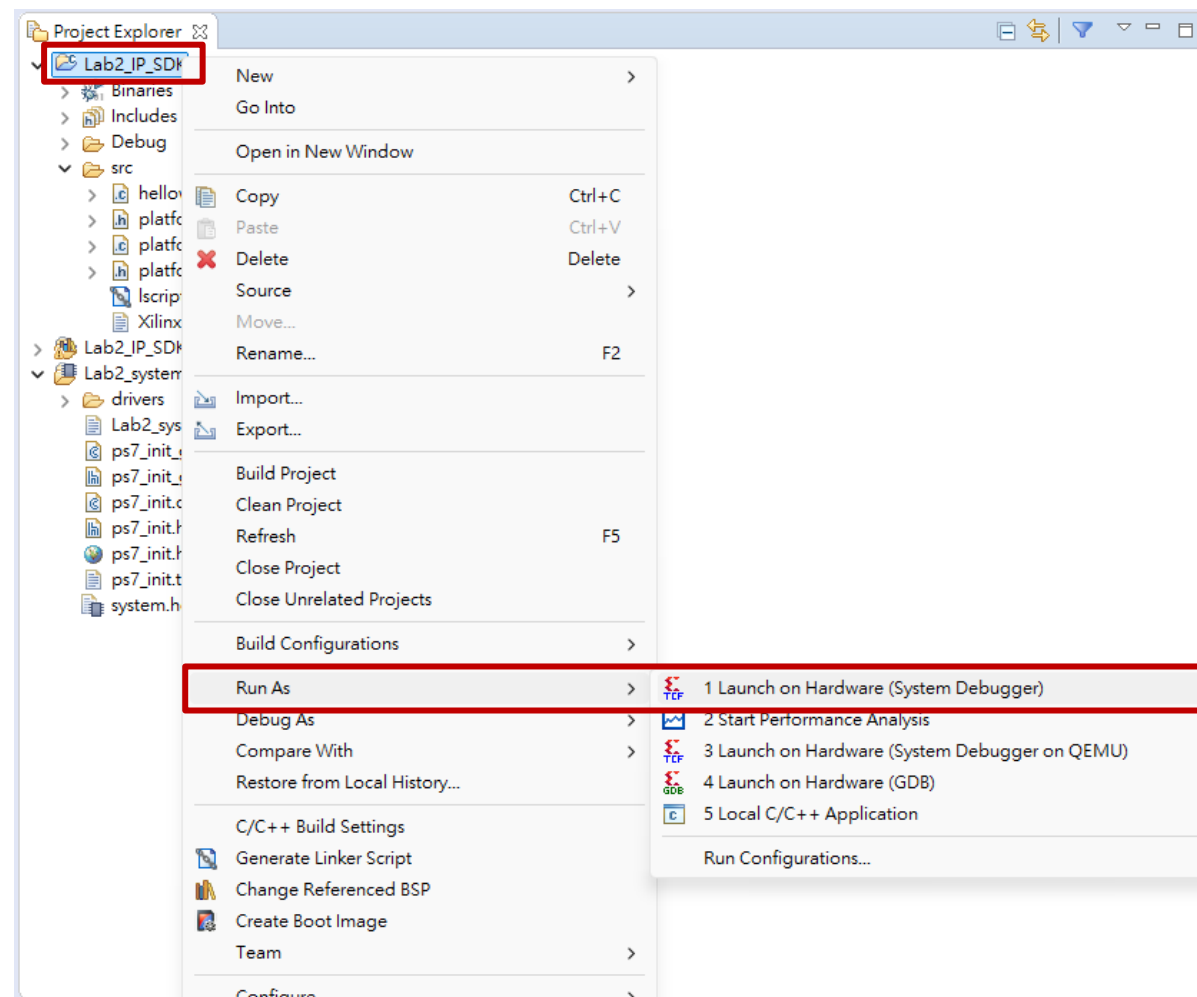
燒錄SDK並執行結果

- ▶ 1.確認 C 程式完成後，將 FPGA 接好開啟電源
- ▶ 2.裝置管理員中尋找 COM 腳
- ▶ 3.點選下面 SDK Terminal  按鈕，選取 COM 腳
- ▶ 4.點選  開啟 Program 介面
- ▶ 5.點選 Program 並等待



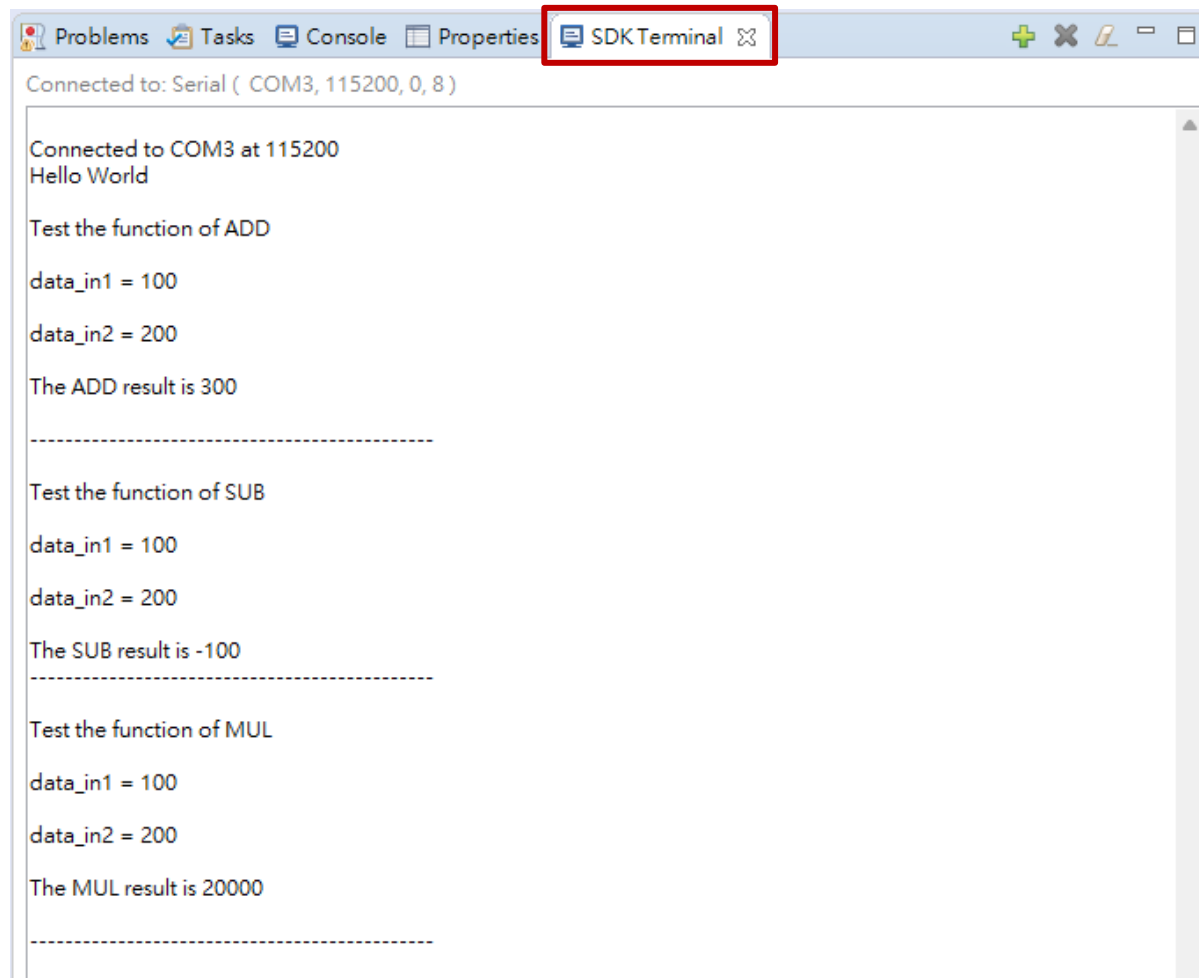
燒錄SDK並執行結果

- ▶ 1.確認 C 程式完成後，將 FPGA 接好開啟電源
- ▶ 2.裝置管理員中尋找 COM 腳
- ▶ 3.點選下面 SDK Terminal  按鈕，選取 COM 腳
- ▶ 4.點選  開啟 Program 介面
- ▶ 5.點選 Program 並等待
- ▶ 6.點選 Lab2_IP_SDK 右鍵 → Run As → Launch on Hardware(System Debugger)



燒錄SDK並執行結果

- ▶ 1.確認 C 程式完成後，將 FPGA 接好開啟電源
- ▶ 2.裝置管理員中尋找 COM 腳
- ▶ 3.點選下面 SDK Terminal  按鈕，選取 COM 腳
- ▶ 4.點選  開啟 Program 介面
- ▶ 5.點選 Program 並等待
- ▶ 6.點選 Lab2_IP_SDK 右鍵 → Run As → Launch on Hardware(System Debugger)
- ▶ 7.確認 SDK Terminal 跑出的結果



```
Connected to: Serial ( COM3, 115200, 0, 8 )

Connected to COM3 at 115200
Hello World

Test the function of ADD

data_in1 = 100
data_in2 = 200
The ADD result is 300
-----

Test the function of SUB

data_in1 = 100
data_in2 = 200
The SUB result is -100
-----

Test the function of MUL

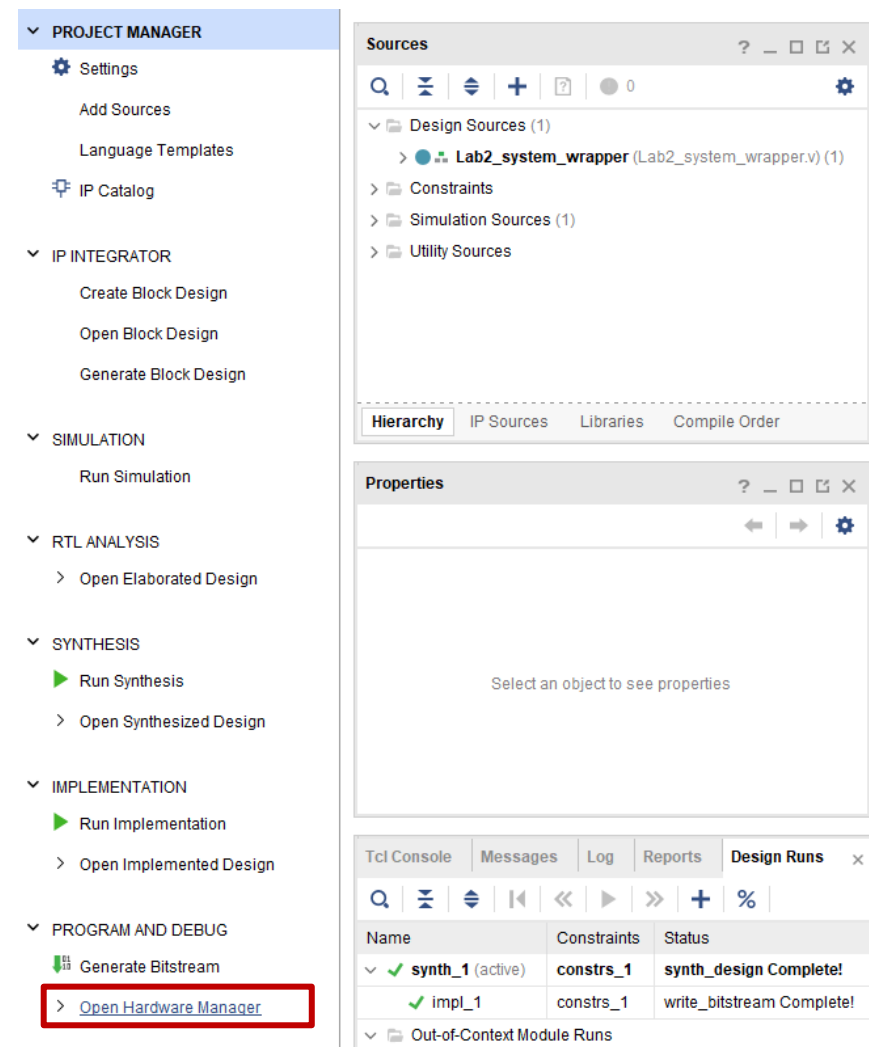
data_in1 = 100
data_in2 = 200
The MUL result is 20000
-----
```

Part 1.3

- ▶ 使用 ILA Core進行 Debug

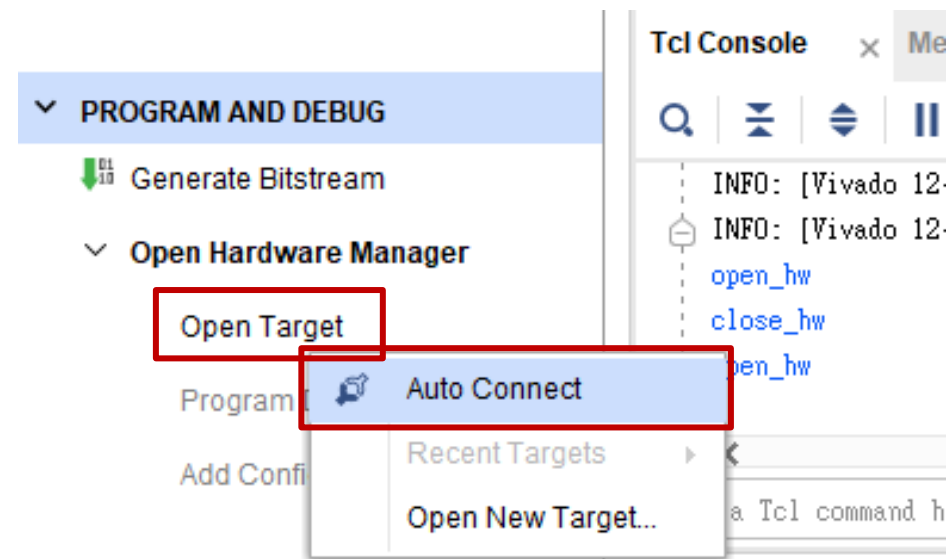
使用ILA Core進行Debug

▶ 1.在 Vivado 介面點選 Open Hardware Manager



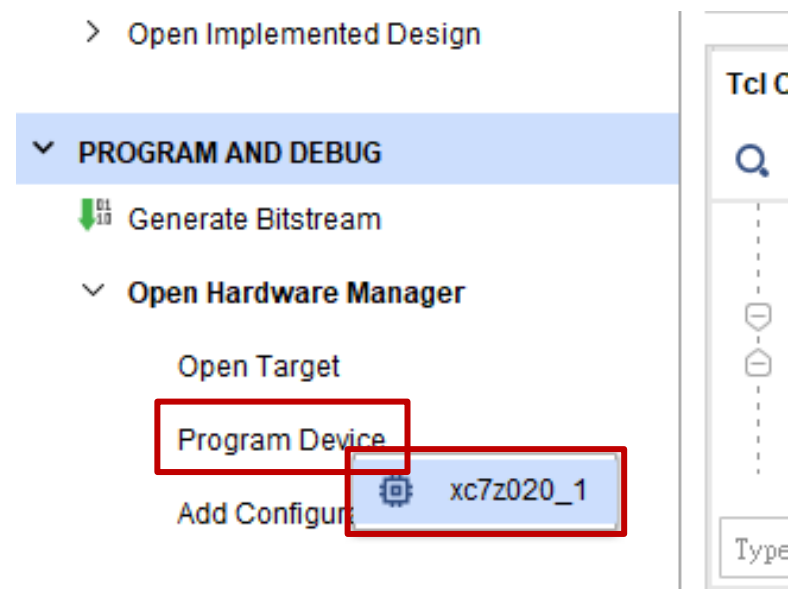
使用ILA Core進行Debug

- ▶ 1.在 Vivado 介面點選 Open Hardware Manager
- ▶ 2.點選 Open Target → Auto Connect



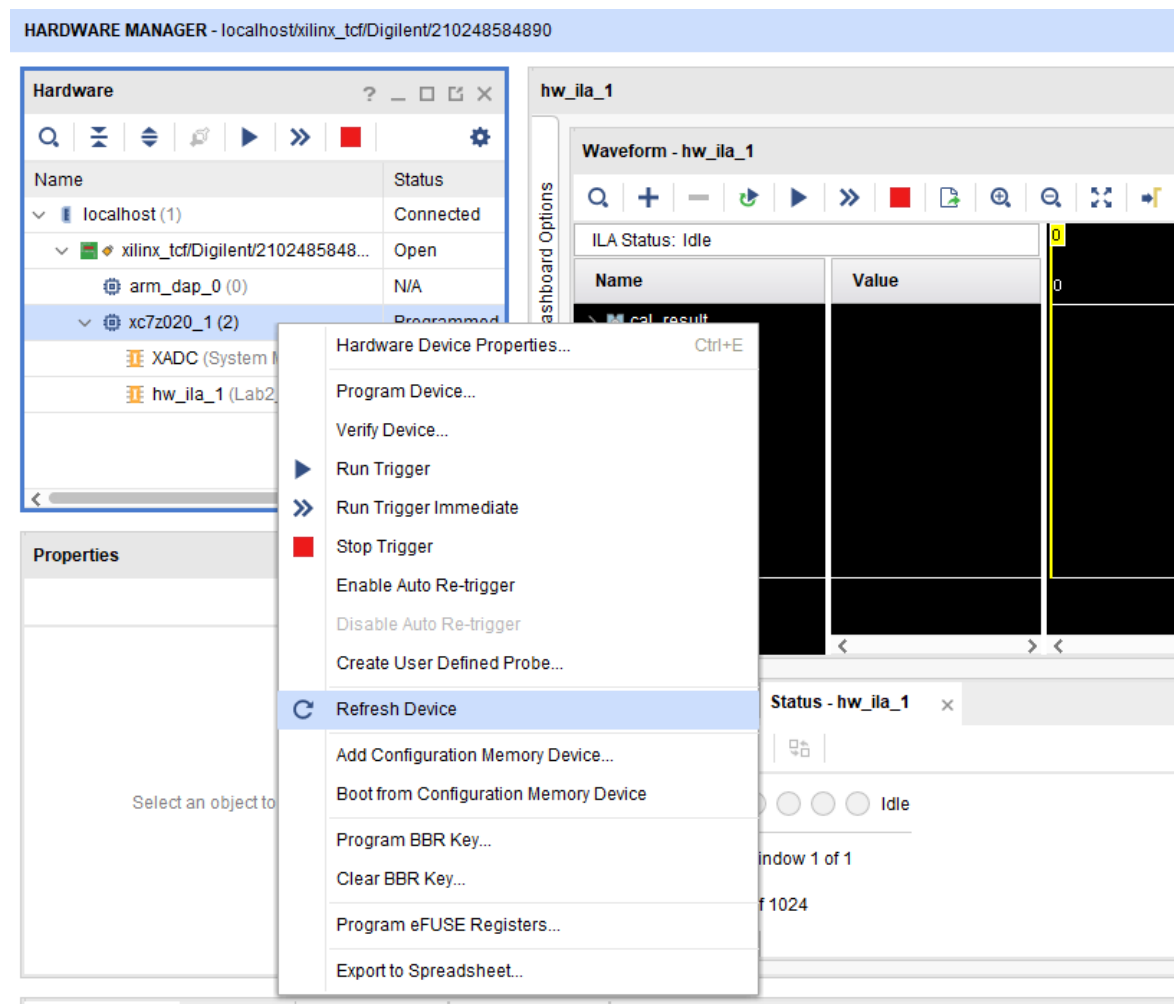
使用ILA Core進行Debug

- ▶ 1.在 Vivado 介面點選 Open Hardware Manager
- ▶ 2.點選 Open Target → Auto Connect
- ▶ 3.點選 Program Device → 選擇板子型號
(xc7z020_1)



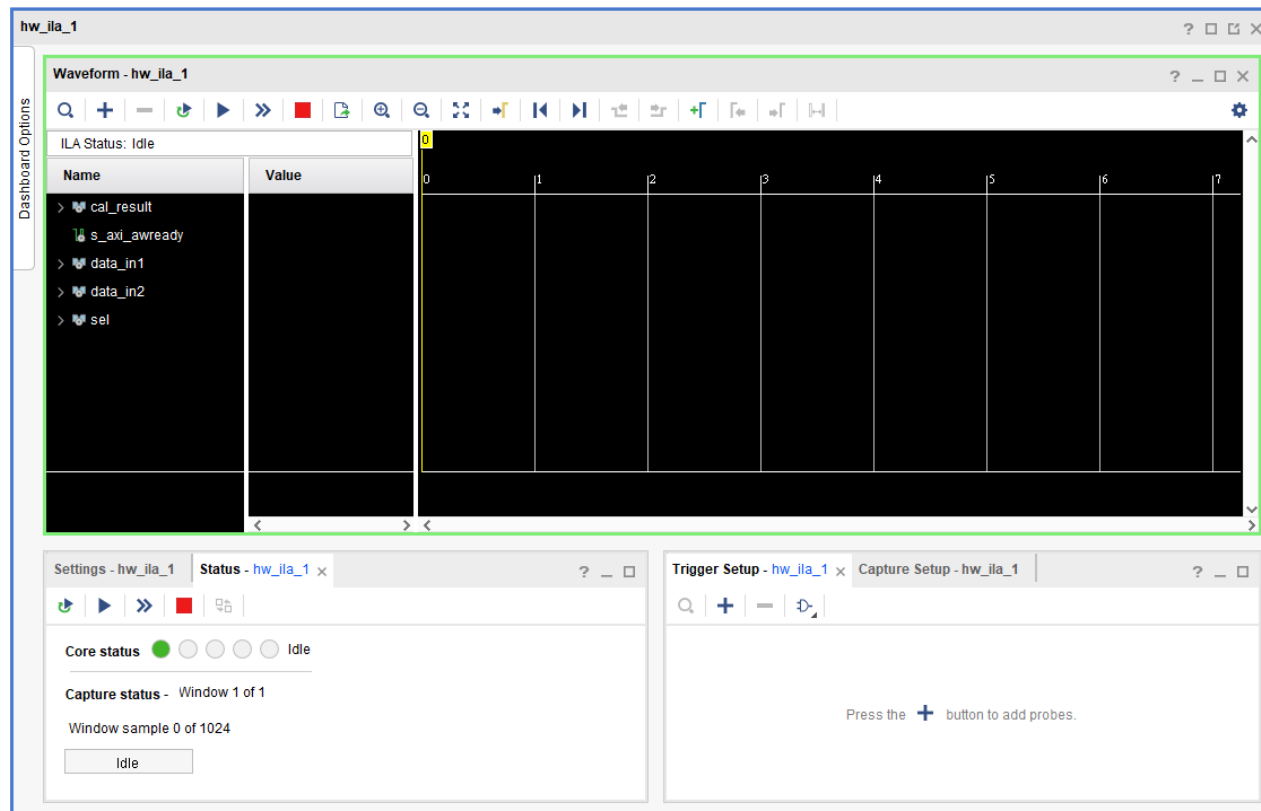
使用ILA Core進行Debug

- ▶ 1.在 Vivado 介面點選 Open Hardware Manager
- ▶ 2.點選 Open Target → Auto Connect
- ▶ 3.點選 Program Device → 選擇板子型號 (xc7z020_1)
- ▶ 4.如果 ILA 沒有顯示的話，點選 Refresh Device



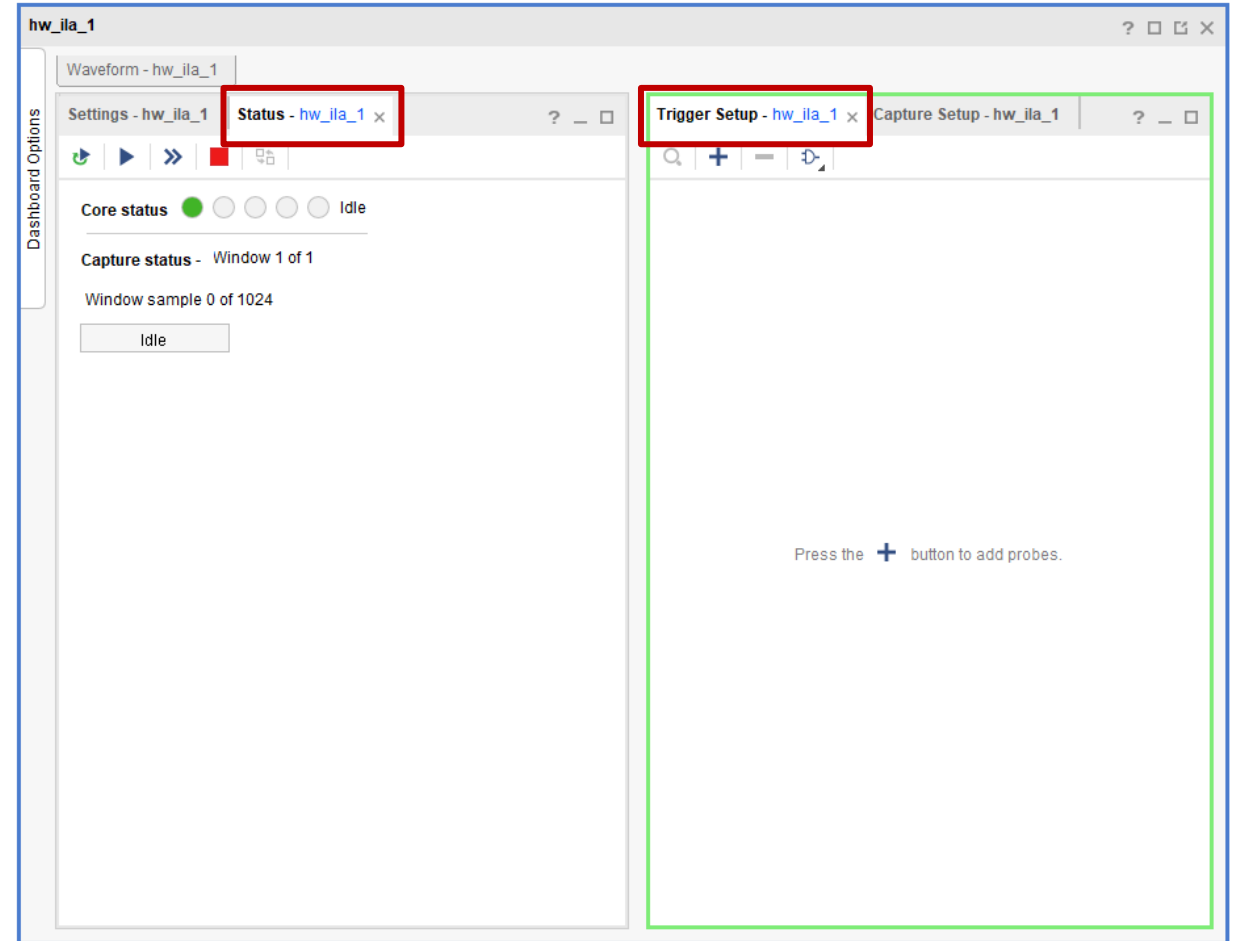
使用ILA Core進行Debug

- ▶ 1.在 Vivado 介面點選 Open Hardware Manager
- ▶ 2.點選 Open Target → Auto Connect
- ▶ 3.點選 Program Device → 選擇板子型號 (xc7z020_1)
- ▶ 4.如果 ILA 沒有顯示的話，點選 Refresh Device
- ▶ 5.ILA screen 顯示



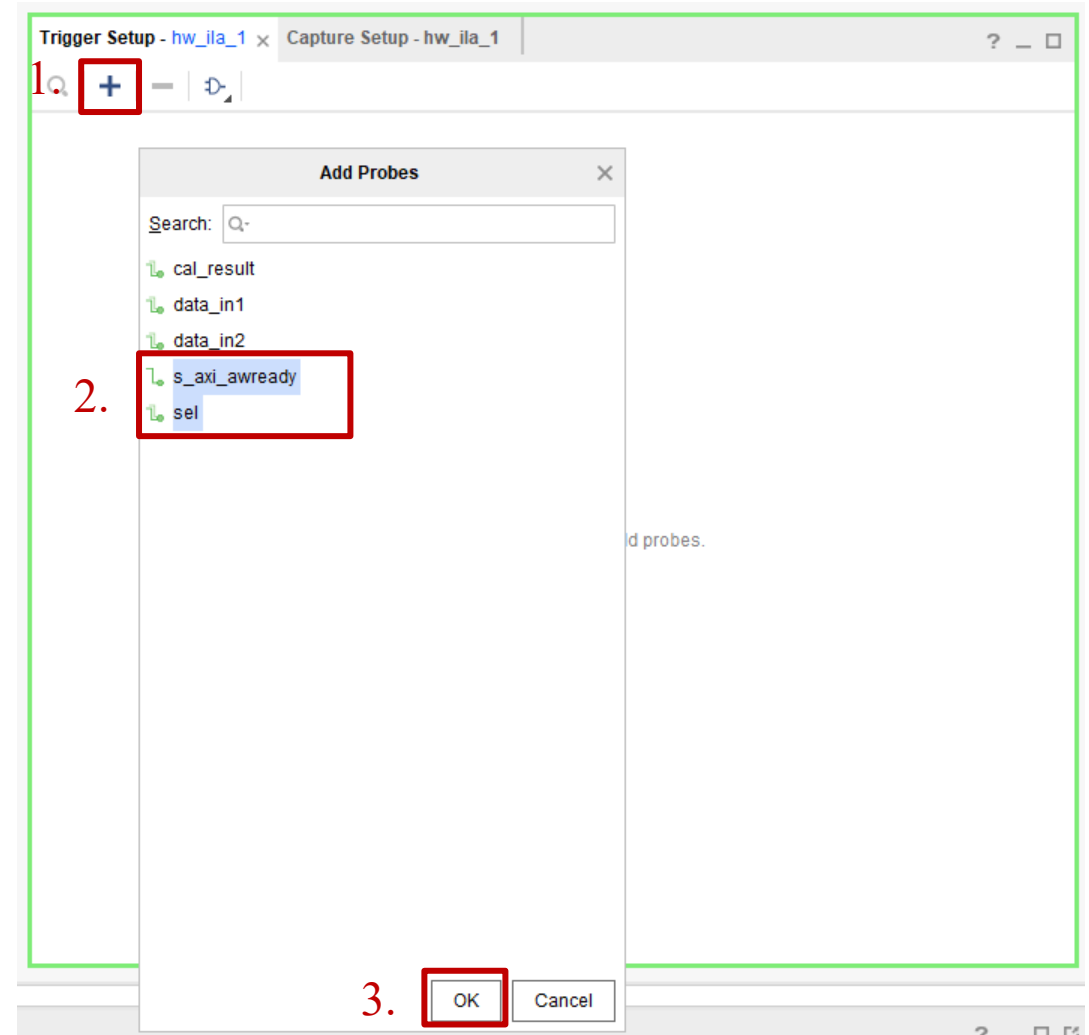
使用ILA Core進行Debug

- ▶ 1.在 Vivado 介面點選 Open Hardware Manager
- ▶ 2.點選 Open Target → Auto Connect
- ▶ 3.點選 Program Device → 選擇板子型號 (xc7z020_1)
- ▶ 4.如果 ILA 沒有顯示的話，點選 Refresh Device
- ▶ 5.ILA screen 顯示
- ▶ 6.設定 Trigger & data



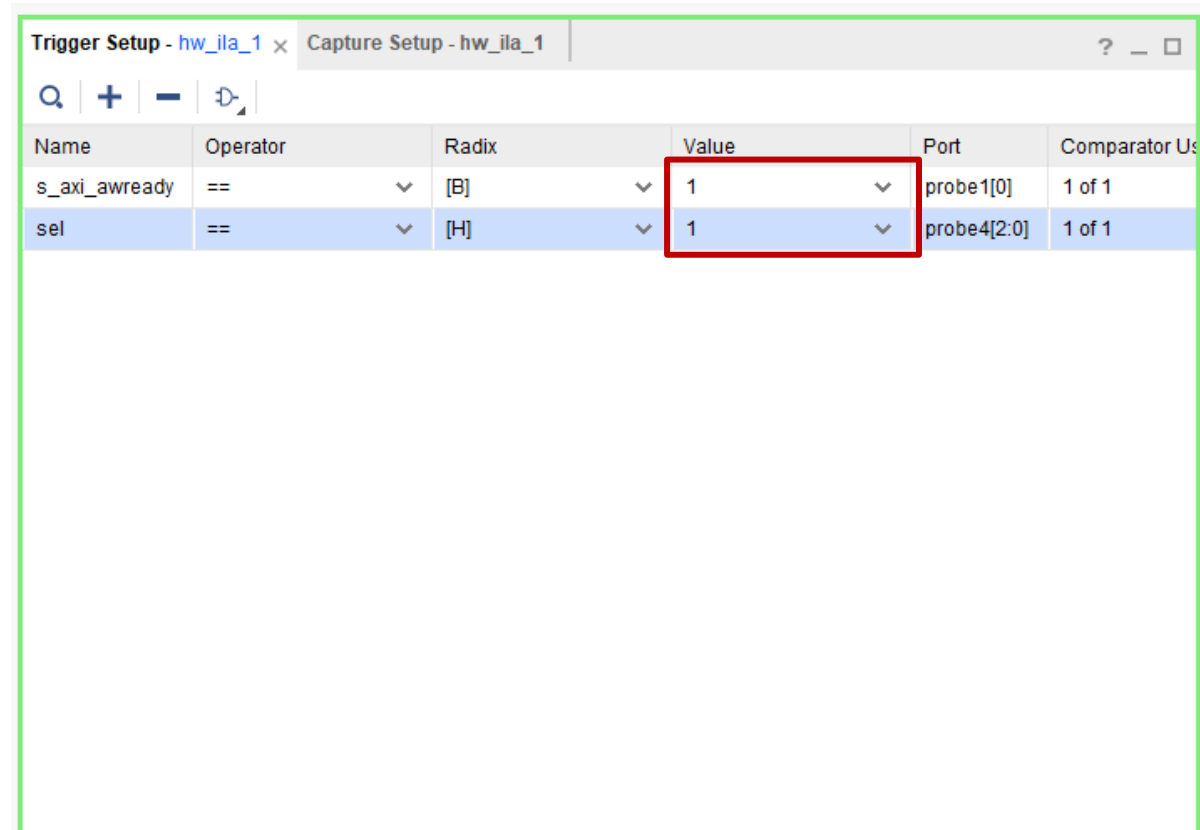
使用ILA Core進行Debug

- ▶ 1.在 Vivado 介面點選 Open Hardware Manager
- ▶ 2.點選 Open Target → Auto Connect
- ▶ 3.點選 Program Device → 選擇板子型號 (xc7z020_1)
- ▶ 4.如果 ILA 沒有顯示的話，點選 Refresh Device
- ▶ 5.ILA screen 顯示
- ▶ 6.設定 Trigger & data
- ▶ 7.點選 + ，並選擇 sel、s_axi_awready → OK



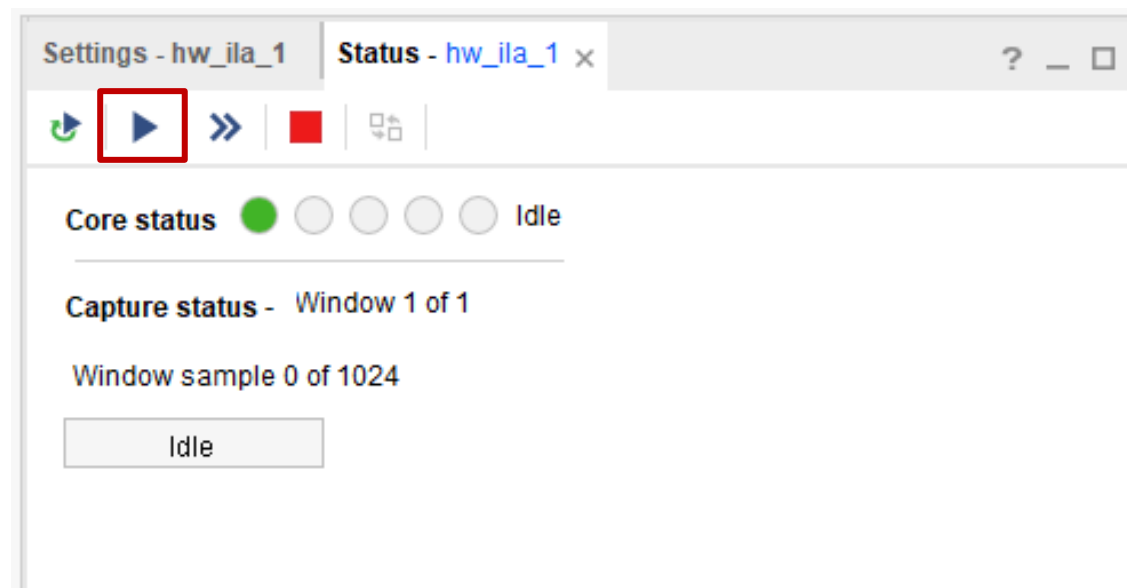
使用ILA Core進行Debug

- ▶ 1.在 Vivado 介面點選 Open Hardware Manager
- ▶ 2.點選 Open Target → Auto Connect
- ▶ 3.點選 Program Device → 選擇板子型號 (xc7z020_1)
- ▶ 4.如果 ILA 沒有顯示的話，點選 Refresh Device
- ▶ 5.ILA screen 顯示
- ▶ 6.設定 Trigger & data
- ▶ 7.點選 + ，並選擇 sel、s_axi_awready → OK
- ▶ 8.將 sel、s_axi_awready 的 value 設為 1 (sel 可以選擇不同的值)



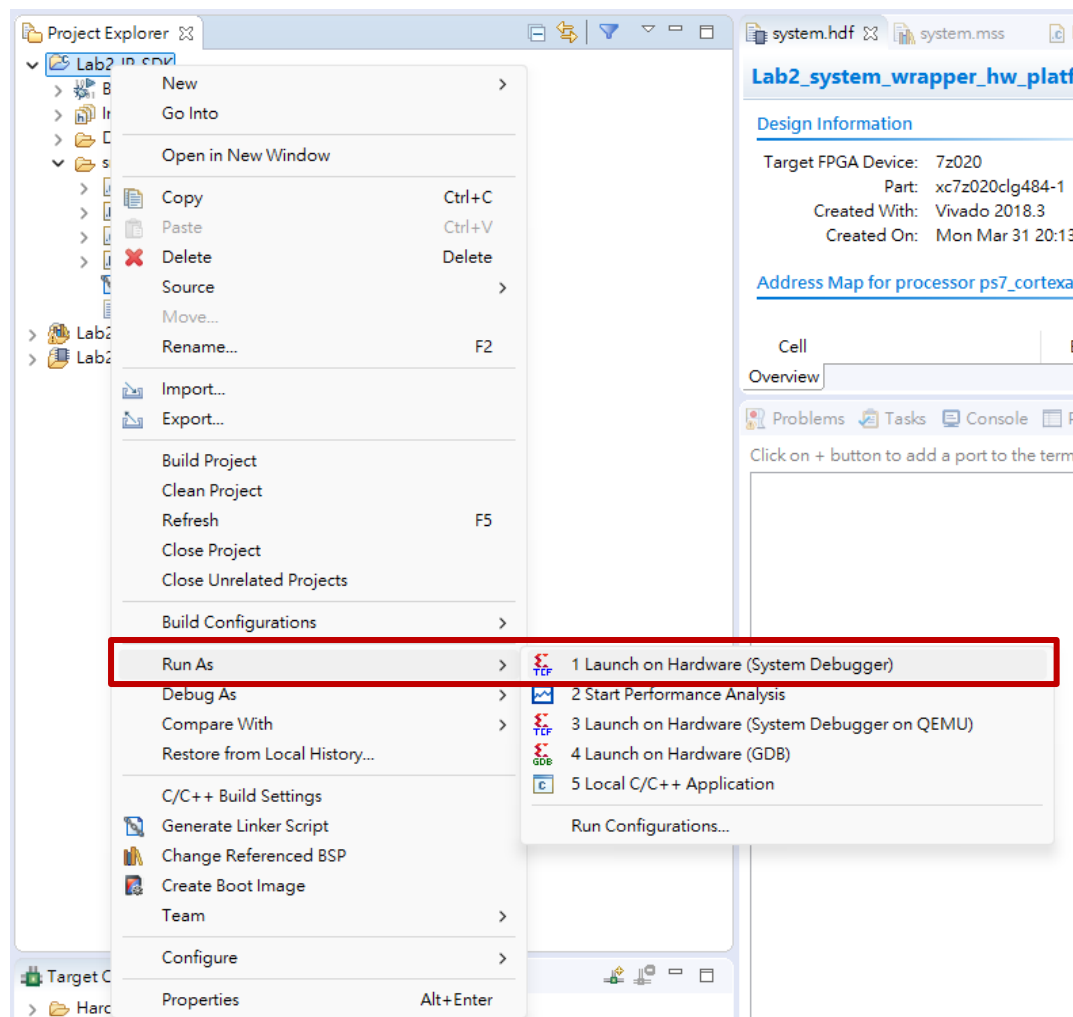
使用ILA Core進行Debug

▶ 9.點選 ▶



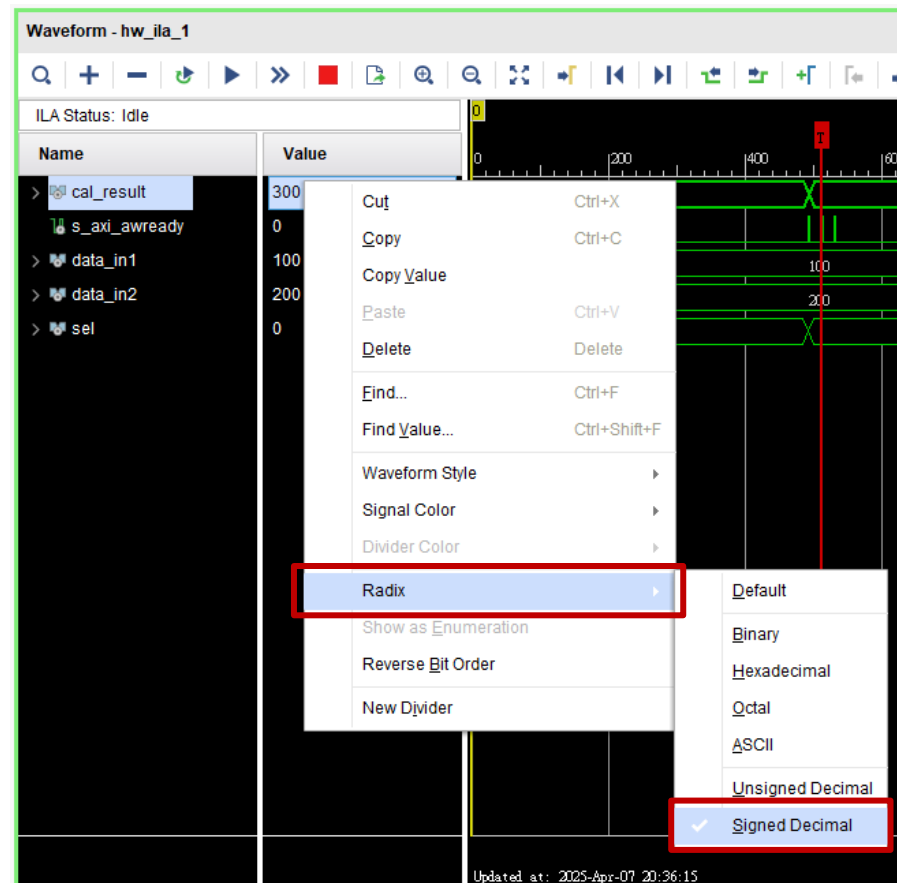
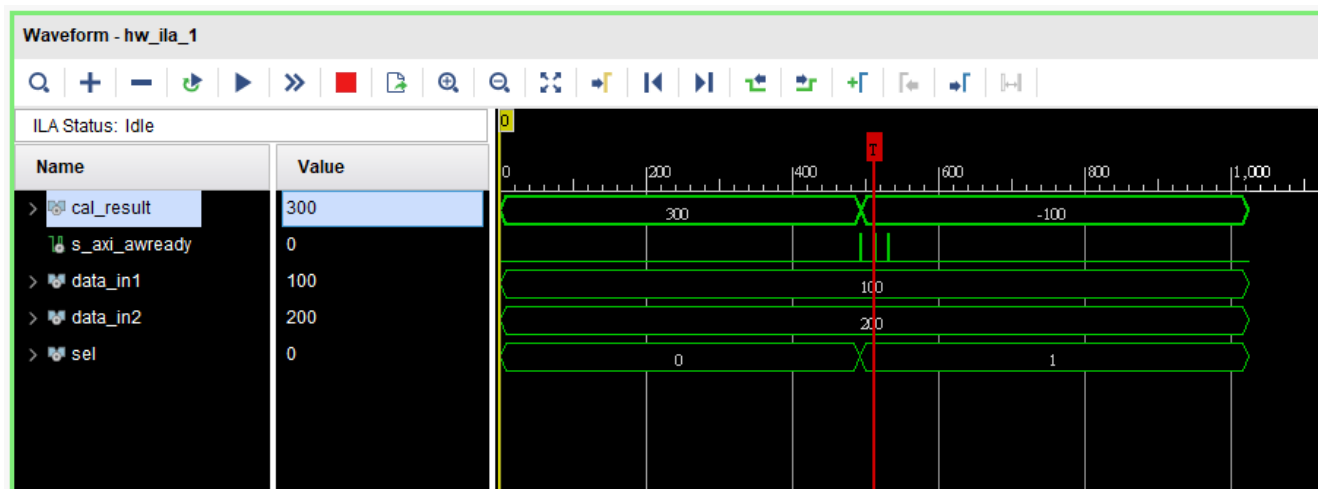
使用ILA Core進行Debug

- ▶ 9.點選 ▶
- ▶ 10.回到 SDK 的介面，點選 Run As → Launch Hardware(System Debugger)



使用ILA Core進行Debug

- ▶ 9.點選 ▶
- ▶ 10.回到 SDK 的介面，點選 Run As → Launch Hardware(System Debugger)
- ▶ 11.再回到 Vivado 介面即可以看到結果



Part 1.4

- ▶ 在前面的基礎上，新增第二個 Slave IP，透過不同的位址分別控制兩個 IP。

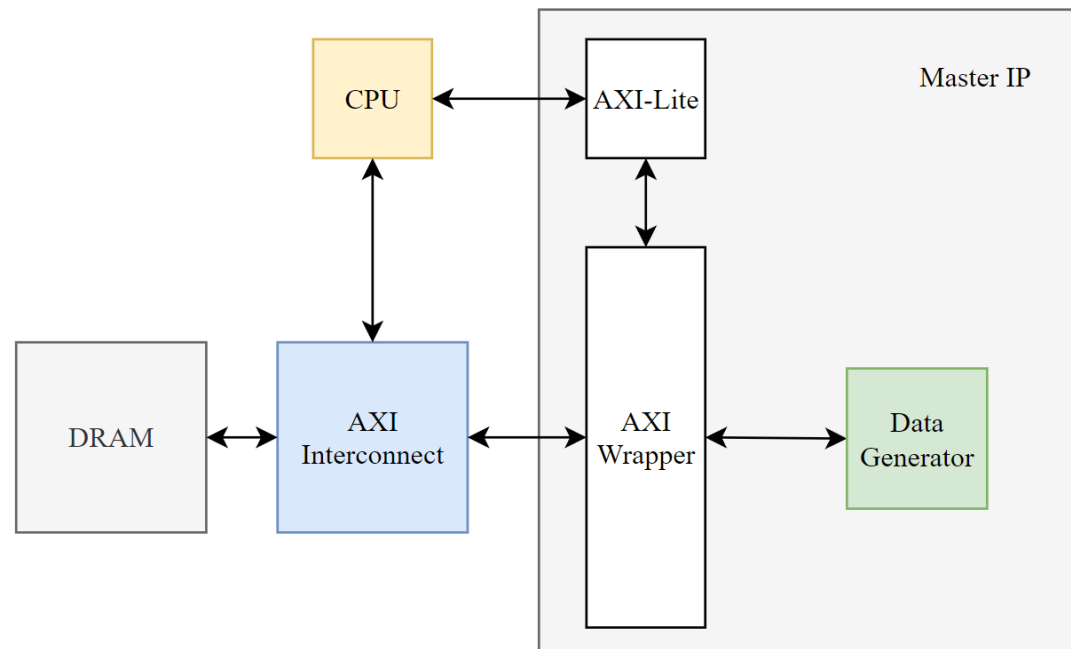
```
=== Slave 1 ===  
data_in1 = 100  
data_in2 = 200  
The ADD result is 300  
The SUB result is -100  
The MUL result is 20000  
  
=== Slave 2 ===  
data_in1 = 50  
data_in2 = 30  
The ADD result is 80  
The SUB result is 20  
The MUL result is 1500
```

Part 2 : 建立 AXI master 並與 CPU 進行溝通

- ▶ Overview of the master IP
- ▶ 建立 master IP
- ▶ 編輯 IP 內容
- ▶ 建立 Hardware IP
- ▶ SDK 操作

Overview of master IP

- ▶ 整個IP主要包含兩個block
 - ▶ AXI4-Lite Interface : 硬體IP與CPU溝通的介面。
 - ▶ AXI4-Full Interface : 硬體IP與DRAM讀寫資料的介面。
- ▶ IP的功能：進行16筆資料的讀寫
 - ▶ CPU發起start訊號給Master IP，Master IP再到指定位置寫入資料(這個address被定義在Master template中)。
 - ▶ 資料寫入完成後，Master IP會到該位置讀取資料並與寫入資料進行比對，完成後會回傳done訊號與error訊號。
 - ▶ 可以使用SDK透過CPU讀取寫入DRAM的值。

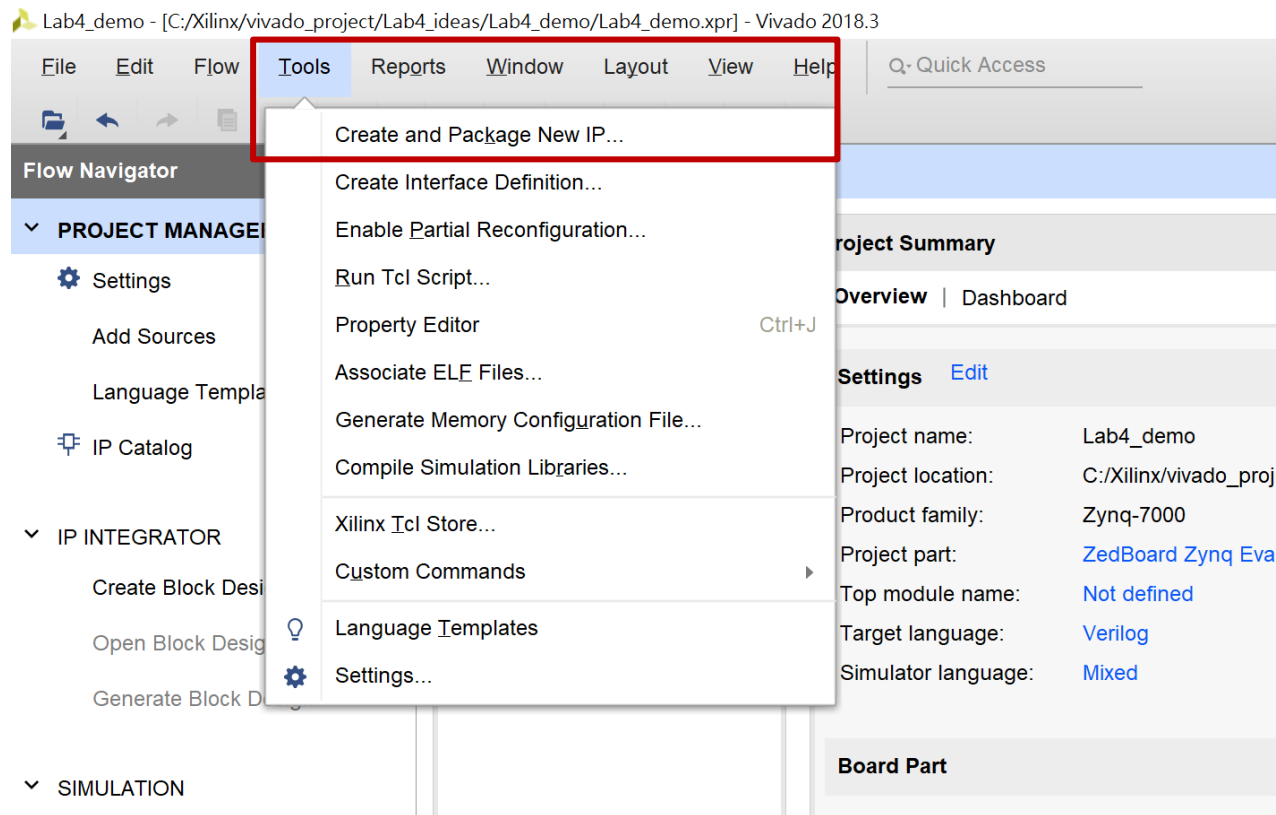


Part 2 : 建立 AXI master 並與 CPU 進行溝通

- ▶ Overview of the master IP
- ▶ 建立 master IP
- ▶ 編輯 IP 內容
- ▶ 建立 Hardware IP
- ▶ SDK 操作

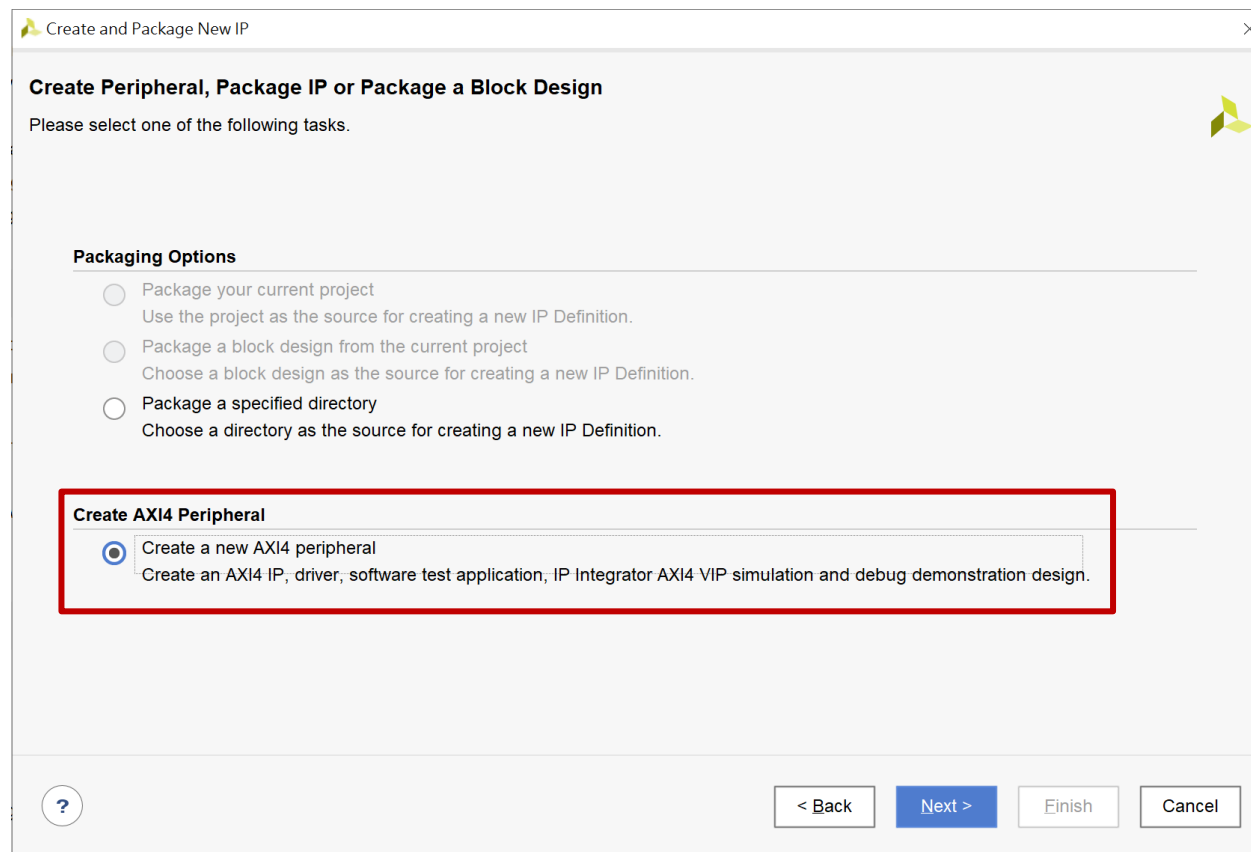
建立master IP

- ▶ 1.先建立一個新的專案
- ▶ 2.Tools → Create and Package New IP



建立master IP

- ▶ 1.先建立一個新的專案
- ▶ 2.Tools → Create and Package New IP
- ▶ 3.Next → Create a new AXI4 peripheral



建立master IP

- ▶ 1.先建立一個新的專案
- ▶ 2.Tools → Create and Package New IP
- ▶ 3.Next → Create a new AXI4 peripheral
- ▶ 4.修改Name → Next

Create and Package New IP

Peripheral Details
Specify name, version and description for the new peripheral

1.

Name: Lab2_master_ip

Version: 1.0

Display name: Lab2_master_ip_v1.0

Description: My new AXI IP

IP location: C:/Users/jam/Desktop/SOPC/lab2/project_2/..../ip_repo

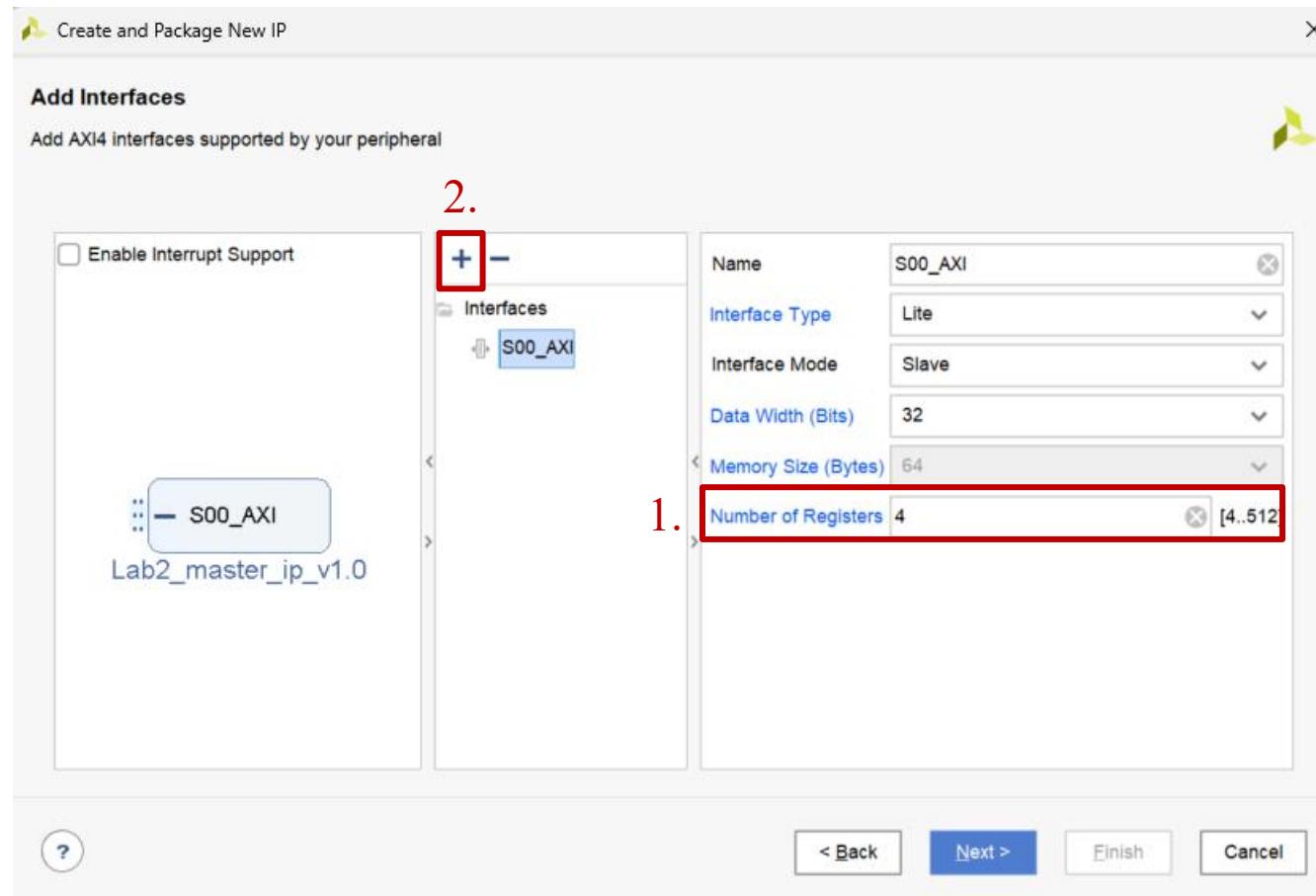
☐ Overwrite existing

2.

< Back Next > Finish Cancel

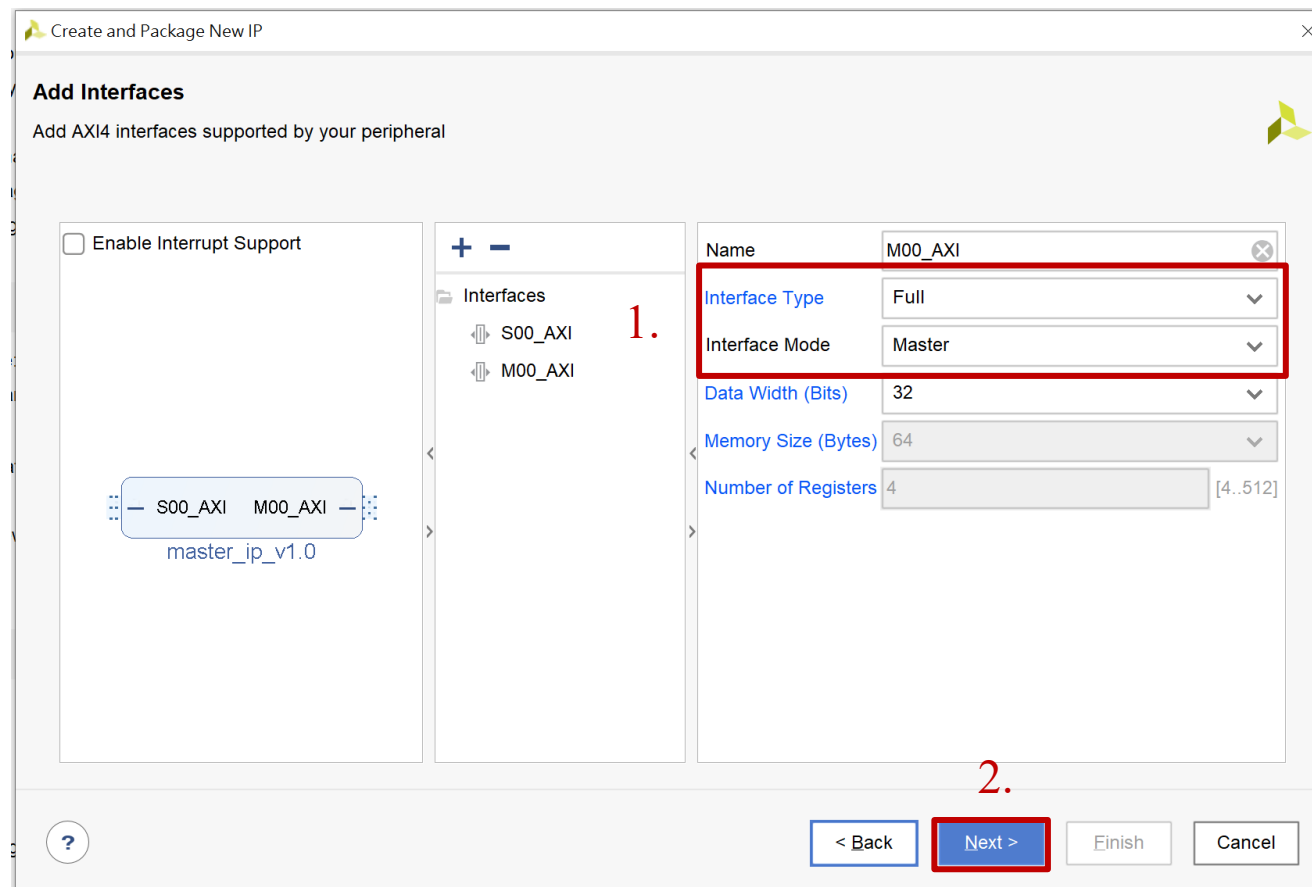
建立master IP

- ▶ 1.先建立一個新的專案
- ▶ 2.Tools → Create and Package New IP
- ▶ 3.Next → Create a new AXI4 peripheral
- ▶ 4.修改Name → Next
- ▶ 5.register數量選擇4 → 按「+」



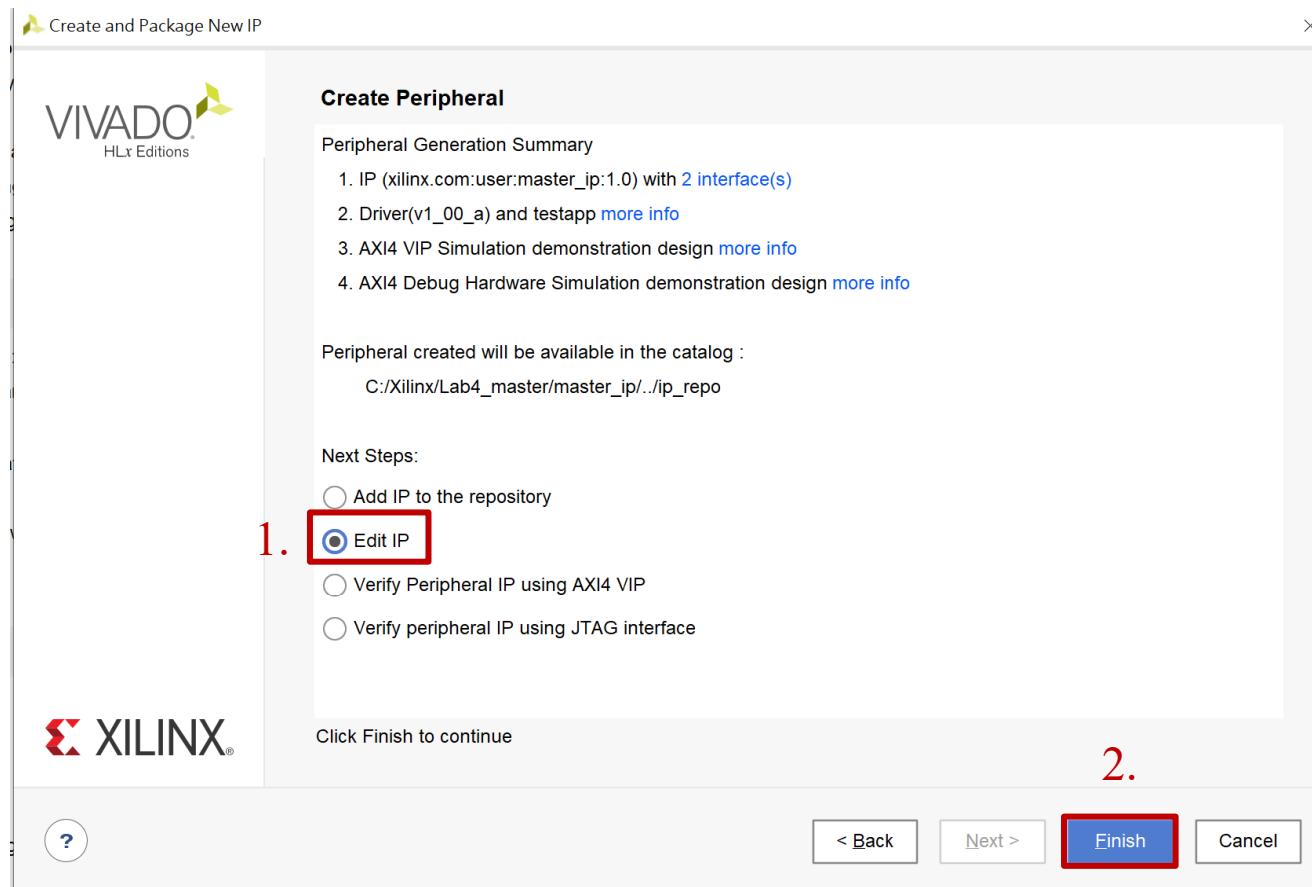
建立master IP

- ▶ 1.先建立一個新的專案
- ▶ 2.Tools → Create and Package New IP
- ▶ 3.Next → Create a new AXI4 peripheral
- ▶ 4.修改Name → Next
- ▶ 5.register數量選擇4 → 按「+」
- ▶ 6.Interface Type 選擇Full ， Interface Mode 選擇Master



建立master IP

- ▶ 1.先建立一個新的專案
- ▶ 2.Tools → Create and Package New IP
- ▶ 3.Next → Create a new AXI4 peripheral
- ▶ 4.修改Name → Next
- ▶ 5.register數量選擇4 → 按「+」
- ▶ 6.Interface Type 選擇Full ，Interface Mode 選擇Master → Next
- ▶ 7.Edit IP → Finish

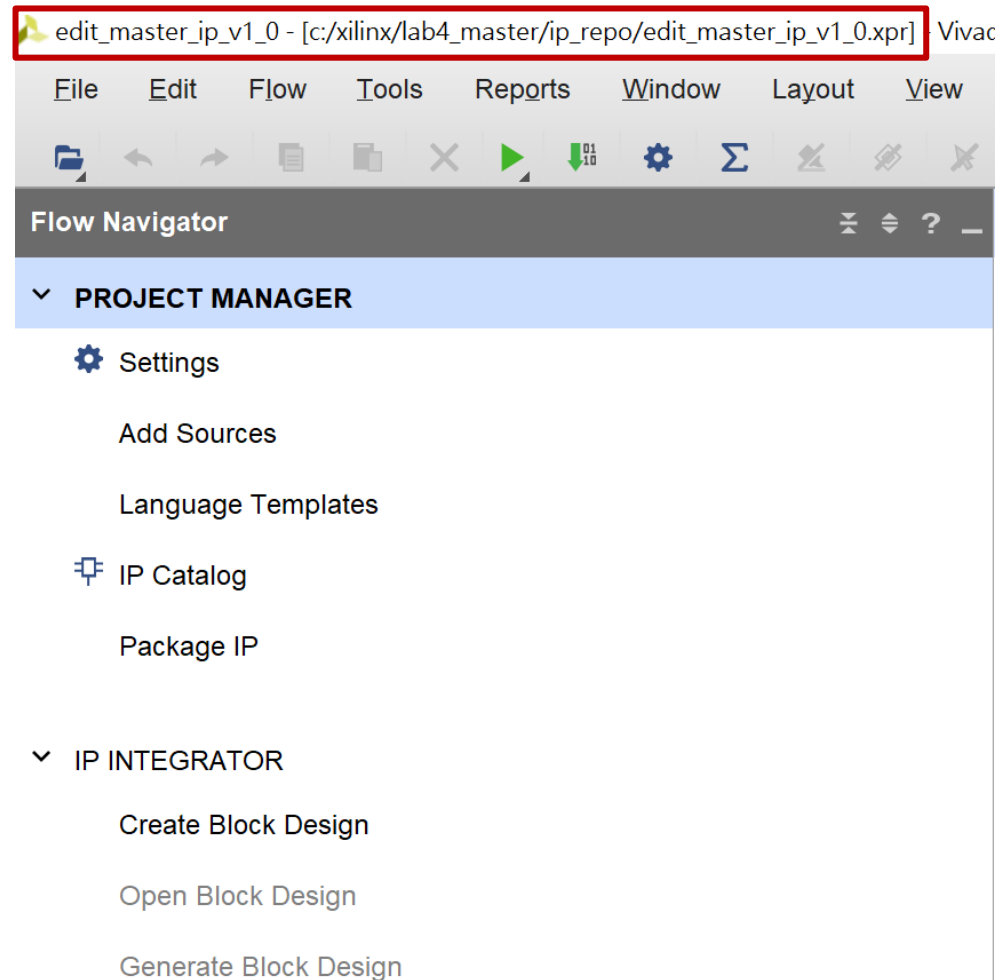


Part 2 : 建立 AXI master 並與 CPU 進行溝通

- ▶ Overview of the master IP
- ▶ 建立 master IP
- ▶ 編輯 IP 內容
- ▶ 建立 Hardware IP
- ▶ SDK 操作

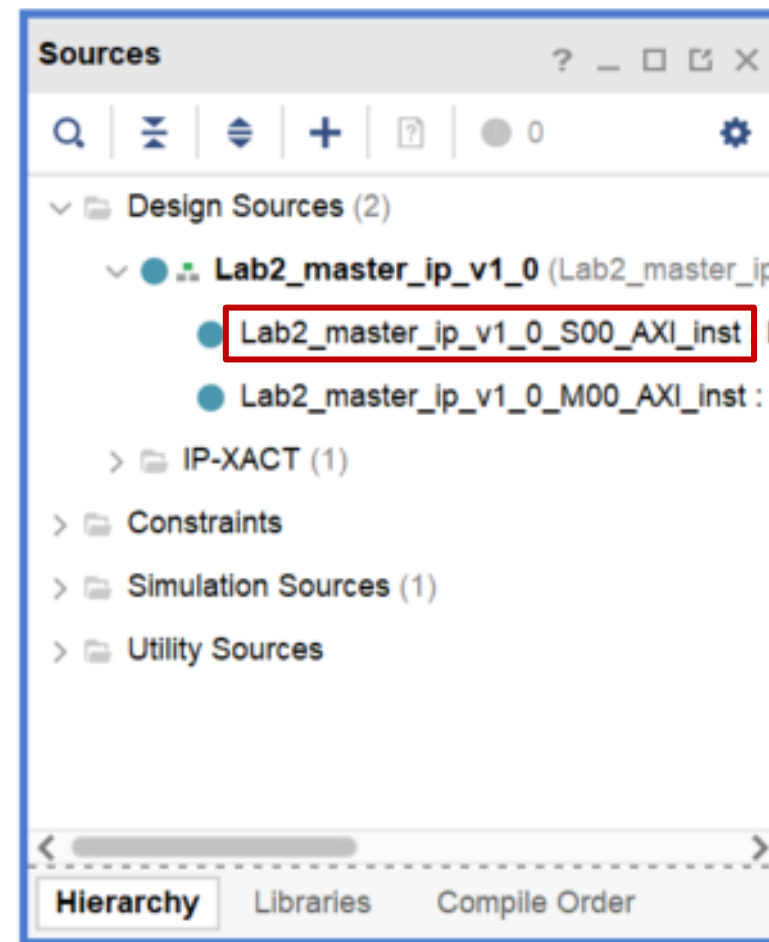
編輯master IP

▶ 1.確認開啟edit IP 的介面(編輯master IP)



編輯master IP

- ▶ 1.確認開啟edit IP 的介面(編輯master IP)
- ▶ 2.點開 `master_ip_v1_0_S00_AXI_inst.v`進行編輯



編輯master IP

- ▶ 1.確認開啟edit IP 的介面(編輯master IP)
- ▶ 2.點開master_ip_v1_0_S00_AXI_inst.v進行編輯
- ▶ 3.新增所需要的port腳

```
// Width of S_AXI data bus
parameter integer C_S_AXI_DATA_WIDTH    = 32,
// Width of S_AXI address bus
parameter integer C_S_AXI_ADDR_WIDTH    = 4
)
(
    // Users to add ports here
    output wire init_txn,
    input wire txn_done,
    input wire txn_error,|
    // User ports ends
    // Do not modify the ports beyond this line
```

編輯master IP

- ▶ 1.確認開啟edit IP 的介面(編輯master IP)
- ▶ 2.點開master_ip_v1_0_S00_AXI_inst.v進行編輯
- ▶ 3.新增所需要的port腳
- ▶ 4.新增CPU給的start訊號定義

```
399  
400 end  
401  
402 // Add user Logic here  
403 assign init_txn = slv_reg0[0:0];  
404 // User Logic ends  
405  
406 endmodule  
407
```

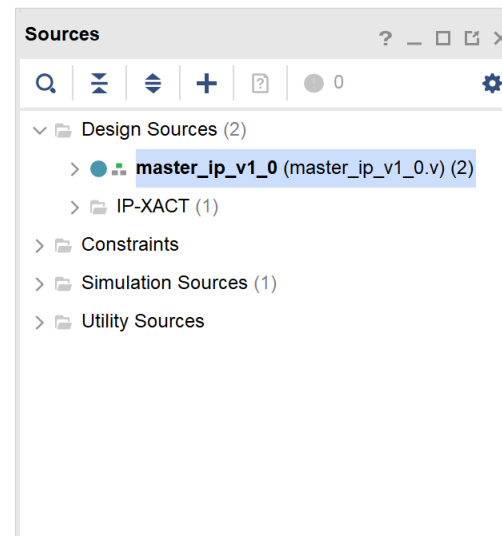
編輯master IP

- ▶ 1.確認開啟edit IP 的介面(編輯master IP)
- ▶ 2.點開master_ip_v1_0_S00_AXI_inst.v進行編輯
- ▶ 3.新增所需要的port腳
- ▶ 4.新增CPU給的start訊號定義
- ▶ 5.將slv_reg1，slv_reg2改為done與error訊號

```
369 // and the slave is ready to accept the read address.
370 assign slv_reg_rden = axi_arready & S_AXI_ARVALID & ~axi_rvalid;
371 always @(*)
372 begin
373     // Address decoding for reading registers
374     case ( axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] )
375         2'h0 : reg_data_out <= slv_reg0;
376         2'h1 : reg_data_out <= {{31{1'b0}},txn_done};
377         2'h2 : reg_data_out <= {{31{1'b0}},txn_error};
378         2'h3 : reg_data_out <= slv_reg3;
379         default : reg_data_out <= 0;
380     endcase
381 end
382
```

編輯master IP

- ▶ 1.確認開啟edit IP 的介面(編輯master IP)
- ▶ 2.點開master_ip_v1_0_S00_AXI_inst.v進行編輯
- ▶ 3.新增所需要的port腳
- ▶ 4.新增CPU給的start訊號定義
- ▶ 5.將slv_reg1 , slv_reg2改為done與error訊號
- ▶ 6.點開IP的Top Wrapper(master_ip_v1_0)，將我們在AXI-lite介面新增的port腳補上



```
106 );|
107 // Instantiation of Axi Bus Interface S00_AXI
108 master_ip_v1_0_S00_AXI # (
109     .C_S_AXI_DATA_WIDTH(C_S00_AXI_DATA_WIDTH),
110     .C_S_AXI_ADDR_WIDTH(C_S00_AXI_ADDR_WIDTH)
111 ) master_ip_v1_0_S00_AXI_inst (
112     .init_txn(m00_axi_init_axi_txn),
113     .txn_done(m00_axi_txn_done),
114     .txn_error(m00_axi_error),
115     .S_AXI_ACLK(s00_axi_aclk),
116     .S_AXI_ARESETN(s00_axi_aresetn),
117     .S_AXI_AWADDR(s00_axi_awaddr)
```

編輯master IP

- ▶ 1.確認開啟edit IP 的介面(編輯master IP)
- ▶ 2.點開master_ip_v1_0_S00_AXI_inst.v進行編輯
- ▶ 3.新增所需要的port腳
- ▶ 4.新增CPU給的start訊號定義
- ▶ 5.將slv_reg1，slv_reg2改為done與error訊號
- ▶ 6.點開IP的Top Wrapper(master_ip_v1_0)，將我們在AXI-lite介面新增的port腳補上
- ▶ 7.將Top Wrapper中，AXI Full interface與AXI Lite interface連接的I/O port刪除

```
57 |  
58 | // Ports of Axi Master Bus Interface M00_AXI  
59 | input wire m00_axi_init_axi_txn,  
60 | output wire m00_axi_txn_done,  
61 | output wire m00_axi_error,  
62 | input wire m00_axi_aclk,
```

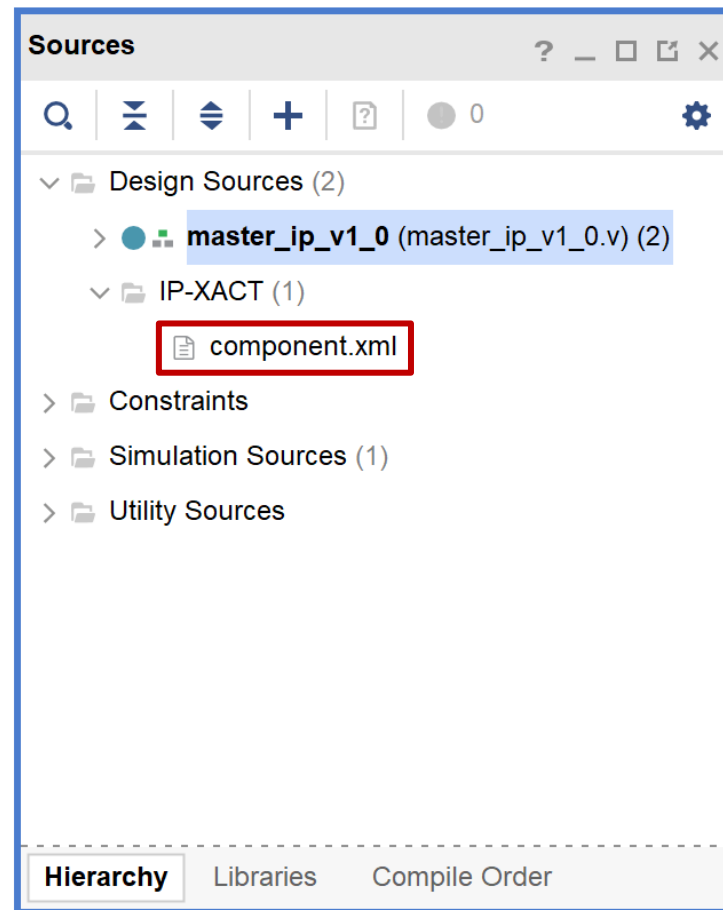
編輯master IP

- ▶ 1.確認開啟edit IP 的介面(編輯master IP)
- ▶ 2.點開master_ip_v1_0_S00_AXI_inst.v進行編輯
- ▶ 3.新增所需要的port腳
- ▶ 4.新增CPU給的start訊號定義
- ▶ 5.將slv_reg1，slv_reg2改為done與error訊號
- ▶ 6.點開IP的Top Wrapper(master_ip_v1_0)，將我們在AXI-lite介面新增的port腳補上
- ▶ 7.將Top Wrapper中，AXI Full interface與AXI Lite interface連接的I/O port刪除
- ▶ 8.將剛剛刪除的port腳改成wire

```
104 |  
105 |  
106 |  
107 |  
108 |  
109 |  
110 | // Instantiation of Axi Bus Interface S00_AXI  
111 | master_ip_v1_0_S00_AXI # (  
    |  
    | wire m00_axi_init_axi_txn;  
    | wire m00_axi_txn_done;  
    | wire m00_axi_error;  
    |  
    |
```

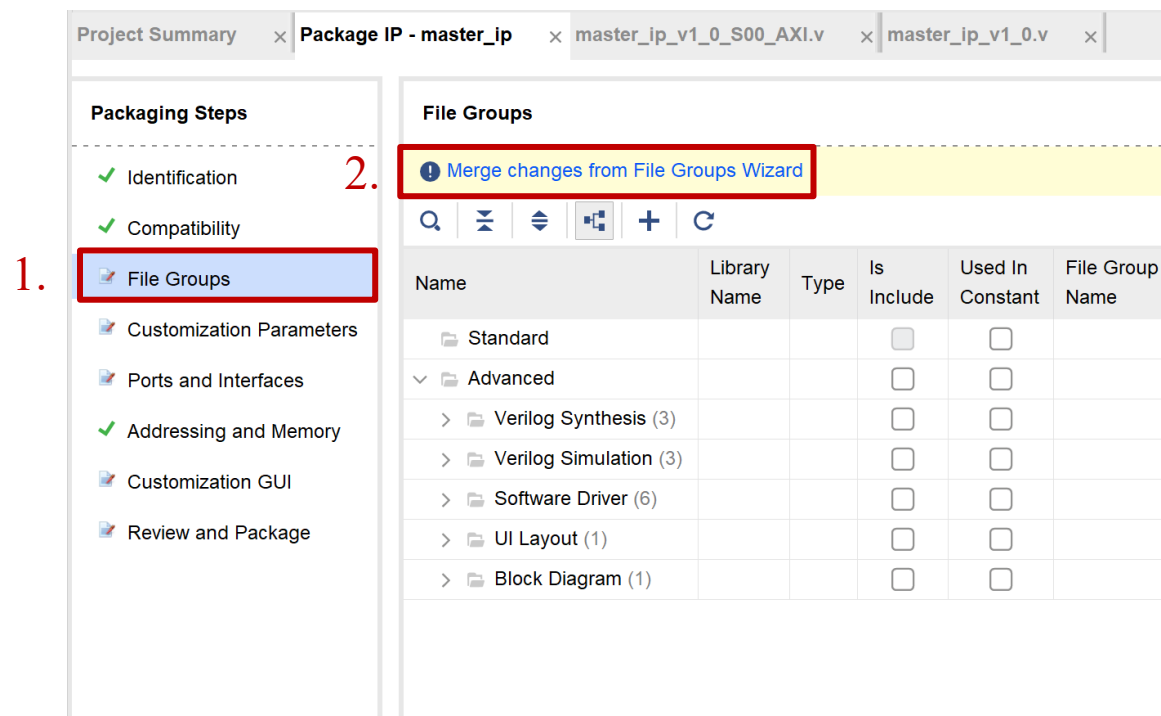
編輯master IP

- ▶ 9.確認上述修改完成後，在IP-XACT點選
component.xml



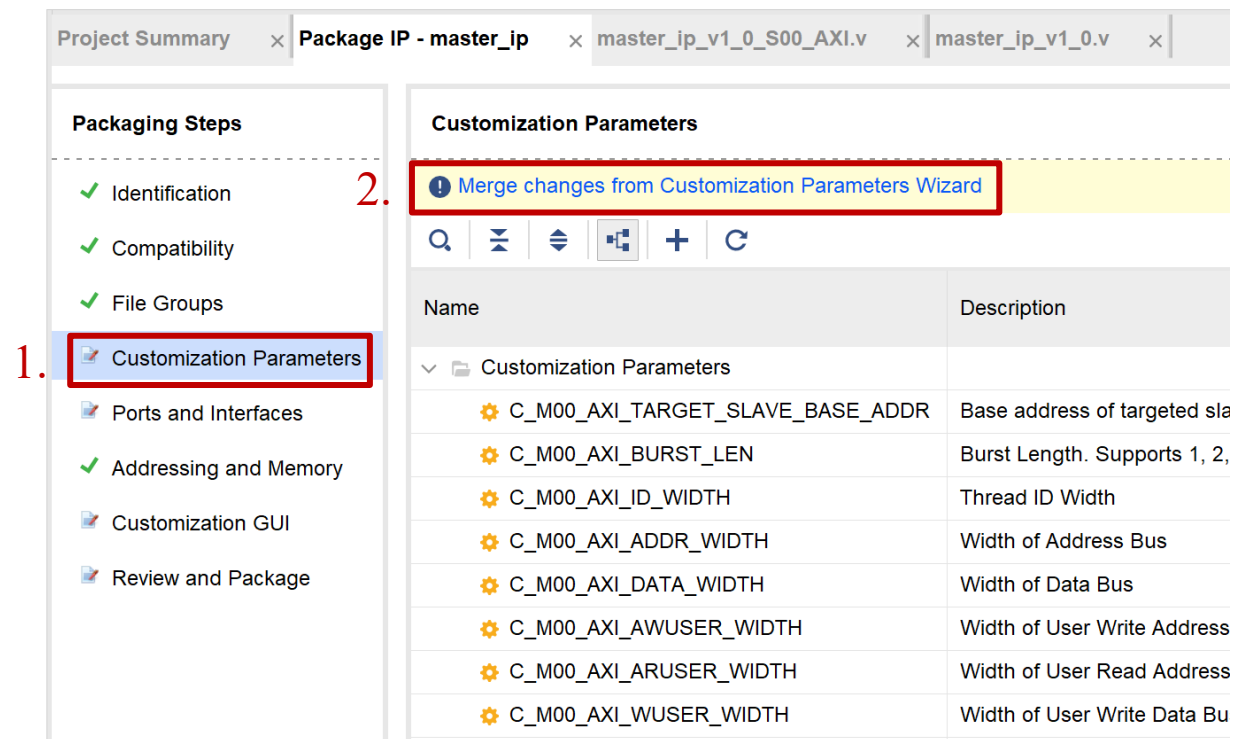
編輯master IP

- ▶ 9.確認上述修改完成後，在IP-XACT點選
component.xml
- ▶ 10.在File Groups中點選Merge changes from File
Groups Wizard



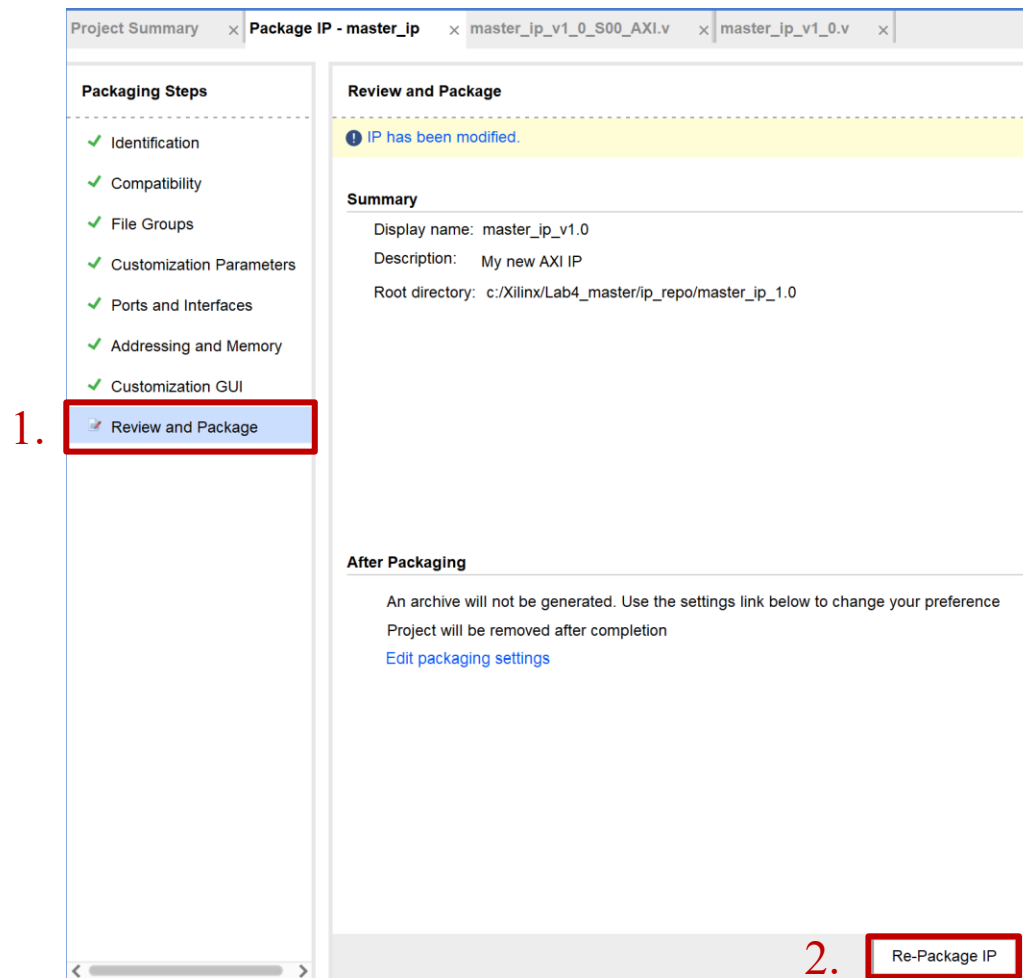
編輯master IP

- ▶ 9. 確認上述修改完成後，在IP-XACT點選 component.xml
- ▶ 10. 在File Groups中點選Merge changes from File Groups Wizard
- ▶ 11. 在Customization Parameters中點選Merge changes from Customization Parameters Wizard



編輯master IP

- ▶ 9.確認上述修改完成後，在IP-XACT點選component.xml
- ▶ 10.在File Groups中點選Merge changes from File Groups Wizard
- ▶ 11.在Customization Parameters中點選Merge changes from Customization Parameters Wizard
- ▶ 12.在Review and Package中點選Re-Package IP
並關掉視窗

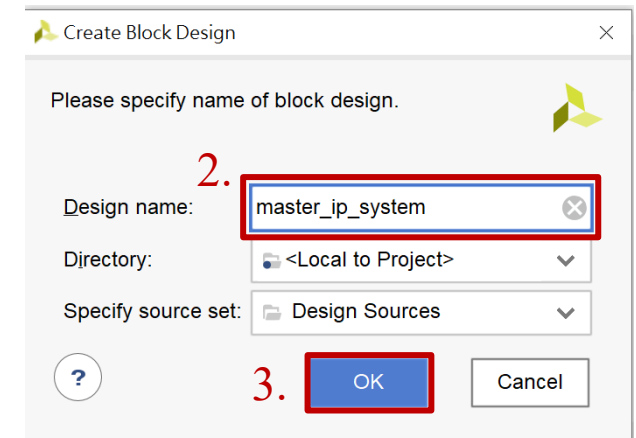
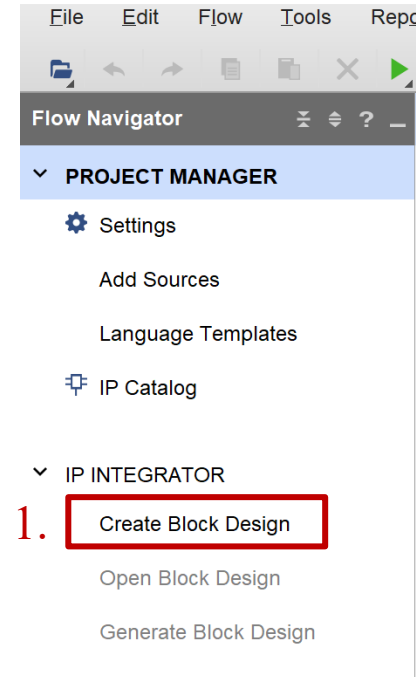


Part 2 : 建立 AXI master 並與 CPU 進行溝通

- ▶ Overview of the master IP
- ▶ 建立 master IP
- ▶ 編輯 IP 內容
- ▶ 建立 Hardware IP
- ▶ SDK 操作

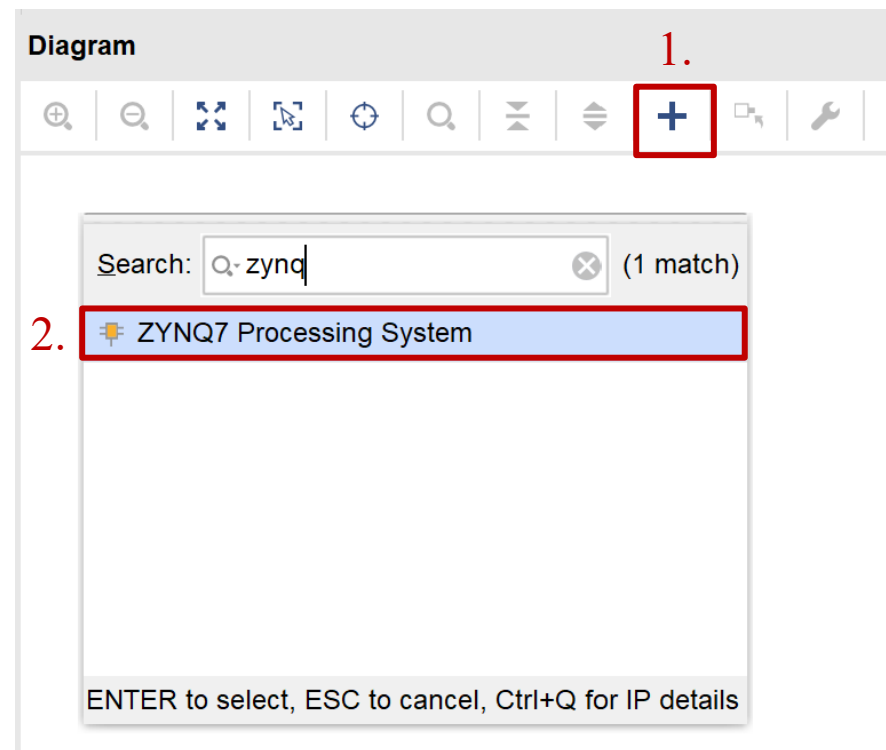
建立Hardware IP

- ▶ 1.點選Create Block Design →OK



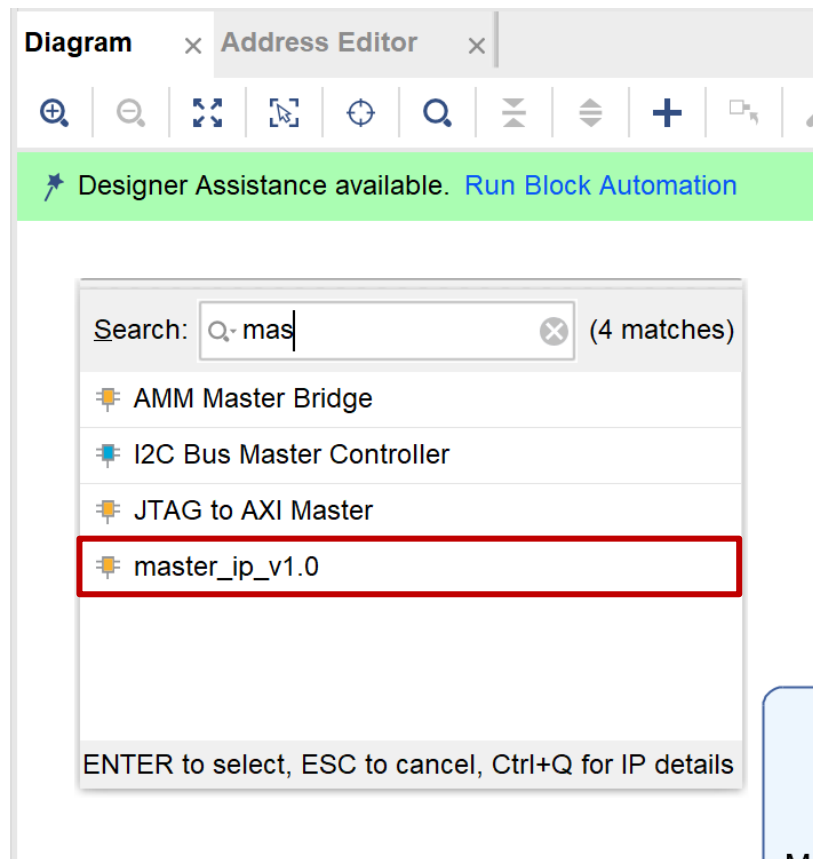
建立Hardware IP

- ▶ 1.點選Create Block Design →OK
- ▶ 2.點選 **+** 並將ZYNQ加入block design



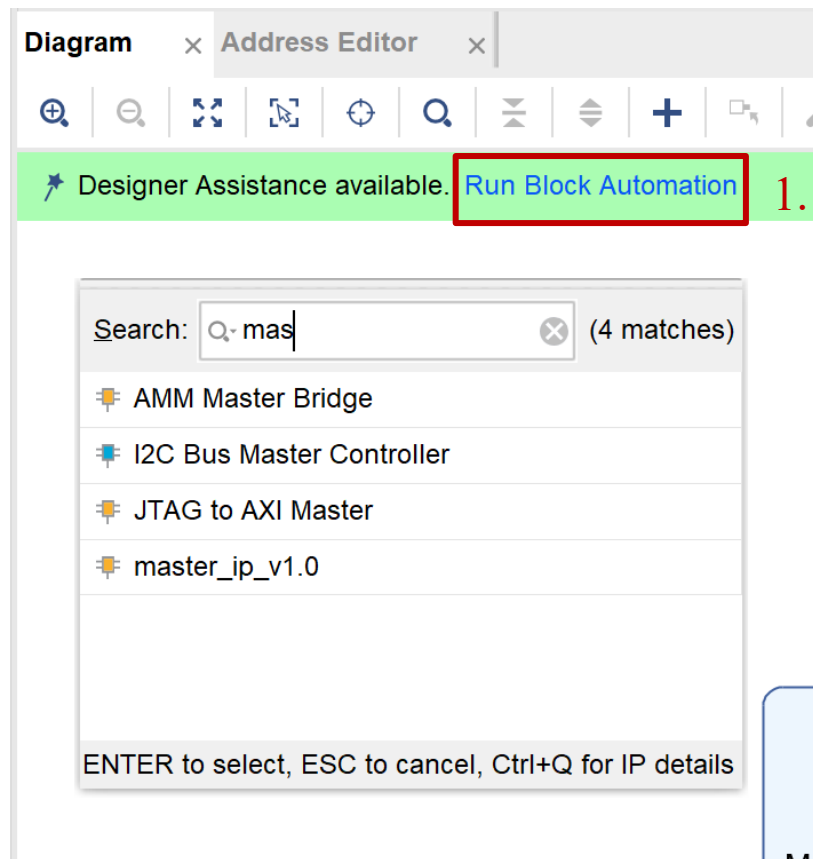
建立Hardware IP

- ▶ 1.點選Create Block Design →OK
- ▶ 2.點選 **+** 並將ZYNQ加入block design
- ▶ 3.點選 **+** 加入剛剛包好的IP



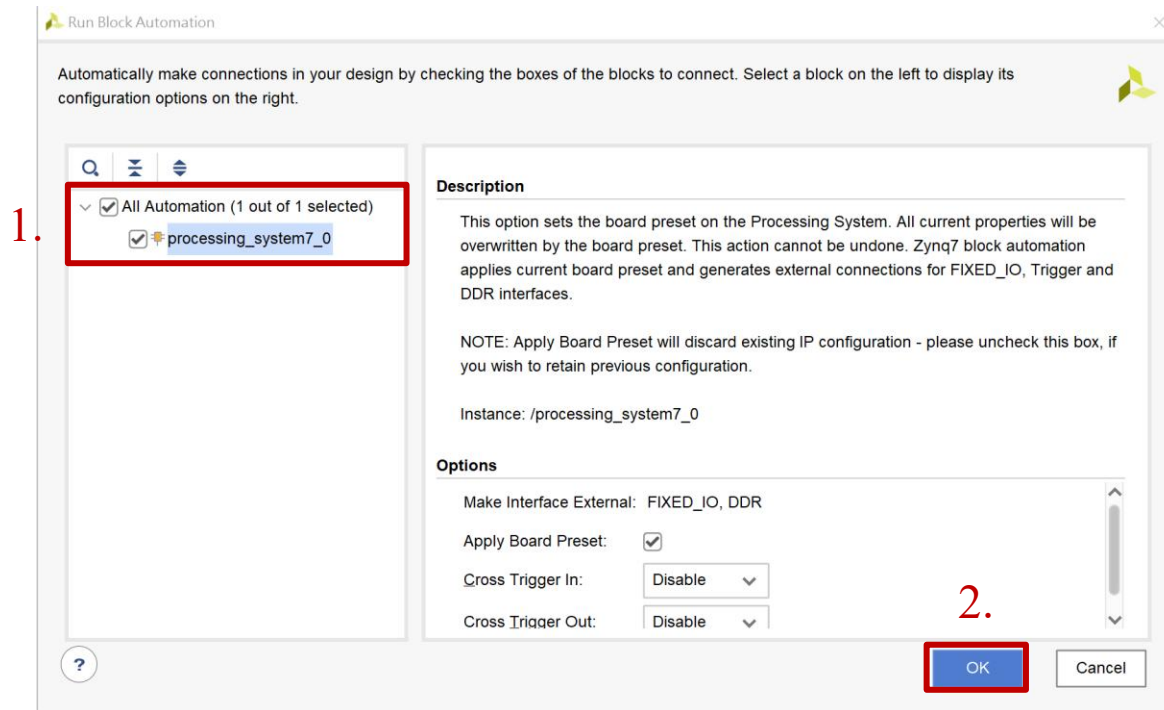
建立Hardware IP

- ▶ 1.點選Create Block Design →OK
- ▶ 2.點選 **+** 並將ZYNQ加入block design
- ▶ 3.點選 **+** 加入剛剛包好的IP
- ▶ 4.點選Run Block Automation



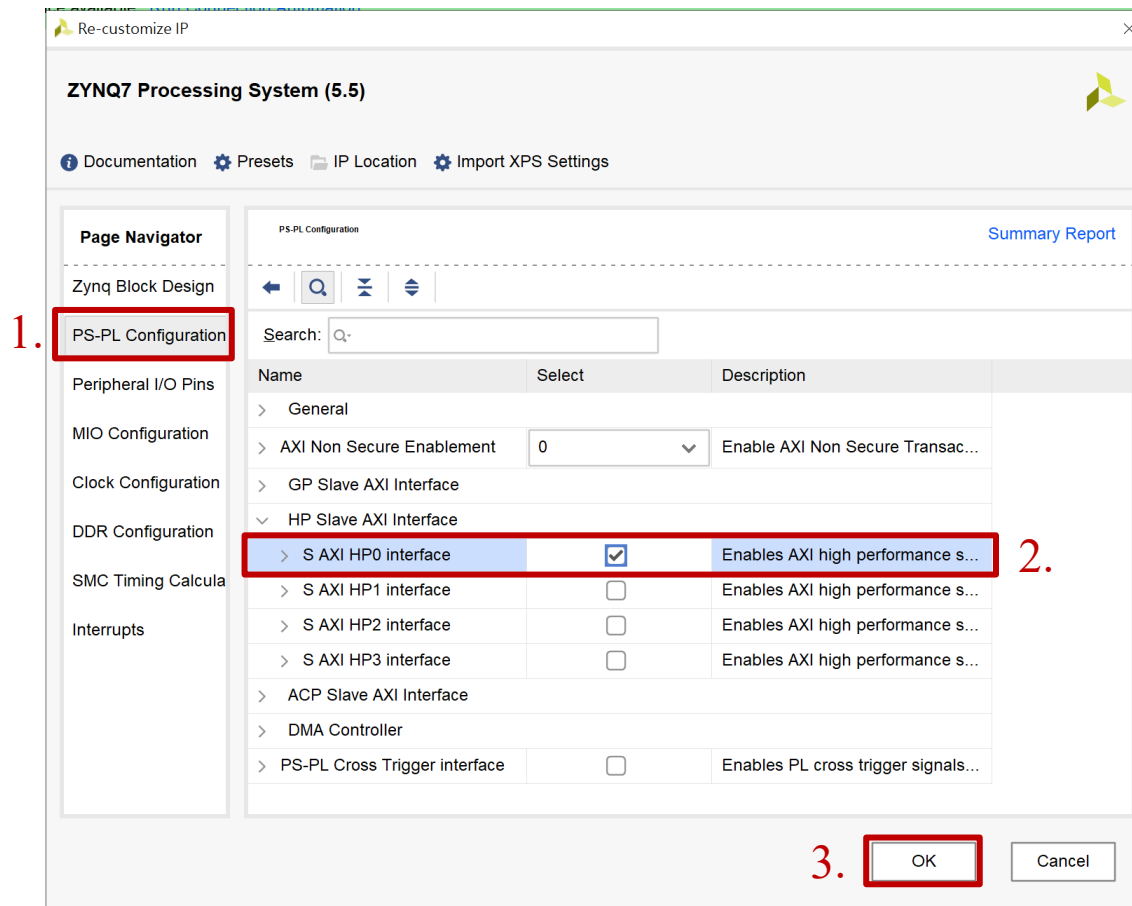
建立Hardware IP

- ▶ 1.點選Create Block Design →OK
- ▶ 2.點選 **+** 並將ZYNQ加入block design
- ▶ 3.點選 **+** 加入剛剛包好的IP
- ▶ 4.點選Run Block Automation
- ▶ 5.確認勾選 → OK



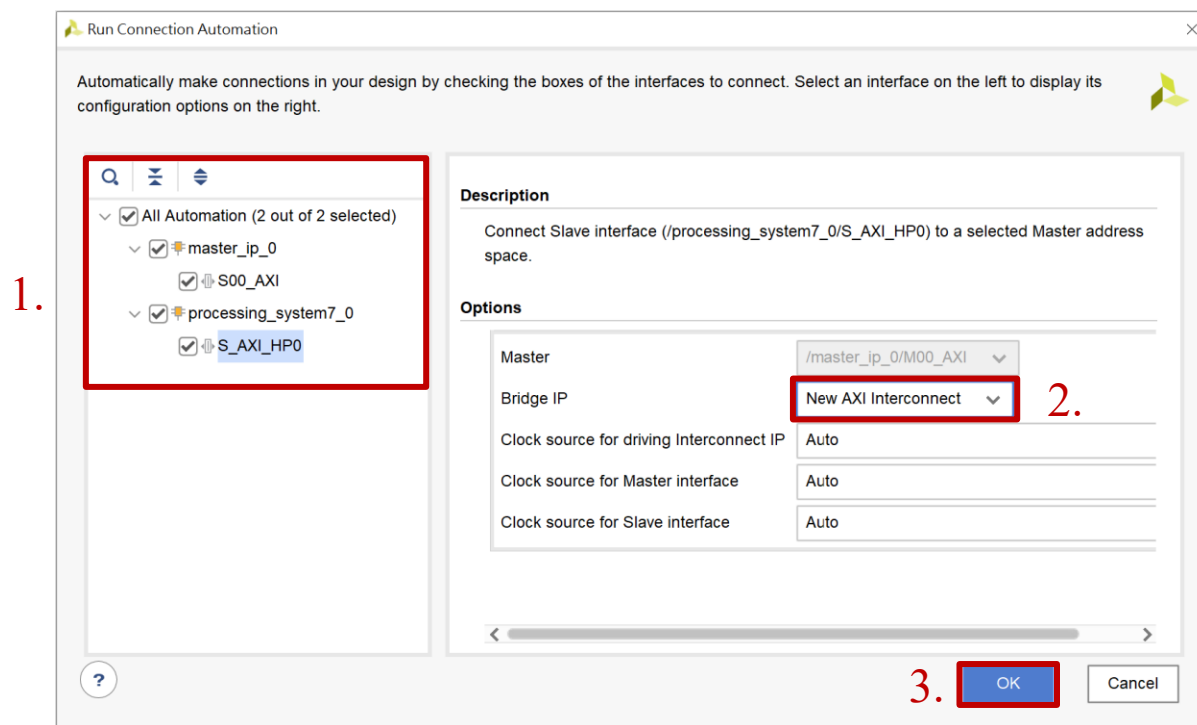
建立Hardware IP

- ▶ 1.點選Create Block Design →OK
- ▶ 2.點選 **+** 並將ZYNQ加入block design
- ▶ 3.點選 **+** 加入剛剛包好的IP
- ▶ 4.點選Run Block Automation
- ▶ 5.確認勾選 → OK
- ▶ 6.double click ZYNQ → PS-PL Configurations →
勾選 S AXI HP0 interface → OK



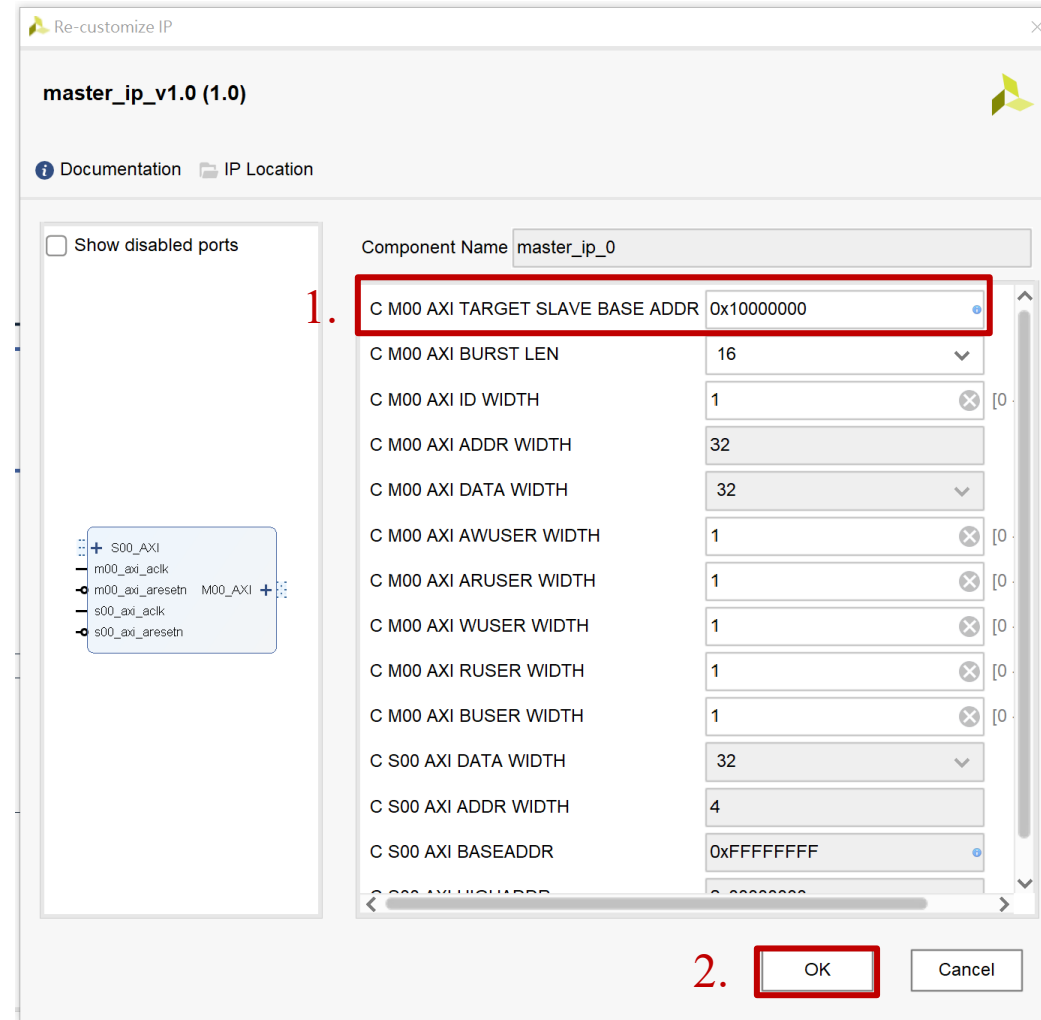
建立Hardware IP

- ▶ 1.點選Create Block Design →OK
- ▶ 2.點選 **+** 並將ZYNQ加入block design
- ▶ 3.點選 **+** 加入剛剛包好的IP
- ▶ 4.點選Run Block Automation
- ▶ 5.確認勾選 → OK
- ▶ 6.double click ZYNQ → PS-PL Configurations →
勾選 S AXI HP0 interface → OK
- ▶ 7.點選Run Connection Automation →全部勾選
→S_AXI_HP0的Bridge IP 選擇New AXI
Interconnect → OK



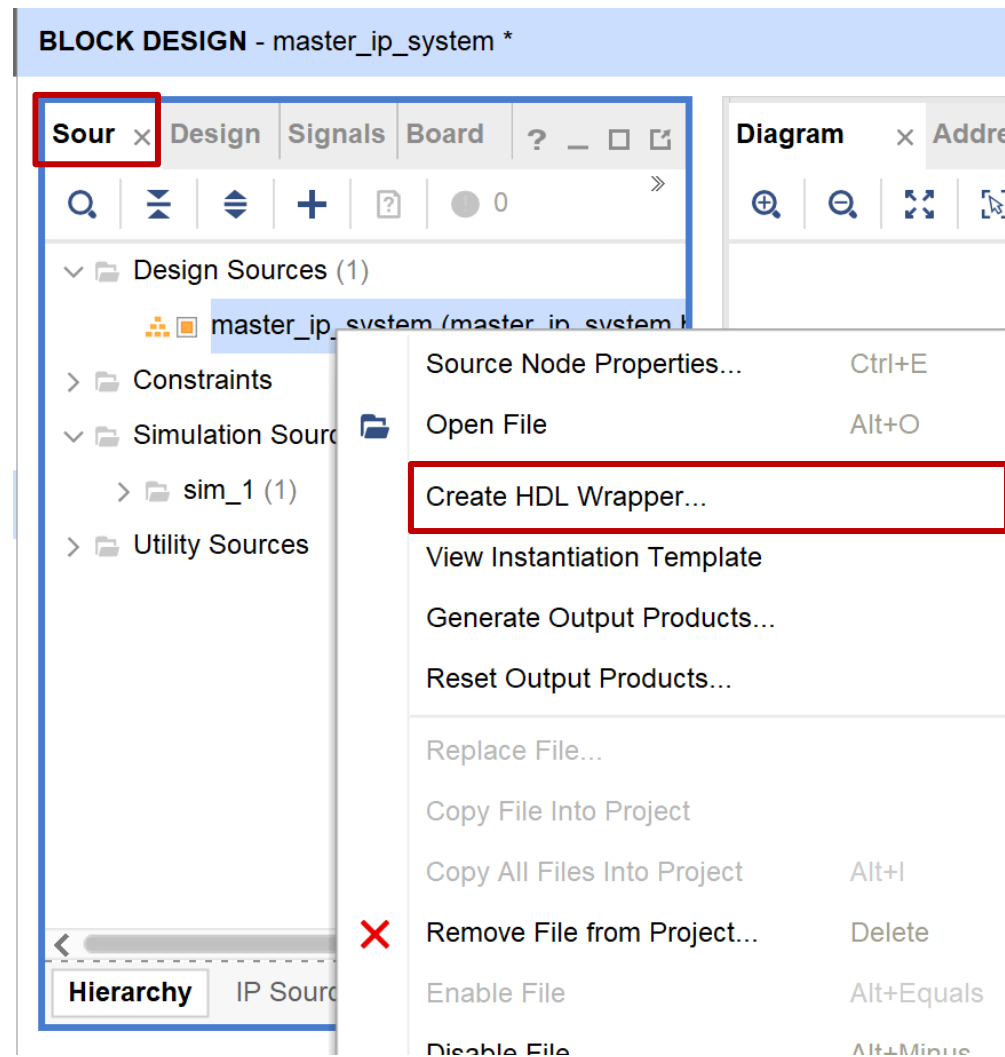
建立Hardware IP

- ▶ 8.double click我們的IP並將
C_M00_AXI_TARGET_SLAVE_BASE_ADDR
的數值改為0x10000000 → OK



建立Hardware IP

- ▶ 8.double click我們的IP並將
C_M00_AXI_TARGET_SLAVE_BASE_ADDR
的數值改為0x10000000 → OK
- ▶ 9.點選Create HDL Wrapper包裝硬體



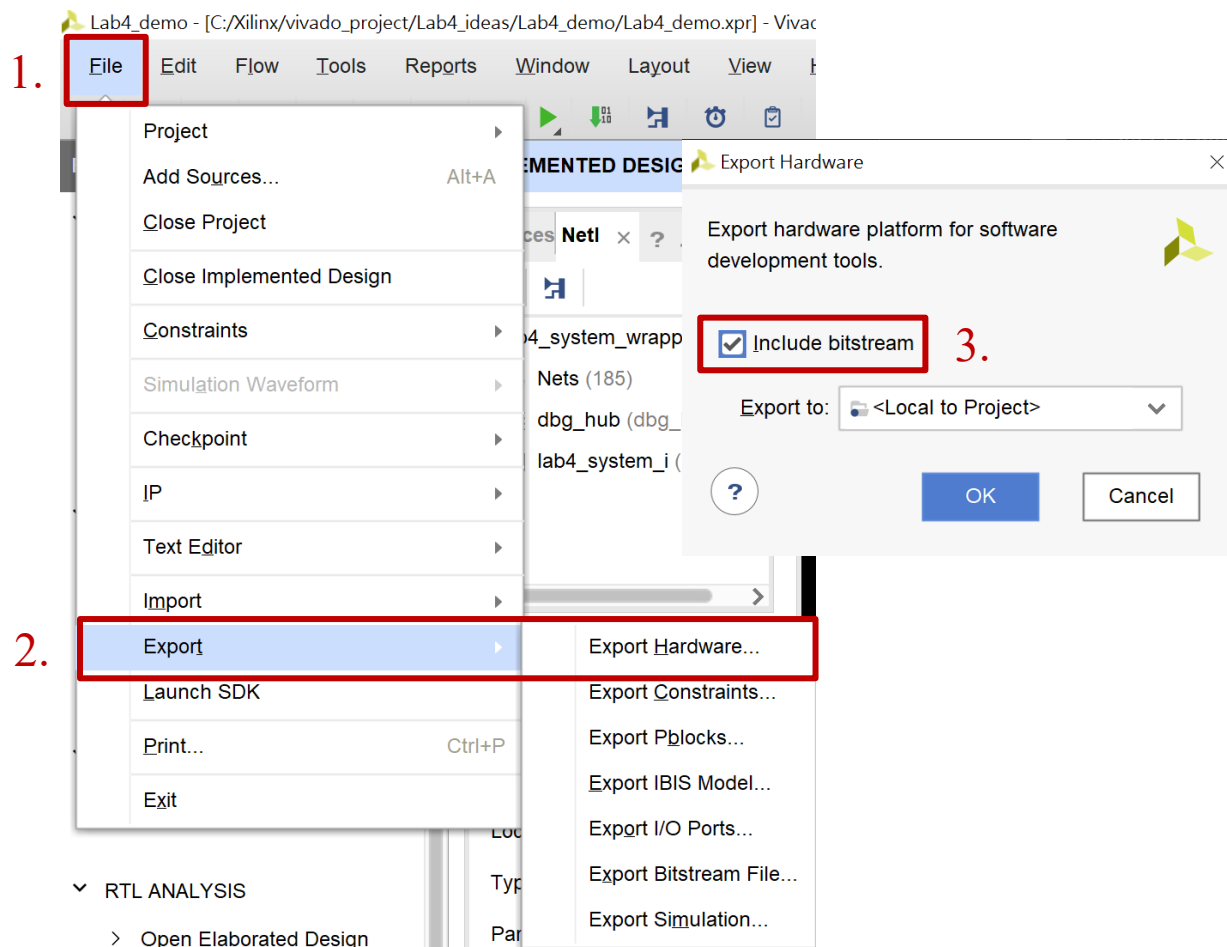
建立Hardware IP

- ▶ 8.double click我們的IP並將
C_M00_AXI_TARGET_SLAVE_BASE_ADDR
的數值改為0x10000000 → OK
- ▶ 9.點選Create HDL Wrapper包裝硬體
- ▶ 10.點選Generate Bitstream → OK

- ▼ SYNTHESIS
 - ▶ Run Synthesis
 - > Open Synthesized Design
- ▼ IMPLEMENTATION
 - ▶ Run Implementation
 - > Open Implemented Design
- ▼ PROGRAM AND DEBUG
 -  Generate Bitstream
 - > Open Hardware Manager

建立Hardware IP

- ▶ 8.double click我們的IP並將
C_M00_AXI_TARGET_SLAVE_BASE_ADDR
的數值改為0x10000000 → OK
- ▶ 9.點選Create HDL Wrapper包裝硬體
- ▶ 10.點選Generate Bitstream → OK
- ▶ 11.File → Export → Export Hardware → Include Bitstream

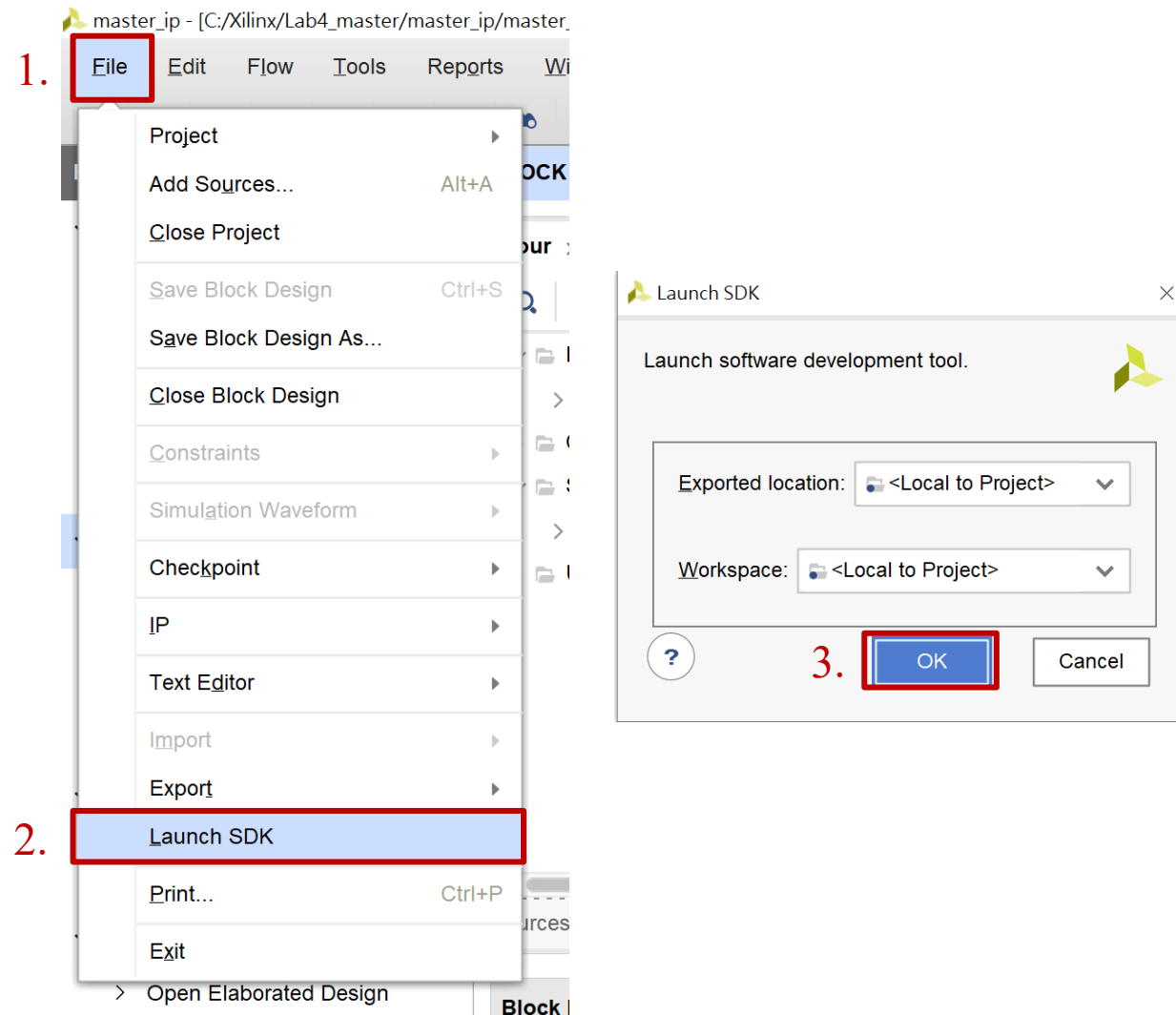


Part 2 : 建立 AXI master 並與 CPU 進行溝通

- ▶ Overview of the master IP
- ▶ 建立 master IP
- ▶ 編輯 IP 內容
- ▶ 建立 Hardware IP
- ▶ SDK 操作

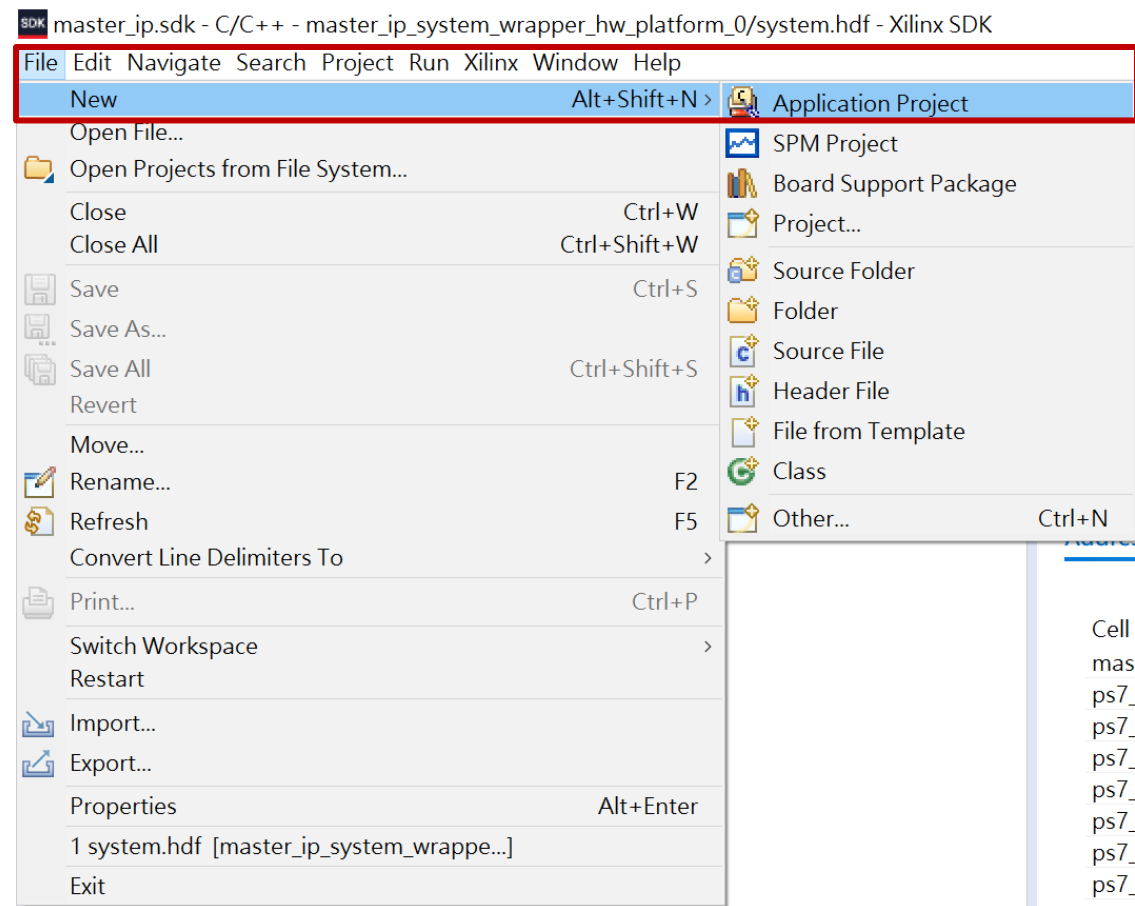
SDK操作

- ▶ 1.File → Launch SDK → OK



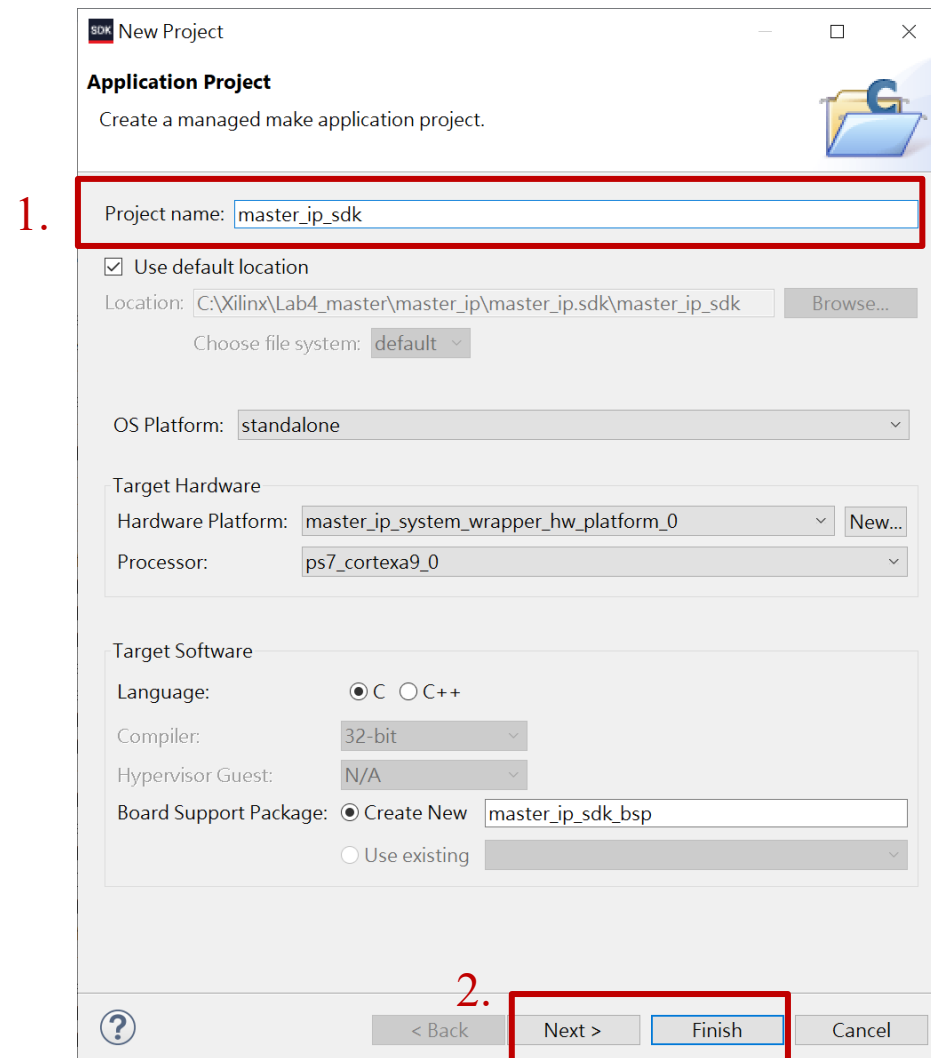
SDK操作

- ▶ 1.File → Launch SDK → OK
- ▶ 2.File → New → Application



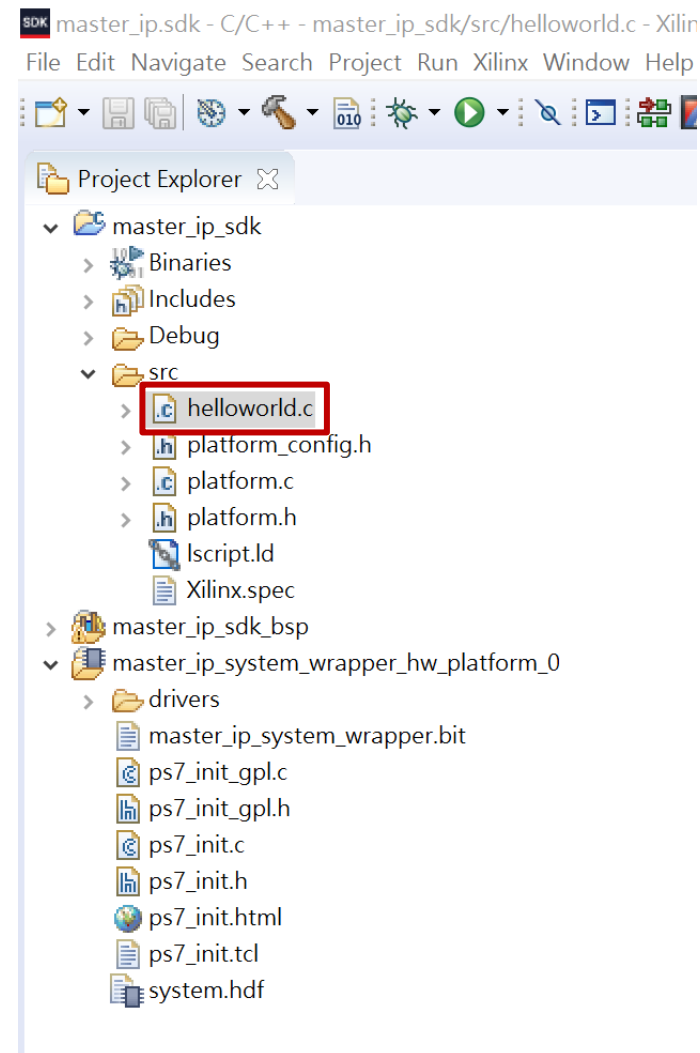
SDK操作

- ▶ 1.File → Launch SDK → OK
- ▶ 2.File → New → Application
- ▶ 3.建立Project → Next → Hello World → Finish



SDK操作

- ▶ 1.File → Launch SDK → OK
- ▶ 2.File → New → Application
- ▶ 3.建立Project → Next → Hello World → Finish
- ▶ 4.點選project → src → helloworld.c 開始編輯



撰寫SDK程式

- ▶ 1.定義指向IP暫存器的記憶體位址的指標
- ▶ 2.定義指向DRAM address的指標
- ▶ 3.CPU發送start訊號給IP
- ▶ 4.CPU檢查start，done，以及error訊號
- ▶ 5.CPU讀取寫入DRAM的資料內容

```
#include "platform.h"
#include "xil_printf.h"
#include "xbasic_types.h"
#include "xparameters.h"
#include <stdio.h>

int main() {

    init_platform();

    printf("== burst test == \n\n");

    // pointer to address the AXI4-Lite slave
    volatile int *slave = (int*)0x43c00000; 1.

    // pointer to memory address 0x10000000
    volatile int *data_p = (int*)0x10000000; 2.

    // check status
    printf("INIT_TXN\t0x%08x\n\n", *(slave+0));
    printf("TXN_DONE\t0x%08x\n\n", *(slave+1));
    printf("ERROR\t\t0x%08x\n\n", *(slave+2));
    printf("\n");

    // start AXI4 write/read burst transaction
    *(slave+0) = 0x00000001;
    *(slave+0) = 0x00000000; 3.

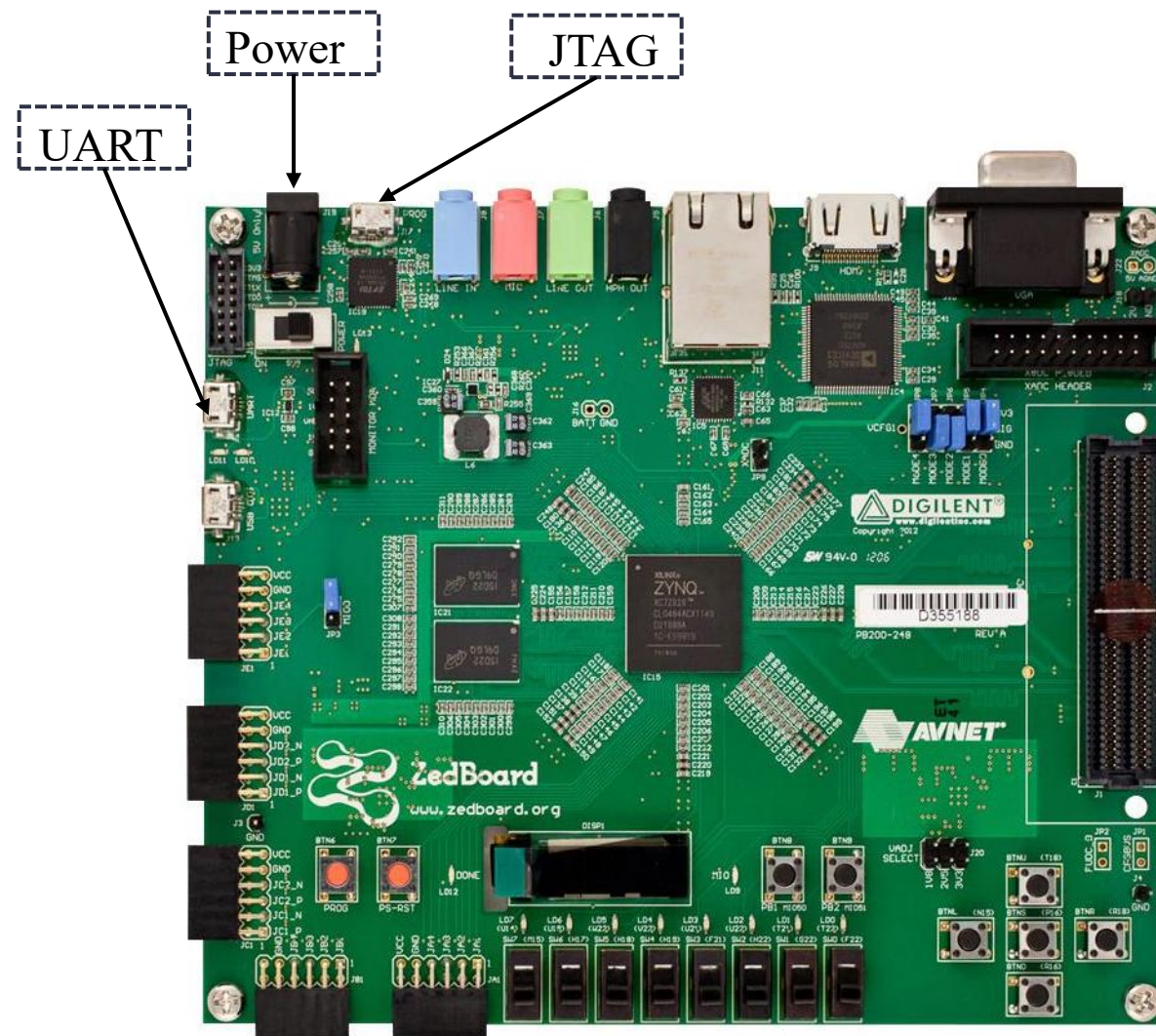
    // check status
    printf("INIT_TXN\t0x%08x\n\n", *(slave+0));
    printf("TXN_DONE\t0x%08x\n\n", *(slave+1));
    printf("ERROR\t\t0x%08x\n\n", *(slave+2));
    printf("\n\n"); 4.

    // print memory content
    int i;
    for(i = 0; i < 16; i++) {
        printf("DATA+%d\t\t0x%08x\n\n", i, *(data_p+i)); 5.
    }

    cleanup_platform();
    return 0;
}
```

燒錄SDK並執行結果

- ▶ 1.確認C程式完成後，將FPGA接好開啟電源



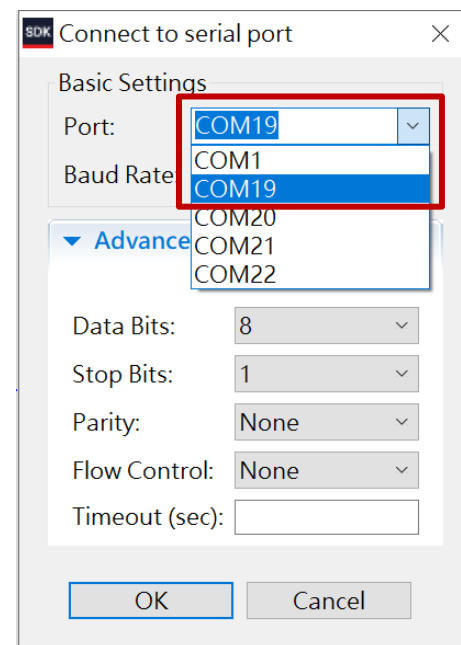
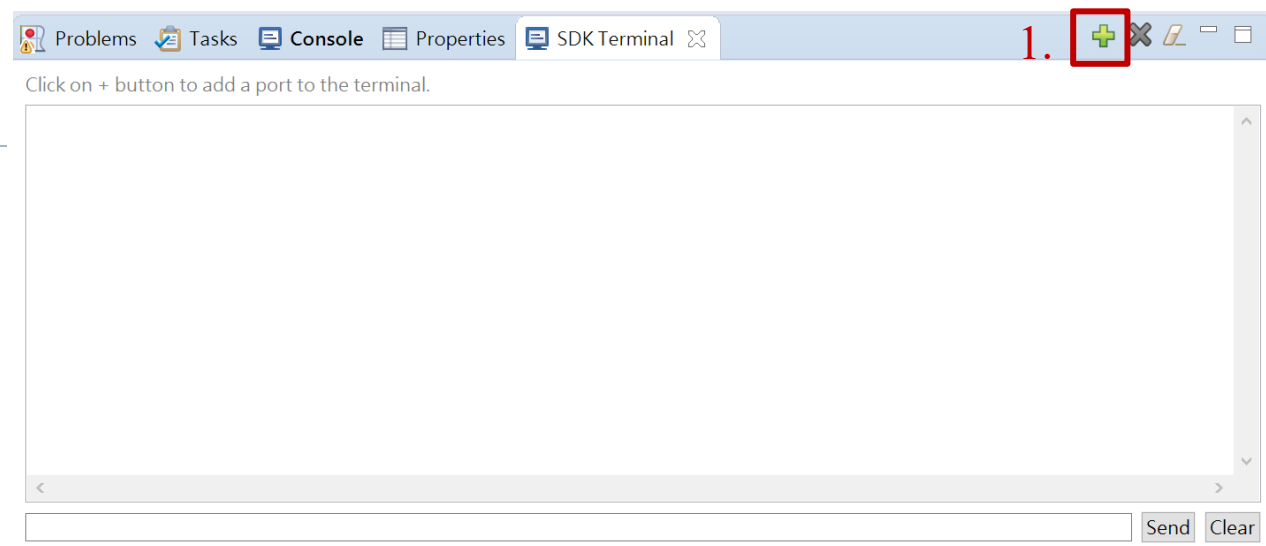
燒錄SDK並執行結果

- ▶ 1.確認C程式完成後，將FPGA接好開啟電源
- ▶ 2.裝置管理員中尋找COM腳



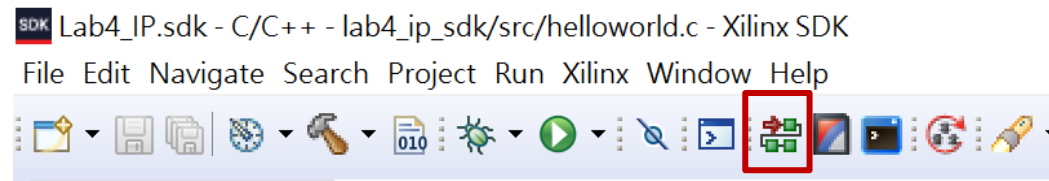
燒錄SDK並執行結果

- ▶ 1. 確認C程式完成後，將FPGA接好開啟電源
- ▶ 2. 裝置管理員中尋找COM腳
- ▶ 3. 點選下面SDK Terminal  按鈕，選取COM腳



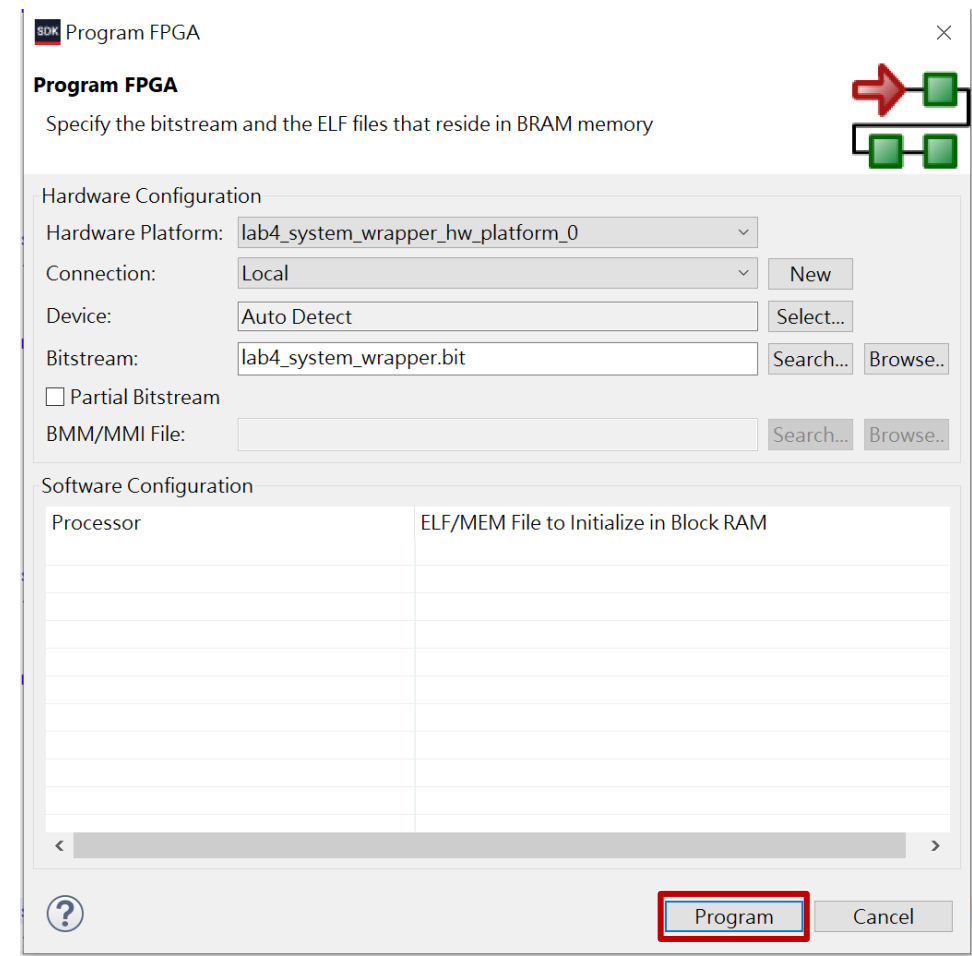
燒錄SDK並執行結果

- ▶ 1.確認C程式完成後，將FPGA接好開啟電源
- ▶ 2.裝置管理員中尋找COM腳
- ▶ 3.點選下面SDK Terminal  按鈕，選取COM腳
- ▶ 4.點選  開啟Program介面



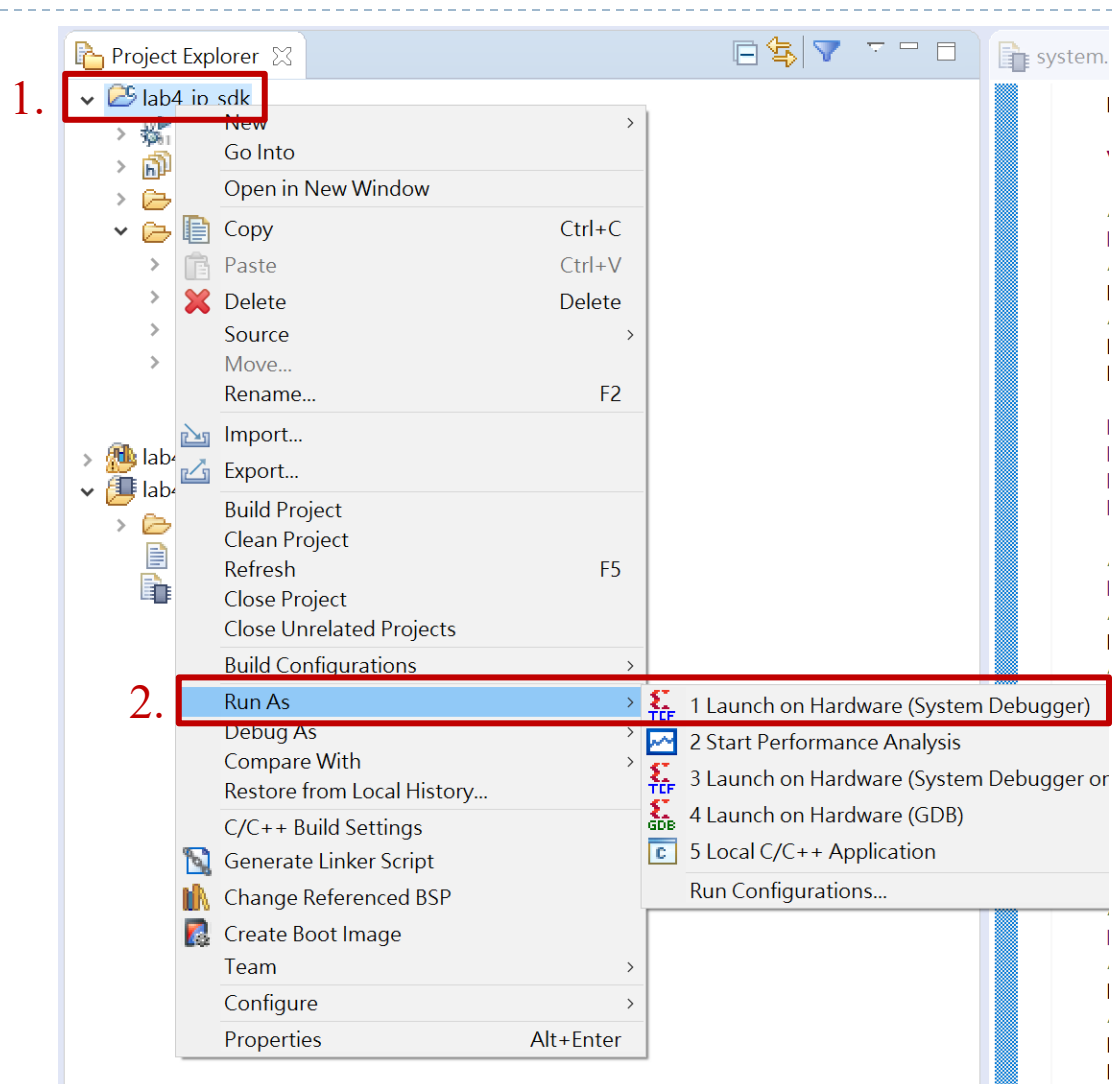
燒錄SDK並執行結果

- ▶ 1.確認C程式完成後，將FPGA接好開啟電源
- ▶ 2.裝置管理員中尋找COM腳
- ▶ 3.點選下面SDK Terminal  按鈕，選取COM腳
- ▶ 4.點選  開啟Program介面
- ▶ 5.點選Program並等待



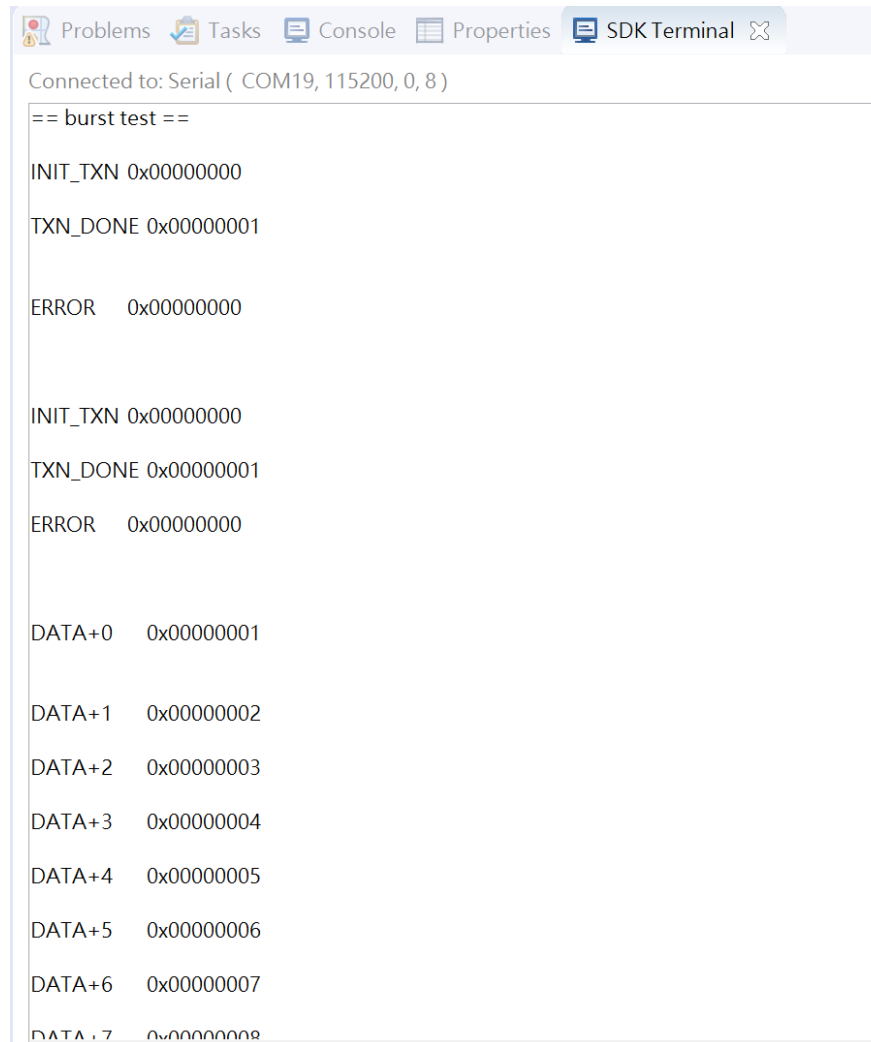
燒錄SDK並執行結果

- ▶ 1.確認C程式完成後，將FPGA接好開啟電源
- ▶ 2.裝置管理員中尋找COM腳
- ▶ 3.點選下面SDK Terminal  按鈕，選取COM腳
- ▶ 4.點選  開啟Program介面
- ▶ 5.點選Program並等待
- ▶ 6.點選Project名稱右鍵 → Run As → Launch On Hardware(System Debugger)



燒錄SDK並執行結果

- ▶ 1.確認C程式完成後，將FPGA接好開啟電源
- ▶ 2.裝置管理員中尋找COM腳
- ▶ 3.點選下面SDK Terminal  按鈕，選取COM腳
- ▶ 4.點選  開啟Program介面
- ▶ 5.點選Program並等待
- ▶ 6.點選Project名稱右鍵 → Run As → Launch On Hardware(System Debugger)
- ▶ 7.確認SDK Terminal跑出的結果



```
Problems Tasks Console Properties SDK Terminal
Connected to: Serial ( COM19, 115200, 0, 8 )

== burst test ==

INIT_TXN 0x00000000
TXN_DONE 0x00000001

ERROR 0x00000000

INIT_TXN 0x00000000
TXN_DONE 0x00000001
ERROR 0x00000000

DATA+0 0x00000001
DATA+1 0x00000002
DATA+2 0x00000003
DATA+3 0x00000004
DATA+4 0x00000005
DATA+5 0x00000006
DATA+6 0x00000007
DATA+7 0x00000008
```

Part 3：練習題

- ▶ 練習：用 CPU 溝通硬體控制 LED 燈，給定一個 counter 訊號，可以使 ZedBoard 上的 LED 燈循環亮滅。
 - ▶ (1) 建立 slave IP
 - ▶ (2) 寫 verilog code 使 CPU 傳來的 counter 訊號用來控制 LED 亮滅的頻率 (注意：需要將 LED 燈的 wire 接到 IP 的 top module)
 - ▶ (3) Package IP
 - ▶ (4) 照前面的步驟到 generate bitstream，並記得透過 Lab1 的方式加入 constrain 檔 (助教會提供 template)
 - ▶ (5) SDK 寫一個 counter 訊號
 - ▶ (6) 執行 SDK 查看功能是否正常

Part 3 : 練習題

- ▶ 練習：以下提供 IP module 的 port 腳定義 (只是參考，同學可自行設計)

Ports Name	Input/Output	Bit width	Description
clk	Input	1	Clock signal of IP.
reset	Input	1	Synchronous active low reset signal.
counter_signal	Input	32	Counter to determine the frequency of LED.
LED	Output	8	Output signal to control LED light.



Thanks for Listening